NeuraEdge CIFAR-10 Neuromorphic Accelerator: Full Pipeline and Line-by-Line Explainer

Generated from repository source and measured Vivado/XSim artifacts

July 10, 2026

1 What Was Built

This document describes the implemented CIFAR-10 accelerator path. The trained Spikingformer backbone is used to export 384-dimensional fixed-point features and a trained classifier head. The FPGA accelerator stores one feature vector, applies a measured low-magnitude feature activity gate, computes ten fixed-point class scores with an on-chip weight memory and one DSP MAC datapath, returns the predicted label, and includes a margin-gated on-chip learning path that updates the true-label and predicted-label classifier rows in FPGA memory.

2 Measured Verification Summary

- Full inference XSim run: 10,000 CIFAR-10 samples, 95.51% top-1 accuracy, exact score match against the fixed-point Python reference.
- Feature activity gate: abs(feature) ≤ 2048 is treated as zero before inference and learning; the exported test set skips 57.81% of feature-MAC opportunities.
- Full on-chip-learning XSim run: 10,000 samples, 25 RTL weight/bias updates, zero score mismatches, zero prediction mismatches, zero update-flag mismatches, and zero update-count mismatches against the independent Python online-learning golden model.
- Vivado implementation target: PYNQ Zynq-7020, xc7z020clg400-1, 100 MHz. Routed timing met with WNS 0.336 ns and WHS 0.100 ns.
- Resource result: 1043 LUTs, 988 FFs, 6 RAMB36E1 blocks, and 1 DSP48E1.
- Vectorless Vivado power: 0.123 W total, 0.018 W dynamic, 0.105 W static.
- XSim-SAIF annotated Vivado power: inference 0.116 W total / 0.011 W dynamic; learning 0.116 W total / 0.011 W dynamic. Vivado reports medium confidence because SAIF annotates the wrapper/core activity while remaining unmatched nets use probabilistic propagation.

3 Research Context And Comparison

The model side follows the Spikingformer family of spike-driven transformer SNNs, whose reported CIFAR-10 accuracy is about 95.81–95.95% in the cited paper/repository. The local fixed-point exported checkpoint reaches 95.51% after mapping into this routed hardware head with the measured feature gate. Recent SNN-transformer targets to beat include Max-Former at 97.04% on CIFAR-10 and alpha-XNOR spiking self-attention at 96.15% for the reported multi-stage ViT setting. The implemented hardware contribution here is the verified FPGA path: local feature and weight memories, exact fixed-point RTL/golden agreement, activity-gated feature silence, and on-chip classifier learning.

- Spikingformer paper/repository: `https://arxiv.org/html/2304.11954v5`, `https://github.com/TheBrainLab/Spikingformer`.
- Max-Former high-frequency spiking transformer reference: `https://arxiv.org/html/2505.18608v4`.
- Alpha-XNOR spiking self-attention reference: `https://openaccess.thecvf.com/content/CVPR2025/papers/Xiao_Rethinking_Spiking_Self-Attention_Mechanism_Implementing_a-XNOR_Similarity_Calculation_in_Spiking_CVPR_2025_paper.pdf`.

- TrueNorth reference: https://research.ibm.com/publications/truenorth-design-and-tool-flow-of-a-65-mw-1-mil
- Loihi on-chip learning reference: https://redwood.berkeley.edu/wp-content/uploads/2021/08/Davies2018.pdf.

4 End-to-End Dataflow

1. Python loads the trained Spikingformer checkpoint and CIFAR-10 data, runs the trained network, and exports fixed-point feature, label, weight, and bias memories.
2. XSim streams each 384-word feature vector into the RTL through the same handshake the PYNQ AXI-stream wrapper uses.
3. The RTL stores the feature vector locally, zeroes features whose magnitude is below the selected gate threshold, loops over the ten classes, reads each class row from BRAM, multiplies effective feature and weight in a DSP, accumulates a 64-bit score, and chooses argmax.
4. If training is enabled and the prediction is wrong but close enough by margin, the RTL latches the label, reads the true and predicted class rows, computes signed shifted deltas from the same effective feature path, saturates int16 weight updates, writes updated weights, and updates the two class biases.
5. Python validators independently replay the same fixed-point inference and online-learning rule from the exported memories, then compare every RTL score, prediction, update flag, and update count.
6. Vivado synthesizes and routes the PYNQ AXI-stream top, producing timing, utilization, power, and checkpoint artifacts.

5 Line-by-Line Source Explanation

5.1 cifar10_neuromorphic\rtl\cifar10_spikingformer_head_accelerator.sv

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `` `timescale 1ns / 1ps ``

This simulator directive sets the time unit and precision used by testbenches and waveform timestamps. It keeps delays such as clock half-periods numerically meaningful in XSim.

Line 2 `` `default_nettype none ``

This disables implicit one-bit wires. It prevents spelling mistakes from silently creating unwanted hardware nets.

Line 3 `` `include "rtl/cifar10_spikingformer_head_params.svh" ``

This includes shared compile-time constants such as feature dimensions and fixed-point widths. The RTL and testbenches therefore agree on the same trained-model shape.

Line 4 `` `ifndef CIFAR10_SF_HEAD_INFER_SKIP_ABS_THRESH ``

This line controls the feature-activity gate. When the absolute fixed-point feature is at or below this threshold, the RTL treats it as zero so the exported Spikingformer head ignores small noisy activations and records a measurable MAC-skip opportunity.

Line 5 `` `define CIFAR10_SF_HEAD_INFER_SKIP_ABS_THRESH 2048 ``

This line controls the feature-activity gate. When the absolute fixed-point feature is at or below this threshold, the RTL treats it as zero so the exported Spikingformer head ignores small noisy activations and records a measurable MAC-skip opportunity.

Line 6 `` `endif ``

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 7 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 8 `module cifar10_spikingformer_head_accelerator #(`

This begins a hardware module, which is the FPGA equivalent of a reusable circuit block. Everything until the matching endmodule describes ports, storage, combinational equations, and clocked behavior for this block.

Line 9 `parameter int FEATURE_DIM = `CIFAR10_SF_HEAD_FEATURE_DIM,`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 10 `parameter int NUM_CLASSES = `CIFAR10_SF_HEAD_NUM_CLASSES,`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 11 `parameter int FEATURE_BITS = `CIFAR10_SF_HEAD_FEATURE_BITS,`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 12 `parameter int WEIGHT_BITS = `CIFAR10_SF_HEAD_WEIGHT_BITS,`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 13 `parameter int ACC_BITS = `CIFAR10_SF_HEAD_ACC_BITS,`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 14 `parameter int ENABLE_ONCHIP_LEARNING = 1,`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 15 `parameter int INFER_SKIP_ABS_THRESH = `CIFAR10_SF_HEAD_INFER_SKIP_ABS_THRESH,`

This line controls the feature-activity gate. When the absolute fixed-point feature is at or below this threshold, the RTL treats it as zero so the exported Spikingformer head ignores small noisy activations and records a measurable MAC-skip opportunity.

Line 16 `parameter int LEARN_SHIFT = 14,`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 17 `parameter int BIAS_LEARN_DELTA = 1,`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 18 `parameter logic signed [ACC_BITS-1:0] LEARN_MARGIN_Q = 64'sd268435456`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 19 `) (`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 20 `input wire clk,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 21 `input wire rst_n,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 22 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 23 `input wire start,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 24 `input wire feature_valid,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 25 `output logic feature_ready,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 26 `input wire signed [FEATURE_BITS-1:0] feature_value,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 27 `input wire frame_done,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 28 `input wire train_enable,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 29 `input wire [3:0] train_label,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 30 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 31 `output logic busy,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 32 `output logic result_valid,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 33 `output logic [3:0] predicted_label,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 34 `output logic signed [ACC_BITS-1:0] predicted_score,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 35 `output logic train_update_valid,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 36 `output logic train_update_applied,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 37 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 38 `output logic [5:0] debug_state,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 39 `output logic [15:0] debug_feature,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 40 `output logic [7:0] debug_class,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 41 `output logic [31:0] debug_update_count,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 42 `output logic [31:0] debug_infer_skip_count,`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 43 `output logic [31:0] debug_train_skip_count`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 44 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 45 `localparam int FEATURE_W = $clog2(FEATURE_DIM + 1);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 46 `localparam int CLASS_W = $clog2(NUM_CLASSES + 1);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 47 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 48 `typedef enum logic [5:0] {`

This declares the finite-state-machine state type. Using named states makes the cycle-by-cycle control flow explicit instead of relying on anonymous numeric constants.

Line 49 `ST_IDLE = 6'd0,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 50 `ST_LOAD = 6'd1,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 51 `ST_CLASS_INIT = 6'd2,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 52 `ST_MAC_READ = 6'd3,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 53 `ST_MAC_MUL = 6'd4,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 54 `ST_MAC_ACCUM = 6'd5,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 55 `ST_CLASS_DONE = 6'd6,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 56 `ST_RESULT = 6'd7,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 57 `ST_TRAIN_CHECK = 6'd8,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 58 `ST_TRAIN_READ_TRUE = 6'd9,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 59 `ST_TRAIN_CALC_TRUE = 6'd10,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 60 `ST_TRAIN_WRITE_TRUE = 6'd11,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 61 `ST_TRAIN_READ_PRED = 6'd12,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 62 `ST_TRAIN_CALC_PRED = 6'd13,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 63 `ST_TRAIN_WRITE_PRED = 6'd14,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 64 `ST_TRAIN_BIAS_TRUE = 6'd15,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 65 `ST_TRAIN_BIAS_PRED = 6'd16`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 66 `} state_t;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 67 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 68 `state_t state;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 69 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 70 `(* ram_style = "distributed" *) logic signed [FEATURE_BITS-1:0] feature_mem [0:FEATURE_DIM-1];`

This line touches the feature memory that holds one 384-dimensional fixed-point Spikingformer feature vector for the current image. Keeping the feature vector local avoids repeatedly streaming the same values for every class score.

Line 71 `(* ram_style = "block" *) logic signed [WEIGHT_BITS-1:0] weight_mem [0:NUM_CLASSES*FEATURE_DIM-1];`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 72 `logic signed [ACC_BITS-1:0] bias_mem [0:NUM_CLASSES-1];`

This line touches the class-bias store. Bias values are part of the affine classifier score and are updated on chip in separate one-write states to keep synthesis clean.

Line 73 `logic signed [ACC_BITS-1:0] class_score_mem [0:NUM_CLASSES-1];`

This stores or reads the ten class scores for the current sample. Those scores are needed for argmax prediction, CSV testbench reporting, and the margin test used before an online update.

Line 74 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 75 `logic [FEATURE_W-1:0] feature_idx;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 76 `logic [CLASS_W-1:0] class_idx;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 77 `logic signed [ACC_BITS-1:0] accum;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 78 `logic signed [FEATURE_BITS-1:0] feature_eff_r;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 79 `logic signed [WEIGHT_BITS-1:0] weight_r;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 80 `logic signed [ACC_BITS-1:0] product_r;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 81 `logic signed [WEIGHT_BITS-1:0] train_true_weight_r;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 82 `logic signed [WEIGHT_BITS-1:0] train_pred_weight_r;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 83 `logic signed [WEIGHT_BITS-1:0] train_write_weight_r;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 84 `logic [3:0] train_label_r;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 85 `logic [3:0] train_pred_label_r;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 86 `logic [31:0] update_count;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 87 `logic [31:0] infer_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 88 `logic [31:0] train_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 89 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 90 `assign feature_ready = (state == ST_LOAD);`

This is continuous combinational wiring. The output follows its source expression immediately in hardware rather than waiting for a clock edge.

Line 91 `assign busy = (state != ST_IDLE);`

This is continuous combinational wiring. The output follows its source expression immediately in hardware rather than waiting for a clock edge.

Line 92 `assign debug_state = state;`

This is continuous combinational wiring. The output follows its source expression immediately in hardware rather than waiting for a clock edge.

Line 93 `assign debug_feature = {{(16-FEATURE_W){1'b0}}, feature_idx};`

This is continuous combinational wiring. The output follows its source expression immediately in hardware rather than waiting for a clock edge.

Line 94 `assign debug_class = {4'd0, 4'(class_idx)};`

This is continuous combinational wiring. The output follows its source expression immediately in hardware rather than waiting for a clock edge.

Line 95 `assign debug_update_count = update_count;`

This is continuous combinational wiring. The output follows its source expression immediately in hardware rather than waiting for a clock edge.

Line 96 `assign debug_infer_skip_count = infer_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 97 `assign debug_train_skip_count = train_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 98 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 99 `initial begin`

This simulation/synthesis initialization block loads trained memories from generated .mem files. On FPGA, Vivado maps these initial values into BRAM/LUTRAM initialization data.

Line 100 `$readmemh("rtl/cifar10_spikingformer_head_weights.mem", weight_mem);`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 101 `$readmemh("rtl/cifar10_spikingformer_head_bias.mem", bias_mem);`

This line touches the class-bias store. Bias values are part of the affine classifier score and are updated on chip in separate one-write states to keep synthesis clean.

Line 102 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 103 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 104 `function automatic logic signed [WEIGHT_BITS-1:0] sat_weight(`

This begins a pure helper function used inside the RTL. It describes fixed combinational math, not a software function running on a CPU.

Line 105 `input logic signed [ACC_BITS-1:0] value`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 106 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 107 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 108 `if (value > ((1 << (WEIGHT_BITS - 1)) - 1)) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 109 `sat_weight = {1'b0, {(WEIGHT_BITS-1){1'b1}}};`

This line is part of signed saturation for 16-bit weights. Saturation prevents online learning from wrapping around when an update would exceed the representable fixed-point range.

Line 110 `end else if (value < -(1 << (WEIGHT_BITS - 1))) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 111 `sat_weight = {1'b1, {(WEIGHT_BITS-1){1'b0}}};`

This line is part of signed saturation for 16-bit weights. Saturation prevents online learning from wrapping around when an update would exceed the representable fixed-point range.

Line 112 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 113 `sat_weight = value[WEIGHT_BITS-1:0];`

This line is part of signed saturation for 16-bit weights. Saturation prevents online learning from wrapping around when an update would exceed the representable fixed-point range.

Line 114 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 115 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 116 `endfunction`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 117 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 118 `function automatic int weight_addr(input int cls, input int feat);`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 119 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 120 `weight_addr = cls * FEATURE_DIM + feat;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 121 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 122 `endfunction`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 123 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 124 `function automatic logic signed [ACC_BITS-1:0] learn_delta(`

This begins a pure helper function used inside the RTL. It describes fixed combinational math, not a software function running on a CPU.

Line 125 `input logic signed [FEATURE_BITS-1:0] value`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 126 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 127 `logic signed [ACC_BITS-1:0] extended;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 128 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 129 `extended = {{(ACC_BITS-FEATURE_BITS){value[FEATURE_BITS-1]}}, value};`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 130 `learn_delta = extended >> LEARN_SHIFT;`

This line implements the hardware learning step size. The feature is sign-extended and arithmetically shifted so positive and negative features update weights symmetrically.

Line 131 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 132 `endfunction`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 133 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 134 `function automatic logic [FEATURE_BITS:0] abs_feature(`

This begins a pure helper function used inside the RTL. It describes fixed combinational math, not a software function running on a CPU.

Line 135 `input logic signed [FEATURE_BITS-1:0] value`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 136 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 137 `logic signed [FEATURE_BITS:0] extended;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 138 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 139 `extended = {value[FEATURE_BITS-1], value};`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 140 `if (extended < 0) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 141 `abs_feature = (0 - extended);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 142 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 143 `abs_feature = extended[FEATURE_BITS:0];`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 144 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 145 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 146 `endfunction`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 147 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 148 `function automatic logic skip_feature(`

This line is part of the combinational decision that detects low-magnitude features. It is the hardware equivalent of asking whether a small activation should be silent for this inference/update pass.

Line 149 `input logic signed [FEATURE_BITS-1:0] value`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 150 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 151 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 152 `if (INFER_SKIP_ABS_THRESH <= 0) begin`

This line controls the feature-activity gate. When the absolute fixed-point feature is at or below this threshold, the RTL treats it as zero so the exported Spikingformer head ignores small noisy activations and records a measurable MAC-skip opportunity.

Line 153 `skip_feature = 1'b0;`

This line is part of the combinational decision that detects low-magnitude features. It is the hardware equivalent of asking whether a small activation should be silent for this inference/update pass.

Line 154 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 155 `skip_feature = abs_feature(value) <= INFER_SKIP_ABS_THRESH;`

This line controls the feature-activity gate. When the absolute fixed-point feature is at or below this threshold, the RTL treats it as zero so the exported Spikingformer head ignores small noisy activations and records a measurable MAC-skip opportunity.

Line 156 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 157 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 158 `endfunction`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 159 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 160 `function automatic logic signed [FEATURE_BITS-1:0] effective_feature(`

This line is part of the denoised feature path. The returned value is either the original signed feature or exactly zero, and that registered value is what feeds both the DSP multiply and the online-learning delta.

Line 161 `input logic signed [FEATURE_BITS-1:0] value`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 162 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 163 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 164 `if (skip_feature(value)) begin`

This line is part of the combinational decision that detects low-magnitude features. It is the hardware equivalent of asking whether a small activation should be silent for this inference/update pass.

Line 165 `effective_feature = '0;`

This line is part of the denoised feature path. The returned value is either the original signed feature or exactly zero, and that registered value is what feeds both the DSP multiply and the online-learning delta.

Line 166 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 167 `effective_feature = value;`

This line is part of the denoised feature path. The returned value is either the original signed feature or exactly zero, and that registered value is what feeds both the DSP multiply and the online-learning delta.

Line 168 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 169 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 170 `endfunction`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 171 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 172 `always_ff @(posedge clk or negedge rst_n) begin`

This begins clocked sequential logic. Registers and RAM writes inside this block update only on a clock edge, which is what makes the design synthesizable and waveform-predictable.

Line 173 `if (!rst_n) begin`

This is the active-low reset branch. It puts control registers and visible outputs into a known starting state before a frame is processed.

Line 174 `state <= ST_IDLE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 175 `feature_idx <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 176 `class_idx <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 177 `accum <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 178 `product_r <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 179 `result_valid <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 180 `train_update_valid <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 181 `train_update_applied <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 182 `predicted_label <= 4'd0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 183 `predicted_score <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 184 `train_write_weight_r <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 185 `train_label_r <= 4'd0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 186 `train_pred_label_r <= 4'd0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 187 `update_count <= 32'd0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 188 `infer_skip_count <= 32'd0;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 189 `train_skip_count <= 32'd0;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 190 `for (int c = 0; c < NUM_CLASSES; c++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 191 `class_score_mem[c] <= '0;`

This stores or reads the ten class scores for the current sample. Those scores are needed for argmax prediction, CSV testbench reporting, and the margin test used before an online update.

Line 192 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 193 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 194 `result_valid <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 195 `train_update_valid <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 196 `train_update_applied <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 197 `unique case (state)`

This starts a state-machine branch table. The unique qualifier tells tools that one legal state should match at a time, improving diagnostics and optimization.

Line 198 `ST_IDLE: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 199 `if (start) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 200 `feature_idx <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 201 `state <= ST_LOAD;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 202 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 203 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 204 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 205 `ST_LOAD: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 206 `if (feature_valid && feature_ready) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 207 `if (feature_idx == FEATURE_DIM - 1) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 208 `feature_idx <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 209 `class_idx <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 210 `state <= ST_CLASS_INIT;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 211 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 212 `feature_idx <= feature_idx + 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 213 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 214 `end else if (frame_done && feature_idx == FEATURE_DIM) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 215 `feature_idx <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 216 `class_idx <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 217 `state <= ST_CLASS_INIT;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 218 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 219 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 220 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 221 `ST_CLASS_INIT: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 222 `accum <= bias_mem[class_idx];`

This line touches the class-bias store. Bias values are part of the affine classifier score and are updated on chip in separate one-write states to keep synthesis clean.

Line 223 `feature_idx <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 224 `state <= ST_MAC_READ;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 225 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 226 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 227 `ST_MAC_READ: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 228 `if (skip_feature(feature_mem[feature_idx])) begin`

This line is part of the combinational decision that detects low-magnitude features. It is the hardware equivalent of asking whether a small activation should be silent for this inference/update pass.

Line 229 `infer_skip_count <= infer_skip_count + 1'b1;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 230 `state <= ST_CLASS_DONE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 231 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 232 `state <= ST_MAC_MUL;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 233 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 234 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 235 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 236 `ST_MAC_MUL: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 237 `product_r <= $signed(feature_eff_r) * $signed(weight_r);`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 238 `state <= ST_MAC_ACCUM;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 239 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 240 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 241 `ST_MAC_ACCUM: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 242 `accum <= accum + product_r;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 243 `state <= ST_CLASS_DONE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 244 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 245 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 246 `ST_CLASS_DONE: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 247 `if (feature_idx == FEATURE_DIM - 1) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 248 `class_score_mem[class_idx] <= accum;`

This stores or reads the ten class scores for the current sample. Those scores are needed for argmax prediction, CSV testbench reporting, and the margin test used before an online update.

Line 249 `if (class_idx == 0) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 250 `predicted_label <= 4'd0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 251 `predicted_score <= accum;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 252 `end else if (accum > predicted_score) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 253 `predicted_label <= 4'(class_idx);`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 254 `predicted_score <= accum;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 255 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 256 `if (class_idx == NUM_CLASSES - 1) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 257 `if (ENABLE_ONCHIP_LEARNING != 0) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 258 `state <= ST_TRAIN_CHECK;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 259 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 260 `state <= ST_RESULT;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 261 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 262 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 263 `class_idx <= class_idx + 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 264 `state <= ST_CLASS_INIT;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 265 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 266 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 267 `feature_idx <= feature_idx + 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 268 `state <= ST_MAC_READ;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 269 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 270 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 271 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 272 `ST_TRAIN_CHECK: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 273 `train_update_valid <= 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 274 `if (train_enable`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 275 `&& train_label < NUM_CLASSES`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 276 `&& train_label != predicted_label`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 277 `&& (predicted_score - class_score_mem[train_label]) <= LEARN_MARGIN_Q) begin`

This stores or reads the ten class scores for the current sample. Those scores are needed for argmax prediction, CSV testbench reporting, and the margin test used before an online update.

Line 278 `feature_idx <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 279 `train_label_r <= train_label;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 280 `train_pred_label_r <= predicted_label;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 281 `train_update_applied <= 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 282 `update_count <= update_count + 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 283 `state <= ST_TRAIN_READ_TRUE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 284 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 285 `state <= ST_RESULT;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 286 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 287 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 288 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 289 `ST_TRAIN_READ_TRUE: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 290 `if (skip_feature(feature_mem[feature_idx])) begin`

This line is part of the combinational decision that detects low-magnitude features. It is the hardware equivalent of asking whether a small activation should be silent for this inference/update pass.

Line 291 `train_skip_count <= train_skip_count + 1'b1;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 292 `if (feature_idx == FEATURE_DIM - 1) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 293 `state <= ST_TRAIN_BIAS_TRUE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 294 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 295 `feature_idx <= feature_idx + 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 296 `state <= ST_TRAIN_READ_TRUE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 297 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 298 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 299 `state <= ST_TRAIN_CALC_TRUE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 300 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 301 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 302 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 303 `ST_TRAIN_CALC_TRUE: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 304 `train_write_weight_r <= sat_weight(`

This line is part of signed saturation for 16-bit weights. Saturation prevents online learning from wrapping around when an update would exceed the representable fixed-point range.

Line 305 `{{(ACC_BITS-WEIGHT_BITS){train_true_weight_r[WEIGHT_BITS-1]}}, train_true_weight_r}`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 306 `+ learn_delta(feature_eff_r)`

This line implements the hardware learning step size. The feature is sign-extended and arithmetically shifted so positive and negative features update weights symmetrically.

Line 307 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 308 `state <= ST_TRAIN_WRITE_TRUE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 309 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 310 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 311 `ST_TRAIN_WRITE_TRUE: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 312 `state <= ST_TRAIN_READ_PRED;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 313 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 314 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 315 `ST_TRAIN_READ_PRED: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 316 `state <= ST_TRAIN_CALC_PRED;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 317 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 318 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 319 `ST_TRAIN_CALC_PRED: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 320 `train_write_weight_r <= sat_weight(`

This line is part of signed saturation for 16-bit weights. Saturation prevents online learning from wrapping around when an update would exceed the representable fixed-point range.

Line 321 `{{(ACC_BITS-WEIGHT_BITS){train_pred_weight_r[WEIGHT_BITS-1]}}, train_pred_weight_r}`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 322 `- learn_delta(feature_eff_r)`

This line implements the hardware learning step size. The feature is sign-extended and arithmetically shifted so positive and negative features update weights symmetrically.

Line 323 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 324 `state <= ST_TRAIN_WRITE_PRED;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 325 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 326 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 327 `ST_TRAIN_WRITE_PRED: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 328 `if (feature_idx == FEATURE_DIM - 1) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 329 `state <= ST_TRAIN_BIAS_TRUE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 330 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 331 `feature_idx <= feature_idx + 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 332 `state <= ST_TRAIN_READ_TRUE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 333 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 334 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 335 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 336 `ST_TRAIN_BIAS_TRUE: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 337 `state <= ST_TRAIN_BIAS_PRED;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 338 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 339 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 340 `ST_TRAIN_BIAS_PRED: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 341 `state <= ST_RESULT;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 342 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 343 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 344 `ST_RESULT: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 345 `result_valid <= 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 346 `state <= ST_IDLE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 347 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 348 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 349 `default:  begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 350 `state <= ST_IDLE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 351 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 352 `endcase`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 353 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 354 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 355 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 356 `always_ff @(posedge clk) begin`

This begins clocked sequential logic. Registers and RAM writes inside this block update only on a clock edge, which is what makes the design synthesizable and waveform-predictable.

Line 357 `unique case (state)`

This starts a state-machine branch table. The unique qualifier tells tools that one legal state should match at a time, improving diagnostics and optimization.

Line 358 `ST_LOAD: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 359 `if (feature_valid && feature_ready) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 360 `feature_mem[feature_idx] <= feature_value;`

This line touches the feature memory that holds one 384-dimensional fixed-point Spikingformer feature vector for the current image. Keeping the feature vector local avoids repeatedly streaming the same values for every class score.

Line 361 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 362 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 363 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 364 `ST_MAC_READ: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 365 `feature_eff_r <= effective_feature(feature_mem[feature_idx]);`

This line is part of the denoised feature path. The returned value is either the original signed feature or exactly zero, and that registered value is what feeds both the DSP multiply and the online-learning delta.

Line 366 `if (!skip_feature(feature_mem[feature_idx])) begin`

This line is part of the combinational decision that detects low-magnitude features. It is the hardware equivalent of asking whether a small activation should be silent for this inference/update pass.

Line 367 `weight_r <= weight_mem[weight_addr(int'(class_idx), int'(feature_idx))];`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 368 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 369 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 370 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 371 `ST_TRAIN_READ_TRUE: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 372 `feature_eff_r <= effective_feature(feature_mem[feature_idx]);`

This line is part of the denoised feature path. The returned value is either the original signed feature or exactly zero, and that registered value is what feeds both the DSP multiply and the online-learning delta.

Line 373 `if (!skip_feature(feature_mem[feature_idx])) begin`

This line is part of the combinational decision that detects low-magnitude features. It is the hardware equivalent of asking whether a small activation should be silent for this inference/update pass.

Line 374 `train_true_weight_r <= weight_mem[weight_addr(int'(train_label_r), int'(feature_idx))];`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 375 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 376 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 377 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 378 `ST_TRAIN_WRITE_TRUE: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 379 `weight_mem[weight_addr(int'(train_label_r), int'(feature_idx))] <= train_write_weight_r;`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 380 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 381 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 382 `ST_TRAIN_READ_PRED: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 383 `train_pred_weight_r <= weight_mem[weight_addr(int'(train_pred_label_r), int'(feature_idx))];`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 384 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 385 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 386 `ST_TRAIN_WRITE_PRED: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 387 `weight_mem[weight_addr(int'(train_pred_label_r), int'(feature_idx))] <= train_write_weight_r;`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 388 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 389 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 390 `ST_TRAIN_BIAS_TRUE: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 391 `bias_mem[train_label_r] <= bias_mem[train_label_r] + BIAS_LEARN_DELTA;`

This line touches the class-bias store. Bias values are part of the affine classifier score and are updated on chip in separate one-write states to keep synthesis clean.

Line 392 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 393 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 394 `ST_TRAIN_BIAS_PRED: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 395 `bias_mem[train_pred_label_r] <= bias_mem[train_pred_label_r] - BIAS_LEARN_DELTA;`

This line touches the class-bias store. Bias values are part of the affine classifier score and are updated on chip in separate one-write states to keep synthesis clean.

Line 396 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 397 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 398 `default:  begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 399 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 400 `endcase`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 401 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 402 `endmodule`

This closes the module definition. Synthesis stops adding logic to this circuit block at this point.

Line 403 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 404 `` `default_nettype wire ``

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

5.2 cifar10_neuromorphic\rtl\cifar10_spikingformer_head_pynq_axi_stream_top.sv

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `` `timescale 1ns / 1ps ``

This simulator directive sets the time unit and precision used by testbenches and waveform timestamps. It keeps delays such as clock half-periods numerically meaningful in XSim.

Line 2 `` `default_nettype none ``

This disables implicit one-bit wires. It prevents spelling mistakes from silently creating unwanted hardware nets.

Line 3 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 4 `module cifar10_spikingformer_head_pynq_axi_stream_top (`

This begins a hardware module, which is the FPGA equivalent of a reusable circuit block. Everything until the matching endmodule describes ports, storage, combinational equations, and clocked behavior for this block.

Line 5 `input wire ap_clk,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 6 `input wire ap_rst_n,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 7 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 8 `input wire s_axis_feature_tvalid,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 9 `output logic s_axis_feature_tready,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 10 `input wire signed [15:0] s_axis_feature_tdata,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 11 `input wire s_axis_feature_tlast,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 12 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 13 `input wire train_enable,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 14 `input wire [3:0] train_label,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 15 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 16 `output logic m_axis_result_tvalid,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 17 `input wire m_axis_result_tready,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 18 `output logic [31:0] m_axis_result_tdata,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 19 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 20 `output logic busy,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 21 `output logic [31:0] debug_infer_skip_count,`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 22 `output logic [31:0] debug_train_skip_count`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 23 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 24 `typedef enum logic [2:0] {`

This declares the finite-state-machine state type. Using named states makes the cycle-by-cycle control flow explicit instead of relying on anonymous numeric constants.

Line 25 `WR_IDLE = 3'd0,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 26 `WR_WAIT_READY = 3'd1,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 27 `WR_SEND_BUFFER = 3'd2,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 28 `WR_STREAM = 3'd3,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 29 `WR_FRAME_DONE = 3'd4,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 30 `WR_WAIT_RESULT = 3'd5,`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 31 `WR_HOLD_RESULT = 3'd6`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 32 `} wrap_state_t;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 33 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 34 `wrap_state_t wrap_state;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 35 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 36 `logic core_start;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 37 `logic core_feature_valid;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 38 `logic core_feature_ready;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 39 `logic signed [15:0] core_feature_value;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 40 `logic core_frame_done;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 41 `logic core_busy;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 42 `logic core_result_valid;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 43 `logic [3:0] core_predicted_label;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 44 `logic signed [63:0] core_predicted_score;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 45 `logic core_train_update_valid;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 46 `logic core_train_update_applied;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 47 `logic [5:0] unused_debug_state;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 48 `logic [15:0] unused_debug_feature;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 49 `logic [7:0] unused_debug_class;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 50 `logic [31:0] core_update_count;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 51 `logic [31:0] core_infer_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 52 `logic [31:0] core_train_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 53 `logic signed [15:0] buffered_feature;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 54 `logic buffered_last;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 55 `logic update_applied_latched;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 56 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 57 `assign busy = core_busy || (wrap_state != WR_IDLE);`

This is continuous combinational wiring. The output follows its source expression immediately in hardware rather than waiting for a clock edge.

Line 58 `assign s_axis_feature_tready = (wrap_state == WR_IDLE) || (wrap_state == WR_STREAM && core_feature_ready);`

This is continuous combinational wiring. The output follows its source expression immediately in hardware rather than waiting for a clock edge.

Line 59 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 60 `cifar10_spikingformer_head_accelerator u_head (`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 61 `.clk(ap_clk),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 62 `.rst_n(ap_rst_n),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 63 `.start(core_start),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 64 `.feature_valid(core_feature_valid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 65 `.feature_ready(core_feature_ready),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 66 `.feature_value(core_feature_value),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 67 `.frame_done(core_frame_done),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 68 `.train_enable(train_enable),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 69 `.train_label(train_label),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 70 `.busy(core_busy),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 71 `.result_valid(core_result_valid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 72 `.predicted_label(core_predicted_label),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 73 `.predicted_score(core_predicted_score),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 74 `.train_update_valid(core_train_update_valid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 75 `.train_update_applied(core_train_update_applied),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 76 `.debug_state(unused_debug_state),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 77 `.debug_feature(unused_debug_feature),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 78 `.debug_class(unused_debug_class),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 79 `.debug_update_count(core_update_count),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 80 `.debug_infer_skip_count(core_infer_skip_count),`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 81 `.debug_train_skip_count(core_train_skip_count)`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 82 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 83 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 84 `assign debug_infer_skip_count = core_infer_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 85 `assign debug_train_skip_count = core_train_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 86 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 87 `always_ff @(posedge ap_clk or negedge ap_rst_n) begin`

This begins clocked sequential logic. Registers and RAM writes inside this block update only on a clock edge, which is what makes the design synthesizable and waveform-predictable.

Line 88 `if (!ap_rst_n) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 89 `wrap_state <= WR_IDLE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 90 `core_start <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 91 `core_feature_valid <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 92 `core_feature_value <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 93 `core_frame_done <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 94 `buffered_feature <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 95 `buffered_last <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 96 `update_applied_latched <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 97 `m_axis_result_tvalid <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 98 `m_axis_result_tdata <= 32'd0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 99 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 100 `core_start <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 101 `core_feature_valid <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 102 `core_frame_done <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 103 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 104 `if (core_train_update_applied) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 105 `update_applied_latched <= 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 106 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 107 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 108 `unique case (wrap_state)`

This starts a state-machine branch table. The unique qualifier tells tools that one legal state should match at a time, improving diagnostics and optimization.

Line 109 `WR_IDLE: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 110 `update_applied_latched <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 111 `if (s_axis_feature_tvalid && s_axis_feature_tready) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 112 `buffered_feature <= s_axis_feature_tdata;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 113 `buffered_last <= s_axis_feature_tlast;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 114 `core_start <= 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 115 `wrap_state <= WR_WAIT_READY;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 116 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 117 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 118 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 119 `WR_WAIT_READY: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 120 `if (core_feature_ready) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 121 `core_feature_value <= buffered_feature;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 122 `core_feature_valid <= 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 123 `wrap_state <= WR_SEND_BUFFER;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 124 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 125 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 126 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 127 `WR_SEND_BUFFER: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 128 `if (buffered_last) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 129 `wrap_state <= WR_FRAME_DONE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 130 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 131 `wrap_state <= WR_STREAM;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 132 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 133 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 134 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 135 `WR_STREAM: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 136 `if (s_axis_feature_tvalid && s_axis_feature_tready) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 137 `core_feature_value <= s_axis_feature_tdata;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 138 `core_feature_valid <= 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 139 `if (s_axis_feature_tlast) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 140 `wrap_state <= WR_FRAME_DONE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 141 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 142 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 143 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 144 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 145 `WR_FRAME_DONE: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 146 `core_frame_done <= 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 147 `wrap_state <= WR_WAIT_RESULT;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 148 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 149 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 150 `WR_WAIT_RESULT: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 151 `if (core_result_valid) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 152 `m_axis_result_tdata <= {`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 153 `core_predicted_score[15:0],`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 154 `core_update_count[10:0],`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 155 `update_applied_latched,`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 156 `core_predicted_label`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 157 `};`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 158 `m_axis_result_tvalid <= 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 159 `wrap_state <= WR_HOLD_RESULT;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 160 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 161 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 162 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 163 `WR_HOLD_RESULT: begin`

This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed.

Line 164 `if (m_axis_result_tvalid && m_axis_result_tready) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 165 `m_axis_result_tvalid <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 166 `wrap_state <= WR_IDLE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 167 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 168 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 169 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 170 `default: begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 171 `wrap_state <= WR_IDLE;`

This schedules the next FSM state for the following clock edge. That transition is the hardware pipeline control path.

Line 172 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 173 `endcase`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 174 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 175 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 176 `endmodule`

This closes the module definition. Synthesis stops adding logic to this circuit block at this point.

Line 177 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 178 `‘default_nettype wire`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

5.3 cifar10_neuromorphic\rtl\tb_spikingformer_head_learning_unit.sv

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `` `timescale 1ns / 1ps ``

This simulator directive sets the time unit and precision used by testbenches and waveform timestamps. It keeps delays such as clock half-periods numerically meaningful in XSim.

Line 2 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 3 `module tb_spikingformer_head_learning_unit;`

This begins a hardware module, which is the FPGA equivalent of a reusable circuit block. Everything until the matching endmodule describes ports, storage, combinational equations, and clocked behavior for this block.

Line 4 `localparam int FEATURE_DIM = 4;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 5 `localparam int NUM_CLASSES = 3;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 6 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 7 `logic clk = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 8 `logic rst_n = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 9 `logic start;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 10 `logic feature_valid;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 11 `logic feature_ready;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 12 `logic signed [15:0] feature_value;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 13 `logic frame_done;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 14 `logic train_enable;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 15 `logic [3:0] train_label;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 16 `logic busy;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 17 `logic result_valid;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 18 `logic [3:0] predicted_label;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 19 `logic signed [63:0] predicted_score;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 20 `logic train_update_valid;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 21 `logic train_update_applied;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 22 `logic [5:0] debug_state;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 23 `logic [15:0] debug_feature;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 24 `logic [7:0] debug_class;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 25 `logic [31:0] debug_update_count;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 26 `logic [31:0] debug_infer_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 27 `logic [31:0] debug_train_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 28 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 29 `logic signed [15:0] x [0:FEATURE_DIM-1];`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 30 `int errors = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 31 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 32 `cifar10_spikingformer_head_accelerator #(`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 33 `.FEATURE_DIM(FEATURE_DIM),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 34 `.NUM_CLASSES(NUM_CLASSES),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 35 `.FEATURE_BITS(16),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 36 `.WEIGHT_BITS(16),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 37 `.ACC_BITS(64),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 38 `.ENABLE_ONCHIP_LEARNING(1),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 39 `.INFER_SKIP_ABS_THRESH(0),`

This line controls the feature-activity gate. When the absolute fixed-point feature is at or below this threshold, the RTL treats it as zero so the exported Spikingformer head ignores small noisy activations and records a measurable MAC-skip opportunity.

Line 40 `.LEARN_SHIFT(1),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 41 `.BIAS_LEARN_DELTA(3),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 42 `.LEARN_MARGIN_Q(64'sd1000000)`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 43 `) dut (`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 44 `.clk(clk),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 45 `.rst_n(rst_n),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 46 `.start(start),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 47 `.feature_valid(feature_valid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 48 `.feature_ready(feature_ready),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 49 `.feature_value(feature_value),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 50 `.frame_done(frame_done),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 51 `.train_enable(train_enable),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 52 `.train_label(train_label),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 53 `.busy(busy),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 54 `.result_valid(result_valid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 55 `.predicted_label(predicted_label),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 56 `.predicted_score(predicted_score),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 57 `.train_update_valid(train_update_valid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 58 `.train_update_applied(train_update_applied),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 59 `.debug_state(debug_state),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 60 `.debug_feature(debug_feature),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 61 `.debug_class(debug_class),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 62 `.debug_update_count(debug_update_count),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 63 `.debug_infer_skip_count(debug_infer_skip_count),`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 64 `.debug_train_skip_count(debug_train_skip_count)`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 65 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 66 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 67 `always #5 clk = ~clk;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 68 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 69 `task automatic expect_i16(input string name, input int got, input int expected);`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 70 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 71 `if (got !== expected) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 72 `$display("ERROR: %s got=%0d expected=%0d", name, got, expected);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 73 `errors++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 74 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 75 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 76 `endtask`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 77 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 78 `initial begin`

This simulation/synthesis initialization block loads trained memories from generated .mem files. On FPGA, Vivado maps these initial values into BRAM/LUTRAM initialization data.

Line 79 `x[0] = 16'sd4;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 80 `x[1] = -16'sd2;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 81 `x[2] = 16'sd6;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 82 `x[3] = -16'sd8;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 83 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 84 `start = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 85 `feature_valid = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 86 `feature_value = 16'sd0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 87 `frame_done = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 88 `train_enable = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 89 `train_label = 4'd1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 90 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 91 `repeat (8) @(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 92 `for (int i = 0; i < NUM_CLASSES * FEATURE_DIM; i++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 93 `dut.weight_mem[i] = 16'sd0;`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 94 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 95 `dut.bias_mem[0] = 64'sd0;`

This line touches the class-bias store. Bias values are part of the affine classifier score and are updated on chip in separate one-write states to keep synthesis clean.

Line 96 `dut.bias_mem[1] = 64'sd0;`

This line touches the class-bias store. Bias values are part of the affine classifier score and are updated on chip in separate one-write states to keep synthesis clean.

Line 97 `dut.bias_mem[2] = 64'sd20;`

This line touches the class-bias store. Bias values are part of the affine classifier score and are updated on chip in separate one-write states to keep synthesis clean.

Line 98 `rst_n = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 99 `repeat (4) @(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 100 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 101 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 102 `train_enable = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 103 `train_label = 4'd1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 104 `start = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 105 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 106 `start = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 107 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 108 `while (!feature_ready) @(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 109 `for (int f = 0; f < FEATURE_DIM; f++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 110 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 111 `feature_value = x[f];`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 112 `feature_valid = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 113 `@(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 114 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 115 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 116 `feature_valid = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 117 `frame_done = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 118 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 119 `frame_done = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 120 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 121 `while (!result_valid) @(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 122 `if (predicted_label !== 4'd2) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 123 `$display("ERROR: predicted_label got=%0d expected=2", predicted_label);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 124 `errors++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 125 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 126 `if (!train_update_applied && debug_update_count != 32'd1) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 127 `$display("ERROR: update did not apply");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 128 `errors++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 129 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 130 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 131 `while (busy) @(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 132 `expect_i16("class1 weight0", dut.weight_mem[1 * FEATURE_DIM + 0], 2);`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 133 `expect_i16("class1 weight1", dut.weight_mem[1 * FEATURE_DIM + 1], -1);`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 134 `expect_i16("class1 weight2", dut.weight_mem[1 * FEATURE_DIM + 2], 3);`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 135 `expect_i16("class1 weight3", dut.weight_mem[1 * FEATURE_DIM + 3], -4);`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 136 `expect_i16("class2 weight0", dut.weight_mem[2 * FEATURE_DIM + 0], -2);`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 137 `expect_i16("class2 weight1", dut.weight_mem[2 * FEATURE_DIM + 1], 1);`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 138 `expect_i16("class2 weight2", dut.weight_mem[2 * FEATURE_DIM + 2], -3);`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 139 `expect_i16("class2 weight3", dut.weight_mem[2 * FEATURE_DIM + 3], 4);`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 140 `if ($signed(dut.bias_mem[1]) !== 64'sd3 || $signed(dut.bias_mem[2]) !== 64'sd17) begin`

This line touches the class-bias store. Bias values are part of the affine classifier score and are updated on chip in separate one-write states to keep synthesis clean.

Line 141 `$display("ERROR: bias update got class1=%0d class2=%0d", $signed(dut.bias_mem[1]), $signed(dut.bias_mem[2]));`

This line touches the class-bias store. Bias values are part of the affine classifier score and are updated on chip in separate one-write states to keep synthesis clean.

Line 142 `errors++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 143 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 144 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 145 `$display("tb_spikingformer_head_learning_unit errors=%0d", errors);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 146 `if (errors != 0) $fatal;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 147 `$finish;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 148 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 149 `endmodule`

This closes the module definition. Synthesis stops adding logic to this circuit block at this point.

5.4 cifar10_neuromorphic\rtl\tb_cifar10_spikingformer_head_accelerator.sv

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `‘timescale 1ns / 1ps`

This simulator directive sets the time unit and precision used by testbenches and waveform timestamps. It keeps delays such as clock half-periods numerically meaningful in XSim.

Line 2 `‘include "rtl/cifar10_spikingformer_head_params.svh"`

This includes shared compile-time constants such as feature dimensions and fixed-point widths. The RTL and testbenches therefore agree on the same trained-model shape.

Line 3 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 4 `module tb_cifar10_spikingformer_head_accelerator;`

This begins a hardware module, which is the FPGA equivalent of a reusable circuit block. Everything until the matching endmodule describes ports, storage, combinational equations, and clocked behavior for this block.

Line 5 `parameter int NUM_SAMPLES = ‘CIFAR10_SF_HEAD_NUM_SAMPLES;`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 6 `parameter int TRACE_SAMPLES = ‘CIFAR10_SF_HEAD_TRACE_SAMPLES;`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 7 `parameter int FEATURE_DIM = ‘CIFAR10_SF_HEAD_FEATURE_DIM;`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 8 `parameter int MAX_SAMPLES = 10000;`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 9 `parameter int MAX_FEATURE_WORDS = MAX_SAMPLES * FEATURE_DIM;`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 10 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 11 `logic clk = 1’b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 12 `logic rst_n = 1’b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 13 `logic start;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 14 `logic feature_valid;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 15 `logic feature_ready;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 16 `logic signed [`CIFAR10_SF_HEAD_FEATURE_BITS-1:0] feature_value;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 17 `logic frame_done;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 18 `logic train_enable;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 19 `logic [3:0] train_label;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 20 `logic busy;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 21 `logic result_valid;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 22 `logic [3:0] predicted_label;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 23 `logic signed [`CIFAR10_SF_HEAD_ACC_BITS-1:0] predicted_score;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 24 `logic train_update_valid;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 25 `logic train_update_applied;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 26 `logic [5:0] debug_state;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 27 `logic [15:0] debug_feature;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 28 `logic [7:0] debug_class;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 29 `logic [31:0] debug_update_count;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 30 `logic [31:0] debug_infer_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 31 `logic [31:0] debug_train_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 32 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 33 `logic signed [`CIFAR10_SF_HEAD_FEATURE_BITS-1:0] feature_mem [0:MAX_FEATURE_WORDS-1];`

This line touches the feature memory that holds one 384-dimensional fixed-point Spikingformer feature vector for the current image. Keeping the feature vector local avoids repeatedly streaming the same values for every class score.

Line 34 `logic [3:0] label_mem [0:MAX_SAMPLES-1];`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 35 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 36 `int feature_ptr = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 37 `int correct = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 38 `int errors = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 39 `int cycle = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 40 `int trace_fh;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 41 `int result_fh;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 42 `bit trace_enable = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 43 `bit vcd_enabled = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 44 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 45 `cifar10_spikingformer_head_accelerator dut (`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 46 `.clk(clk),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 47 `.rst_n(rst_n),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 48 `.start(start),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 49 `.feature_valid(feature_valid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 50 `.feature_ready(feature_ready),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 51 `.feature_value(feature_value),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 52 `.frame_done(frame_done),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 53 `.train_enable(train_enable),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 54 `.train_label(train_label),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 55 `.busy(busy),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 56 `.result_valid(result_valid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 57 `.predicted_label(predicted_label),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 58 `.predicted_score(predicted_score),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 59 `.train_update_valid(train_update_valid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 60 `.train_update_applied(train_update_applied),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 61 `.debug_state(debug_state),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 62 `.debug_feature(debug_feature),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 63 `.debug_class(debug_class),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 64 `.debug_update_count(debug_update_count),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 65 `.debug_infer_skip_count(debug_infer_skip_count),`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 66 `.debug_train_skip_count(debug_train_skip_count)`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 67 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 68 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 69 `always #5 clk = ~clk;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 70 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 71 `task automatic require_file(input string path);`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 72 `int fh;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 73 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 74 `fh = $fopen(path, "r");`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 75 `if (fh == 0) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 76 `$display("ERROR: missing required file %s", path);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 77 `$fatal;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 78 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 79 `$fclose(fh);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 80 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 81 `endtask`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 82 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 83 `task automatic send_sample(input int sample_idx);`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 84 `int waited;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 85 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 86 `trace_enable = (sample_idx < TRACE_SAMPLES);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 87 `if (sample_idx == TRACE_SAMPLES && vcd_enabled) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 88 `$dumpoff;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 89 `vcd_enabled = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 90 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 91 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 92 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 93 `start = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 94 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 95 `start = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 96 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 97 `waited = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 98 `while (!feature_ready && waited < 10000) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 99 `@(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 100 `waited++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 101 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 102 `if (!feature_ready) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 103 `$display("ERROR: sample %0d never reached feature_ready", sample_idx);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 104 `errors++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 105 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 106 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 107 `for (int f = 0; f < FEATURE_DIM; f++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 108 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 109 `feature_value = feature_mem[feature_ptr];`

This line touches the feature memory that holds one 384-dimensional fixed-point Spikingformer feature vector for the current image. Keeping the feature vector local avoids repeatedly streaming the same values for every class score.

Line 110 `feature_ptr++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 111 `feature_valid = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 112 `@(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 113 `#1;`

This is a human-facing comment. It documents intent for the reader and is ignored by the compiler or interpreter.

Line 114 `if (f < FEATURE_DIM - 1 && !feature_ready) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 115 `$display("ERROR: feature_ready dropped during sample %0d feature %0d", sample_idx, f);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 116 `errors++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 117 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 118 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 119 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 120 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 121 `feature_valid = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 122 `frame_done = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 123 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 124 `frame_done = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 125 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 126 `waited = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 127 `while (!result_valid && waited < 100000) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 128 `@(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 129 `waited++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 130 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 131 `if (!result_valid) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 132 `$display("ERROR: sample %0d no result_valid", sample_idx);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 133 `errors++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 134 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 135 `if (predicted_label == label_mem[sample_idx]) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 136 `correct++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 137 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 138 `$fwrite(result_fh, "%0d,%0d,%0d,%0d,%0d,%0d",`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 139 `sample_idx, label_mem[sample_idx], predicted_label, predicted_score,`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 140 `predicted_label == label_mem[sample_idx], debug_infer_skip_count, debug_train_skip_count);`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 141 `for (int c = 0; c < 10; c++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 142 `$fwrite(result_fh, ",%0d", $signed(dut.class_score_mem[c]));`

This stores or reads the ten class scores for the current sample. Those scores are needed for argmax prediction, CSV testbench reporting, and the margin test used before an online update.

Line 143 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 144 `$fwrite(result_fh, "\n");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 145 `if (sample_idx < TRACE_SAMPLES || (sample_idx % 1000) == 0 || sample_idx == NUM_SAMPLES - 1) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 146 `$display("sample=%0d label=%0d pred=%0d correct=%0d",`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 147 `sample_idx, label_mem[sample_idx], predicted_label,`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 148 `predicted_label == label_mem[sample_idx]);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 149 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 150 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 151 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 152 `waited = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 153 `while (busy && waited < 1000) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 154 `@(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 155 `waited++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 156 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 157 `trace_enable = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 158 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 159 `endtask`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 160 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 161 `always @(posedge clk) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 162 `if (trace_fh != 0 && trace_enable) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 163 `$fwrite(trace_fh, "%0t,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d\n",`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 164 `$time, cycle, rst_n, debug_state, start, feature_valid, feature_ready,`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 165 `debug_feature, debug_class, result_valid, predicted_label,`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 166 `debug_infer_skip_count, debug_train_skip_count);`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 167 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 168 `cycle++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 169 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 170 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 171 `initial begin`

This simulation/synthesis initialization block loads trained memories from generated .mem files. On FPGA, Vivado maps these initial values into BRAM/LUTRAM initialization data.

Line 172 `if (NUM_SAMPLES > MAX_SAMPLES) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 173 `$display("ERROR: NUM_SAMPLES=%0d exceeds MAX_SAMPLES=%0d", NUM_SAMPLES, MAX_SAMPLES);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 174 `$fatal;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 175 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 176 `require_file("rtl/cifar10_spikingformer_head_features.mem");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 177 `require_file("rtl/cifar10_spikingformer_head_weights.mem");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 178 `require_file("rtl/cifar10_spikingformer_head_bias.mem");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 179 `require_file("rtl/cifar10_spikingformer_head_labels.mem");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 180 `$readmemh("rtl/cifar10_spikingformer_head_features.mem", feature_mem);`

This line touches the feature memory that holds one 384-dimensional fixed-point Spikingformer feature vector for the current image. Keeping the feature vector local avoids repeatedly streaming the same values for every class score.

Line 181 `$readmemh("rtl/cifar10_spikingformer_head_labels.mem", label_mem);`

This reads a hexadecimal memory file into an RTL array. It is how exported trained weights and biases become deterministic FPGA memory contents.

Line 182 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 183 `$dumpfile("cifar10_spikingformer_head_accelerator.vcd");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 184 `$dumpvars(0, tb_cifar10_spikingformer_head_accelerator);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 185 `trace_fh = $fopen("cifar10_spikingformer_head_trace.csv", "w");`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 186 `result_fh = $fopen("cifar10_spikingformer_head_results.csv", "w");`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 187 `$fwrite(trace_fh, "time_ps,cycle,rst_n,state,start,feature_valid,feature_ready,feature_idx,class_idx,result_valid,predicted_label,infer_skip_count,train_ski`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 188 `$fwrite(result_fh, "sample,label,predicted,score,correct,infer_skip_count,train_skip_count");`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 189 `for (int c = 0; c < 10; c++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 190 `$fwrite(result_fh, ",score%0d", c);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 191 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 192 `$fwrite(result_fh, "\n");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 193 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 194 `start = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 195 `feature_valid = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 196 `feature_value = '0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 197 `frame_done = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 198 `train_enable = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 199 `train_label = 4'd0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 200 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 201 `repeat (8) @(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 202 `rst_n = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 203 `repeat (4) @(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 204 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 205 `for (int s = 0; s < NUM_SAMPLES; s++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 206 `send_sample(s);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 207 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 208 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 209 `repeat (10) @(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 210 `$fclose(trace_fh);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 211 `$fclose(result_fh);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 212 `$display("tb_cifar10_spikingformer_head_accelerator samples=%0d correct=%0d errors=%0d", NUM_SAMPLES, correct, errors);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 213 `if (errors != 0) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 214 `$fatal;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 215 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 216 `$finish;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 217 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 218 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 219 `initial begin`

This simulation/synthesis initialization block loads trained memories from generated .mem files. On FPGA, Vivado maps these initial values into BRAM/LUTRAM initialization data.

Line 220 `#2_000_000_000;`

This is a human-facing comment. It documents intent for the reader and is ignored by the compiler or interpreter.

Line 221 `$display("ERROR: tb_cifar10_spikingformer_head_accelerator watchdog timeout");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 222 `$fatal;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 223 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 224 `endmodule`

This closes the module definition. Synthesis stops adding logic to this circuit block at this point.

5.5 cifar10_neuromorphic\rtl\tb_cifar10_spikingformer_head_onchip_learning.sv

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `‘timescale 1ns / 1ps`

This simulator directive sets the time unit and precision used by testbenches and waveform timestamps. It keeps delays such as clock half-periods numerically meaningful in XSim.

Line 2 `‘include "rtl/cifar10_spikingformer_head_params.svh"`

This includes shared compile-time constants such as feature dimensions and fixed-point widths. The RTL and testbenches therefore agree on the same trained-model shape.

Line 3 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 4 `module tb_cifar10_spikingformer_head_onchip_learning;`

This begins a hardware module, which is the FPGA equivalent of a reusable circuit block. Everything until the matching endmodule describes ports, storage, combinational equations, and clocked behavior for this block.

Line 5 `parameter int NUM_SAMPLES = ‘CIFAR10_SF_HEAD_NUM_SAMPLES;`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 6 `parameter int TRACE_SAMPLES = ‘CIFAR10_SF_HEAD_TRACE_SAMPLES;`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 7 `parameter int FEATURE_DIM = ‘CIFAR10_SF_HEAD_FEATURE_DIM;`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 8 `parameter int MAX_SAMPLES = 10000;`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 9 `parameter int MAX_FEATURE_WORDS = MAX_SAMPLES * FEATURE_DIM;`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 10 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 11 `logic clk = 1’b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 12 `logic rst_n = 1’b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 13 `logic start;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 14 `logic feature_valid;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 15 `logic feature_ready;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 16 `logic signed [‘CIFAR10_SF_HEAD_FEATURE_BITS-1:0] feature_value;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 17 `logic frame_done;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 18 `logic train_enable;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 19 `logic [3:0] train_label;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 20 `logic busy;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 21 `logic result_valid;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 22 `logic [3:0] predicted_label;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 23 `logic signed [‘CIFAR10_SF_HEAD_ACC_BITS-1:0] predicted_score;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 24 `logic train_update_valid;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 25 `logic train_update_applied;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 26 `logic [5:0] debug_state;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 27 `logic [15:0] debug_feature;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 28 `logic [7:0] debug_class;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 29 `logic [31:0] debug_update_count;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 30 `logic [31:0] debug_infer_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 31 `logic [31:0] debug_train_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 32 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 33 `logic signed [`CIFAR10_SF_HEAD_FEATURE_BITS-1:0] feature_mem [0:MAX_FEATURE_WORDS-1];`

This line touches the feature memory that holds one 384-dimensional fixed-point Spikingformer feature vector for the current image. Keeping the feature vector local avoids repeatedly streaming the same values for every class score.

Line 34 `logic [3:0] label_mem [0:MAX_SAMPLES-1];`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 35 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 36 `int feature_ptr = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 37 `int correct = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 38 `int errors = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 39 `int updates = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 40 `int cycle = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 41 `int trace_fh;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 42 `int result_fh;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 43 `int weight_fh;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 44 `int bias_fh;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 45 `bit trace_enable = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 46 `bit vcd_enabled = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 47 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 48 `cifar10_spikingformer_head_accelerator #(`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 49 `.ENABLE_ONCHIP_LEARNING(1),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 50 `.LEARN_SHIFT(14),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 51 `.BIAS_LEARN_DELTA(1),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 52 `.LEARN_MARGIN_Q(64'sd268435456)`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 53 `) dut (`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 54 `.clk(clk),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 55 `.rst_n(rst_n),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 56 `.start(start),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 57 `.feature_valid(feature_valid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 58 `.feature_ready(feature_ready),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 59 `.feature_value(feature_value),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 60 `.frame_done(frame_done),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 61 `.train_enable(train_enable),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 62 `.train_label(train_label),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 63 `.busy(busy),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 64 `.result_valid(result_valid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 65 `.predicted_label(predicted_label),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 66 `.predicted_score(predicted_score),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 67 `.train_update_valid(train_update_valid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 68 `.train_update_applied(train_update_applied),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 69 `.debug_state(debug_state),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 70 `.debug_feature(debug_feature),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 71 `.debug_class(debug_class),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 72 `.debug_update_count(debug_update_count),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 73 `.debug_infer_skip_count(debug_infer_skip_count),`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 74 `.debug_train_skip_count(debug_train_skip_count)`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 75 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 76 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 77 `always #5 clk = ~clk;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 78 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 79 `task automatic require_file(input string path);`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 80 `int fh;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 81 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 82 `fh = $fopen(path, "r");`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 83 `if (fh == 0) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 84 `$display("ERROR: missing required file %s", path);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 85 `$fatal;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 86 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 87 `$fclose(fh);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 88 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 89 `endtask`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 90 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 91 `task automatic send_sample(input int sample_idx);`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 92 `int waited;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 93 `bit update_seen;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 94 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 95 `trace_enable = (sample_idx < TRACE_SAMPLES);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 96 `if (sample_idx == TRACE_SAMPLES && vcd_enabled) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 97 `$dumpoff;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 98 `vcd_enabled = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 99 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 100 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 101 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 102 `train_label = label_mem[sample_idx];`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 103 `train_enable = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 104 `start = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 105 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 106 `start = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 107 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 108 `waited = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 109 `while (!feature_ready && waited < 10000) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 110 `@(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 111 `waited++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 112 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 113 `if (!feature_ready) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 114 `$display("ERROR: sample %0d never reached feature_ready", sample_idx);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 115 `errors++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 116 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 117 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 118 `for (int f = 0; f < FEATURE_DIM; f++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 119 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 120 `feature_value = feature_mem[feature_ptr];`

This line touches the feature memory that holds one 384-dimensional fixed-point Spikingformer feature vector for the current image. Keeping the feature vector local avoids repeatedly streaming the same values for every class score.

Line 121 `feature_ptr++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 122 `feature_valid = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 123 `@(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 124 `#1;`

This is a human-facing comment. It documents intent for the reader and is ignored by the compiler or interpreter.

Line 125 `if (f < FEATURE_DIM - 1 && !feature_ready) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 126 `$display("ERROR: feature_ready dropped during sample %0d feature %0d", sample_idx, f);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 127 `errors++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 128 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 129 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 130 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 131 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 132 `feature_valid = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 133 `frame_done = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 134 `@(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 135 `frame_done = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 136 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 137 `waited = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 138 `update_seen = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 139 `while (!result_valid && waited < 200000) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 140 `@(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 141 `if (train_update_applied) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 142 `update_seen = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 143 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 144 `waited++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 145 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 146 `if (!result_valid) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 147 `$display("ERROR: sample %0d no result_valid", sample_idx);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 148 `errors++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 149 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 150 `if (predicted_label == label_mem[sample_idx]) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 151 `correct++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 152 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 153 `if (update_seen) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 154 `updates++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 155 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 156 `$fwrite(result_fh, "%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d",`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 157 `sample_idx, label_mem[sample_idx], predicted_label, predicted_score,`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 158 `predicted_label == label_mem[sample_idx], update_seen, debug_update_count,`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 159 `debug_infer_skip_count, debug_train_skip_count);`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 160 `for (int c = 0; c < 10; c++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 161 `$fwrite(result_fh, ",%0d", $signed(dut.class_score_mem[c]));`

This stores or reads the ten class scores for the current sample. Those scores are needed for argmax prediction, CSV testbench reporting, and the margin test used before an online update.

Line 162 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 163 `$fwrite(result_fh, "\n");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 164 `if (sample_idx < TRACE_SAMPLES || (sample_idx % 1000) == 0 || sample_idx == NUM_SAMPLES - 1) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 165 `$display("learn_sample=%0d label=%0d pred=%0d correct=%0d update=%0d",`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 166 `sample_idx, label_mem[sample_idx], predicted_label,`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 167 `predicted_label == label_mem[sample_idx], update_seen);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 168 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 169 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 170 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 171 `waited = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 172 `while (busy && waited < 1000) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 173 `@(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 174 `waited++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 175 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 176 `train_enable = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 177 `trace_enable = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 178 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 179 `endtask`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 180 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 181 `always @(posedge clk) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 182 `if (trace_fh != 0 && trace_enable) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 183 `$fwrite(trace_fh, "%0t,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d,%0d\n",`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 184 `$time, cycle, rst_n, debug_state, start, feature_valid, feature_ready,`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 185 `debug_feature, debug_class, result_valid, predicted_label,`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 186 `train_update_applied, debug_update_count, debug_infer_skip_count, debug_train_skip_count);`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 187 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 188 `cycle++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 189 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 190 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 191 `initial begin`

This simulation/synthesis initialization block loads trained memories from generated .mem files. On FPGA, Vivado maps these initial values into BRAM/LUTRAM initialization data.

Line 192 `if (NUM_SAMPLES > MAX_SAMPLES) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 193 `$display("ERROR: NUM_SAMPLES=%0d exceeds MAX_SAMPLES=%0d", NUM_SAMPLES, MAX_SAMPLES);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 194 `$fatal;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 195 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 196 `require_file("rtl/cifar10_spikingformer_head_features.mem");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 197 `require_file("rtl/cifar10_spikingformer_head_weights.mem");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 198 `require_file("rtl/cifar10_spikingformer_head_bias.mem");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 199 `require_file("rtl/cifar10_spikingformer_head_labels.mem");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 200 `$readmemh("rtl/cifar10_spikingformer_head_features.mem", feature_mem);`

This line touches the feature memory that holds one 384-dimensional fixed-point Spikingformer feature vector for the current image. Keeping the feature vector local avoids repeatedly streaming the same values for every class score.

Line 201 `$readmemh("rtl/cifar10_spikingformer_head_labels.mem", label_mem);`

This reads a hexadecimal memory file into an RTL array. It is how exported trained weights and biases become deterministic FPGA memory contents.

Line 202 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 203 `$dumpfile("cifar10_spikingformer_head_onchip_learning.vcd");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 204 `$dumpvars(0, tb_cifar10_spikingformer_head_onchip_learning);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 205 `trace_fh = $fopen("cifar10_spikingformer_head_onchip_learning_trace.csv", "w");`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 206 `result_fh = $fopen("cifar10_spikingformer_head_onchip_learning_results.csv", "w");`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 207
```
$fwrite(trace_fh,
"time_ps,cycle,rst_n,state,start,feature_valid,feature_ready,feature_idx,class_idx,result_valid,predicted_label,train_update_applied,updat
```

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 208 `$fwrite(result_fh, "sample,label,predicted,score,correct,update_applied,update_count,infer_skip_count,train_skip_count");`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 209 `for (int c = 0; c < 10; c++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 210 `$fwrite(result_fh, ",score%0d", c);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 211 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 212 `$fwrite(result_fh, "\n");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 213 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 214 `start = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 215 `feature_valid = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 216 `feature_value = '0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 217 `frame_done = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 218 `train_enable = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 219 `train_label = 4'd0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 220 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 221 `repeat (8) @(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 222 `rst_n = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 223 `repeat (4) @(negedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 224 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 225 `for (int s = 0; s < NUM_SAMPLES; s++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 226 `send_sample(s);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 227 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 228 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 229 `repeat (10) @(posedge clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 230 `weight_fh = $fopen("cifar10_spikingformer_head_onchip_learning_weights_after.mem", "w");`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 231 `bias_fh = $fopen("cifar10_spikingformer_head_onchip_learning_bias_after.mem", "w");`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 232 `for (int c = 0; c < 10; c++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 233 `for (int f = 0; f < FEATURE_DIM; f++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 234 `$fwrite(weight_fh, "%0d\n", $signed(dut.weight_mem[c * FEATURE_DIM + f]));`

This line touches the trained classifier weight store. In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine.

Line 235 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 236 `$fwrite(bias_fh, "%0d\n", $signed(dut.bias_mem[c]));`

This line touches the class-bias store. Bias values are part of the affine classifier score and are updated on chip in separate one-write states to keep synthesis clean.

Line 237 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 238 `$fclose(weight_fh);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 239 `$fclose(bias_fh);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 240 `$fclose(trace_fh);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 241 `$fclose(result_fh);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 242 `$display("tb_cifar10_spikingformer_head_onchip_learning samples=%0d correct=%0d updates=%0d errors=%0d", NUM_SAMPLES, correct, updates, errors);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 243 `if (errors != 0) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 244 `$fatal;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 245 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 246 `$finish;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 247 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 248 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 249 `initial begin`

This simulation/synthesis initialization block loads trained memories from generated .mem files. On FPGA, Vivado maps these initial values into BRAM/LUTRAM initialization data.

Line 250 `#2_000_000_000;`

This is a human-facing comment. It documents intent for the reader and is ignored by the compiler or interpreter.

Line 251 `$display("ERROR: tb_cifar10_spikingformer_head_onchip_learning watchdog timeout");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 252 `$fatal;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 253 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 254 `endmodule`

This closes the module definition. Synthesis stops adding logic to this circuit block at this point.

5.6 cifar10_neuromorphic\rtl\tb_cifar10_spikingformer_head_axi_activity.sv

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `` `timescale 1ns / 1ps ``

This simulator directive sets the time unit and precision used by testbenches and waveform timestamps. It keeps delays such as clock half-periods numerically meaningful in XSim.

Line 2 `` `default_nettype none ``

This disables implicit one-bit wires. It prevents spelling mistakes from silently creating unwanted hardware nets.

Line 3 `` `include "rtl/cifar10_spikingformer_head_params.svh" ``

This includes shared compile-time constants such as feature dimensions and fixed-point widths. The RTL and testbenches therefore agree on the same trained-model shape.

Line 4 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 5 `module tb_cifar10_spikingformer_head_axi_activity_base #(`

This begins a hardware module, which is the FPGA equivalent of a reusable circuit block. Everything until the matching endmodule describes ports, storage, combinational equations, and clocked behavior for this block.

Line 6 `parameter bit LEARNING_MODE = 1'b0,`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 7 `parameter int NUM_SAMPLES = `CIFAR10_SF_HEAD_NUM_SAMPLES,`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 8 `parameter int FEATURE_DIM = `CIFAR10_SF_HEAD_FEATURE_DIM,`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 9 `parameter int MAX_SAMPLES = 10000,`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 10 `parameter int MAX_FEATURE_WORDS = MAX_SAMPLES * FEATURE_DIM`

This is a compile-time hardware setting. Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead.

Line 11 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 12 `logic ap_clk = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 13 `logic ap_rst_n = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 14 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 15 `logic s_axis_feature_tvalid = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 16 `logic s_axis_feature_tready;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 17 `logic signed [15:0] s_axis_feature_tdata = '0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 18 `logic s_axis_feature_tlast = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 19 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 20 `logic train_enable = 1'b0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 21 `logic [3:0] train_label = 4'd0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 22 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 23 `logic m_axis_result_tvalid;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 24 `logic m_axis_result_tready = 1'b1;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 25 `logic [31:0] m_axis_result_tdata;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 26 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 27 `logic busy;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 28 `logic [31:0] debug_infer_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 29 `logic [31:0] debug_train_skip_count;`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 30 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 31 `logic signed [`CIFAR10_SF_HEAD_FEATURE_BITS-1:0] feature_mem [0:MAX_FEATURE_WORDS-1];`

This line touches the feature memory that holds one 384-dimensional fixed-point Spikingformer feature vector for the current image. Keeping the feature vector local avoids repeatedly streaming the same values for every class score.

Line 32 `logic [3:0] label_mem [0:MAX_SAMPLES-1];`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 33 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 34 `int correct = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 35 `int updates = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 36 `int errors = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 37 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 38 `cifar10_spikingformer_head_pynq_axi_stream_top dut (`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 39 `.ap_clk(ap_clk),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 40 `.ap_rst_n(ap_rst_n),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 41 `.s_axis_feature_tvalid(s_axis_feature_tvalid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 42 `.s_axis_feature_tready(s_axis_feature_tready),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 43 `.s_axis_feature_tdata(s_axis_feature_tdata),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 44 `.s_axis_feature_tlast(s_axis_feature_tlast),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 45 `.train_enable(train_enable),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 46 `.train_label(train_label),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 47 `.m_axis_result_tvalid(m_axis_result_tvalid),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 48 `.m_axis_result_tready(m_axis_result_tready),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 49 `.m_axis_result_tdata(m_axis_result_tdata),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 50 `.busy(busy),`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 51 `.debug_infer_skip_count(debug_infer_skip_count),`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 52 `.debug_train_skip_count(debug_train_skip_count)`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 53 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 54 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 55 `always #5 ap_clk = ~ap_clk;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 56 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 57 `task automatic require_file(input string path);`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 58 `int fh;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 59 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 60 `fh = $fopen(path, "r");`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 61 `if (fh == 0) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 62 `$display("ERROR: missing required file %s", path);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 63 `$fatal;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 64 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 65 `$fclose(fh);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 66 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 67 `endtask`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 68 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 69 `task automatic send_axis_word(`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 70 `input logic signed [15:0] value,`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 71 `input logic is_last`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 72 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 73 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 74 `@(negedge ap_clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 75 `s_axis_feature_tdata <= value;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 76 `s_axis_feature_tlast <= is_last;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 77 `s_axis_feature_tvalid <= 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 78 `do begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 79 `@(posedge ap_clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 80 `end while (!s_axis_feature_tready);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 81 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 82 `endtask`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 83 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 84 `task automatic send_sample(input int sample_idx);`

This line declares a module port. A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries.

Line 85 `logic [3:0] predicted;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 86 `logic update_applied;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 87 `int wait_cycles;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 88 `begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 89 `train_label <= label_mem[sample_idx];`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 90 `train_enable <= LEARNING_MODE;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 91 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 92 `for (int f = 0; f < FEATURE_DIM; f++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 93 `send_axis_word(`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 94 `feature_mem[sample_idx * FEATURE_DIM + f],`

This line touches the feature memory that holds one 384-dimensional fixed-point Spikingformer feature vector for the current image. Keeping the feature vector local avoids repeatedly streaming the same values for every class score.

Line 95 `f == FEATURE_DIM - 1`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 96 `);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 97 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 98 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 99 `@(negedge ap_clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 100 `s_axis_feature_tvalid <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 101 `s_axis_feature_tlast <= 1'b0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 102 `s_axis_feature_tdata <= '0;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 103 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 104 `wait_cycles = 0;`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 105 `while (!m_axis_result_tvalid && wait_cycles < 20000) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 106 `@(posedge ap_clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 107 `wait_cycles++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 108 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 109 `if (!m_axis_result_tvalid) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 110 `$display("ERROR: timeout waiting for AXI result at sample %0d", sample_idx);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 111 `errors++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 112 `end else begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 113 `predicted = m_axis_result_tdata[3:0];`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 114 `update_applied = m_axis_result_tdata[4];`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 115 `if (predicted == label_mem[sample_idx]) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 116 `correct++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 117 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 118 `if (update_applied) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 119 `updates++;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 120 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 121 `if (sample_idx < 8 || (sample_idx % 1000) == 0 || sample_idx == NUM_SAMPLES - 1) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 122 `$display("axi_activity_sample=%0d label=%0d pred=%0d correct=%0d update=%0d infer_skip=%0d train_skip=%0d",`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 123 `sample_idx, label_mem[sample_idx], predicted,`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 124 `predicted == label_mem[sample_idx], update_applied,`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 125 `debug_infer_skip_count, debug_train_skip_count);`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 126 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 127 `@(posedge ap_clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 128 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 129 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 130 `endtask`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 131 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 132 `initial begin`

This simulation/synthesis initialization block loads trained memories from generated .mem files. On FPGA, Vivado maps these initial values into BRAM/LUTRAM initialization data.

Line 133 `if (NUM_SAMPLES > MAX_SAMPLES) begin`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 134 `$display("ERROR: NUM_SAMPLES=%0d exceeds MAX_SAMPLES=%0d", NUM_SAMPLES, MAX_SAMPLES);`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 135 `$fatal;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 136 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 137 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 138 `require_file("rtl/cifar10_spikingformer_head_features.mem");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 139 `require_file("rtl/cifar10_spikingformer_head_labels.mem");`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 140 `$readmemh("rtl/cifar10_spikingformer_head_features.mem", feature_mem);`

This line touches the feature memory that holds one 384-dimensional fixed-point Spikingformer feature vector for the current image. Keeping the feature vector local avoids repeatedly streaming the same values for every class score.

Line 141 `$readmemh("rtl/cifar10_spikingformer_head_labels.mem", label_mem);`

This reads a hexadecimal memory file into an RTL array. It is how exported trained weights and biases become deterministic FPGA memory contents.

Line 142 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 143 `repeat (8) @(posedge ap_clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 144 `ap_rst_n <= 1'b1;`

This is a nonblocking register or memory assignment. The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge.

Line 145 `repeat (4) @(posedge ap_clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 146 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 147 `for (int s = 0; s < NUM_SAMPLES; s++) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 148 `send_sample(s);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 149 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 150 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 151 `repeat (10) @(posedge ap_clk);`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 152
```
$display("tb_cifar10_spikingformer_head_axi_activity learning=%0d samples=%0d correct=%0d updates=%0d errors=%0d infer_skip=%0d
train_skip=%0d",
```

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 153 `LEARNING_MODE, NUM_SAMPLES, correct, updates, errors,`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 154 `debug_infer_skip_count, debug_train_skip_count);`

This line exposes or updates a hardware activity counter. The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics.

Line 155 `if (errors != 0) begin`

This is a local compile-time or procedural assignment. In context it either defines a helper value or connects a temporary expression used by nearby hardware logic.

Line 156 `$fatal;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 157 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 158 `$finish;`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 159 `end`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 160 `endmodule`

This closes the module definition. Synthesis stops adding logic to this circuit block at this point.

Line 161 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 162 `module tb_cifar10_spikingformer_head_axi_inference_activity;`

This begins a hardware module, which is the FPGA equivalent of a reusable circuit block. Everything until the matching endmodule describes ports, storage, combinational equations, and clocked behavior for this block.

Line 163 `tb_cifar10_spikingformer_head_axi_activity_base #(.LEARNING_MODE(1'b0)) base();`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 164 `endmodule`

This closes the module definition. Synthesis stops adding logic to this circuit block at this point.

Line 165 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 166 `module tb_cifar10_spikingformer_head_axi_learning_activity;`

This begins a hardware module, which is the FPGA equivalent of a reusable circuit block. Everything until the matching endmodule describes ports, storage, combinational equations, and clocked behavior for this block.

Line 167 `tb_cifar10_spikingformer_head_axi_activity_base #(.LEARNING_MODE(1'b1)) base();`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

Line 168 `endmodule`

This closes the module definition. Synthesis stops adding logic to this circuit block at this point.

Line 169 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 170 `‘default_nettype wire`

This SystemVerilog line contributes to the structural or sequential description of the accelerator. Its effect is resolved by XSim for verification and by Vivado during synthesis.

5.7 cifar10_neuromorphic\export_spikingformer_head_accelerator.py

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `from __future__ import annotations`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 2 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 3 `import argparse`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 4 `import csv`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 5 `import json`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 6 `import os`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 7 `import sys`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 8 `from contextlib import nullcontext`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 9 `from functools import partial`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 10 `from pathlib import Path`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 11 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 12 `import matplotlib.pyplot as plt`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 13 `import numpy as np`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 14 `import torch`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 15 `import torch.nn as nn`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 16 `from matplotlib.backends.backend_pdf import PdfPages`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 17 `from timm.data import create_transform`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 18 `from timm.models import create_model`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 19 `from torch.utils.data import DataLoader, Subset`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 20 `from torchvision import datasets`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 21 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 22 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 23 `CIFAR10_CLASSES = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 24 `"airplane",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 25 `"automobile",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 26 `"bird",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 27 `"cat",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 28 `"deer",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 29 `"dog",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 30 `"frog",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 31 `"horse",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 32 `"ship",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 33 `"truck",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 34 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 35 `CIFAR10_MEAN = (0.4914, 0.4822, 0.4465)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 36 `CIFAR10_STD = (0.2470, 0.2435, 0.2616)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 37 `NUM_CLASSES = 10`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 38 `FEATURE_DIM = 384`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 39 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 40 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 41 `def repo_root() -> Path:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 42 `return Path(__file__).resolve().parents[1]`

This uses path objects so generated artifacts are addressed consistently across Windows shells and Python code.

Line 43 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 44 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 45 `def add_runtime_dlls() -> None:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 46 `env_root = repo_root() / "external_research" / "spikingformer_py310"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 47 `for path in (`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 48 `env_root / "Lib" / "site-packages" / "torch" / "lib",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 49 `env_root / "Lib" / "site-packages" / "nvidia" / "cuda_runtime" / "bin",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 50 `env_root / "Lib" / "site-packages" / "nvidia" / "cuda_nvrtc" / "bin",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 51 `):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 52 `if path.exists():`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 53 `try:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 54 `os.add_dll_directory(str(path))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 55 `except (AttributeError, OSError):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 56 `pass`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 57 `os.environ["PATH"] = str(path) + os.pathsep + os.environ.get("PATH", "")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 58 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 59 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 60 `def import_upstream_model() -> None:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 61 `upstream = repo_root() / "external_research" / "Spikingformer" / "cifar10"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 62 `if not upstream.exists():`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 63 `raise FileNotFoundError(f"Missing upstream Spikingformer source: {upstream}")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 64 `sys.path.insert(0, str(upstream))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 65 `import model # noqa: F401`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 66 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 67 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 68 `def signed_hex(value: int, bits: int) -> str:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 69 `return f"{value & ((1 << bits) - 1):0{bits // 4}x}"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 70 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 71 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 72 `def choose_scale(values: torch.Tensor, bits: int) -> float:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 73 `qmax = float((1 << (bits - 1)) - 1)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 74 `max_abs = float(values.detach().abs().max().cpu())`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 75 `return max(max_abs / qmax, 1.0e-12)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 76 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 77 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 78 `def quantize_signed(values: torch.Tensor, scale: float, bits: int) -> torch.Tensor:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 79 `qmin = -(1 << (bits - 1))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 80 `qmax = (1 << (bits - 1)) - 1`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 81 `return torch.round(values / scale).clamp(qmin, qmax).to(torch.int64)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 82 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 83 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 84 `def load_checkpoint(path: Path) -> dict[str, torch.Tensor]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 85 `checkpoint = torch.load(path, map_location="cpu")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 86 `state = checkpoint.get("state_dict", checkpoint)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 87 `clean: dict[str, torch.Tensor] = {}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 88 `for key, value in state.items():`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 89 `clean[key.removeprefix("module.")] = value`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 90 `return clean`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 91 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 92 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 93 `def build_model(device:  torch.device) -> nn.Module:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 94 `model = create_model(`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 95 `"Spikingformer",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 96 `pretrained=False,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 97 `drop_rate=0.0,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 98 `drop_path_rate=0.0,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 99 `drop_block_rate=None,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 100 `img_size_h=32,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 101 `img_size_w=32,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 102 `patch_size=4,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 103 `embed_dims=FEATURE_DIM,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 104 `num_heads=12,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 105 `mlp_ratios=4,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 106 `in_channels=3,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 107 `num_classes=NUM_CLASSES,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 108 `qkv_bias=False,`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 109 `norm_layer=partial(nn.LayerNorm, eps=1.0e-6),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 110 `depths=4,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 111 `sr_ratios=1,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 112 `T=4,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 113 `)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 114 `return model.to(device)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 115 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 116 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 117 `def make_loader(args:  argparse.Namespace) -> DataLoader:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 118 `transform = create_transform(`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 119 `input_size=(3, 32, 32),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 120 `is_training=False,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 121 `interpolation="bicubic",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 122 `mean=CIFAR10_MEAN,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 123 `std=CIFAR10_STD,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 124 `crop_pct=1.0,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 125 `)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 126 `dataset = datasets.CIFAR10(root=str(args.data_dir), train=False, download=True, transform=transform)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 127 `if args.samples > 0:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 128 `dataset = Subset(dataset, range(min(args.samples, len(dataset))))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 129 `return DataLoader(`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 130 `dataset,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 131 `batch_size=args.batch_size,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 132 `shuffle=False,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 133 `num_workers=args.workers,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 134 `pin_memory=torch.cuda.is_available(),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 135 `persistent_workers=args.workers > 0,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 136 `)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 137 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 138 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 139 `def autocast_context(device: torch.device, enabled: bool):`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 140 `if device.type == "cuda":`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 141 `return torch.amp.autocast(device_type="cuda", enabled=enabled)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 142 `return nullcontext()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 143 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 144 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 145 `def collect_features(args:  argparse.Namespace) -> tuple[torch.Tensor, torch.Tensor, torch.Tensor]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 146 `add_runtime_dlls()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 147 `import_upstream_model()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 148 `from spikingjelly.clock_driven import functional`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 149 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 150 `device_name = "cuda" if args.device == "auto" and torch.cuda.is_available() else ("cpu" if args.device == "auto" else args.device)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 151 `device = torch.device(device_name)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 152 `if device.type == "cuda":`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 153 `torch.backends.cudnn.benchmark = True`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 154 `model = build_model(device)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 155 `model.load_state_dict(load_checkpoint(args.checkpoint), strict=True)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 156 `model.eval()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 157 `loader = make_loader(args)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 158 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 159 `all_features: list[torch.Tensor] = []`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 160 `all_logits: list[torch.Tensor] = []`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 161 `all_labels: list[torch.Tensor] = []`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 162 `with torch.no_grad():`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 163 `for images, labels in loader:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 164 `images = images.to(device, non_blocking=True)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 165 `with autocast_context(device, args.amp):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 166 `time_images = images.unsqueeze(0).repeat(model.T, 1, 1, 1, 1)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 167 `time_features = model.forward_features(time_images)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 168 `features = time_features.mean(0)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 169 `logits = model.head(features)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 170 `functional.reset_net(model)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 171 `all_features.append(features.detach().float().cpu())`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 172 `all_logits.append(logits.detach().float().cpu())`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 173 `all_labels.append(labels.detach().cpu())`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 174 `return torch.cat(all_features), torch.cat(all_logits), torch.cat(all_labels)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 175 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 176 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 177 `def fold_batch_norm(weight: torch.Tensor, bias: torch.Tensor | None, bn_prefix: str, state: dict[str, torch.Tensor]) -> tuple[torch.Tensor, torch.Tensor]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 178 `out_channels = weight.shape[0]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 179 `if bias is None:`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 180 `bias = torch.zeros(out_channels, dtype=weight.dtype)`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 181 `gamma = state[f"{bn_prefix}.weight"]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 182 `beta = state[f"{bn_prefix}.bias"]`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 183 `mean = state[f"{bn_prefix}.running_mean"]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 184 `var = state[f"{bn_prefix}.running_var"]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 185 `scale = gamma / torch.sqrt(var + 1.0e-5)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 186 `folded_weight = weight * scale.reshape(-1, *([1] * (weight.ndim - 1)))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 187 `folded_bias = (bias - mean) * scale + beta`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 188 `return folded_weight, folded_bias`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 189 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 190 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 191 `def tensor_stats(tensor: torch.Tensor) -> dict[str, float | list[int]]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 192 `t = tensor.detach().float()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 193 `return {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 194 `"shape": list(t.shape),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 195 `"min": float(t.min()),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 196 `"max": float(t.max()),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 197 `"mean": float(t.mean()),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 198 `"std": float(t.std(unbiased=False)),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 199 `}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 200 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 201 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 202 `def full_checkpoint_mapping(args: argparse.Namespace) -> dict[str, object]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 203 `state = load_checkpoint(args.checkpoint)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 204 `layers: list[dict[str, object]] = []`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 205 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 206 `conv_bn_pairs = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 207 `("patch_embed.block0_conv", "patch_embed.block0_bn"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 208 `("patch_embed.block1_conv", "patch_embed.block1_bn"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 209 `("patch_embed.block2_conv", "patch_embed.block2_bn"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 210 `("patch_embed.block3_conv", "patch_embed.block3_bn"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 211 `("patch_embed.block4_conv", "patch_embed.block4_bn"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 212 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 213 `for block in range(4):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 214 `base = f"block.{block}"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 215 `conv_bn_pairs.extend(`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 216 `[`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 217 `(f"{base}.attn.q_conv", f"{base}.attn.q_bn"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 218 `(f"{base}.attn.k_conv", f"{base}.attn.k_bn"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 219 `(f"{base}.attn.v_conv", f"{base}.attn.v_bn"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 220 `(f"{base}.attn.proj_conv", f"{base}.attn.proj_bn"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 221 `(f"{base}.mlp.mlp1_conv", f"{base}.mlp.mlp1_bn"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 222 `(f"{base}.mlp.mlp2_conv", f"{base}.mlp.mlp2_bn"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 223 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 224 `)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 225 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 226 `total_weights = 0`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 227 `total_biases = 0`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 228 `for conv_prefix, bn_prefix in conv_bn_pairs:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 229 `weight = state[f"{conv_prefix}.weight"].float()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 230 `bias = state.get(f"{conv_prefix}.bias")`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 231 `if bias is not None:`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 232 `bias = bias.float()`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 233 `folded_weight, folded_bias = fold_batch_norm(weight, bias, bn_prefix, state)`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 234 `total_weights += folded_weight.numel()`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 235 `total_biases += folded_bias.numel()`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 236 `layers.append(`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 237 `{`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 238 `"name": conv_prefix,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 239 `"kind": "conv_or_pointwise_with_batchnorm_folded",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 240 `"weight": tensor_stats(folded_weight),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 241 `"bias": tensor_stats(folded_bias),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 242 `"weight_values": int(folded_weight.numel()),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 243 `"bias_values": int(folded_bias.numel()),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 244 `}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 245 `)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 246 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 247 `head_weight = state["head.weight"].float()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 248 `head_bias = state["head.bias"].float()`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 249 `total_weights += head_weight.numel()`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 250 `total_biases += head_bias.numel()`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 251 `layers.append(`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 252 `{`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 253 `"name": "head",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 254 `"kind": "linear_classifier",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 255 `"weight": tensor_stats(head_weight),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 256 `"bias": tensor_stats(head_bias),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 257 `"weight_values": int(head_weight.numel()),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 258 `"bias_values": int(head_bias.numel()),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 259 `}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 260 `)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 261 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 262 `return {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 263 `"checkpoint": str(args.checkpoint),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 264 `"checkpoint_metric_top1": None,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 265 `"checkpoint_metric_note": "Current measured accuracy is written after export as exported_float_top1_pct and exported_fixed_top1_pct.",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 266 `"source_model": "external_research/Spikingformer/cifar10/model.py",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 267 `"topology": {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 268 `"time_steps": 4,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 269 `"image": [3, 32, 32],`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 270 `"tokenizer": "5 convolution stages with LIF gates after stages 0-3 and 2 max-pool stages",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 271 `"transformer_blocks": 4,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 272 `"embed_dim": FEATURE_DIM,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 273 `"attention_heads": 12,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 274 `"mlp_ratio": 4,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 275 `"classifier": [FEATURE_DIM, NUM_CLASSES],`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 276 `},`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 277 `"mapped_layers": layers,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 278 `"parameter_counts": {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 279 `"weights": int(total_weights),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 280 `"biases": int(total_biases),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 281 `"total": int(total_weights + total_biases),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 282 `"int8_weight_bytes": int(total_weights),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 283 `"int16_weight_bytes": int(total_weights * 2),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 284 `"float32_checkpoint_bytes": int((total_weights + total_biases) * 4),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 285 `},`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 286 `"implementation_note": (`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 287 `"The shipped head RTL consumes fixed-point Spikingformer features and the trained classifier weights. "`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 288 `"The manifest above enumerates every trained backbone tensor that must be implemented by a DDR-backed "`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 289 `"streaming convolution/attention engine for a full bitstream-resident Spikingformer."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 290 `),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 291 `}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 292 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 293 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 294 `def write_head_memories(`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 295 `args: argparse.Namespace,`

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 296 `features: torch.Tensor,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 297 `logits: torch.Tensor,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 298 `labels: torch.Tensor,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 299 `) -> dict[str, object]:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 300 `state = load_checkpoint(args.checkpoint)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 301 `head_weight = state["head.weight"].float()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 302 `head_bias = state["head.bias"].float()`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 303 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 304 `feature_scale = choose_scale(features, args.feature_bits)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 305 `weight_scale = choose_scale(head_weight, args.weight_bits)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 306 `product_scale = feature_scale * weight_scale`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 307 `features_q = quantize_signed(features, feature_scale, args.feature_bits)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 308 `weights_q = quantize_signed(head_weight, weight_scale, args.weight_bits)`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 309 `bias_q = torch.round(head_bias / product_scale).to(torch.int64)`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 310 `features_eff_q = features_q.clone()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 311 `if args.infer_skip_abs_thresh > 0:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 312 `features_eff_q[features_eff_q.abs() <= args.infer_skip_abs_thresh] = 0`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 313 `logits_q = features_eff_q @ weights_q.t() + bias_q`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 314 `pred_float = logits.argmax(dim=1)`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 315 `pred_q = logits_q.argmax(dim=1)`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 316 `correct_float = pred_float.eq(labels)`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 317 `correct_q = pred_q.eq(labels)`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 318 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 319 `rtl_dir = args.rtl_dir`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 320 `rtl_dir.mkdir(parents=True, exist_ok=True)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 321 `out_dir = args.out_dir`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 322 `out_dir.mkdir(parents=True, exist_ok=True)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 323 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 324 `with (rtl_dir / "cifar10_spikingformer_head_features.mem").open("w", encoding="ascii") as fh:`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 325 `for row in features_q.tolist():`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 326 `for value in row:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 327 `fh.write(signed_hex(int(value), args.feature_bits) + "\n")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 328 `with (rtl_dir / "cifar10_spikingformer_head_weights.mem").open("w", encoding="ascii") as fh:`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 329 `for row in weights_q.tolist():`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 330 `for value in row:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 331 `fh.write(signed_hex(int(value), args.weight_bits) + "\n")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 332 `with (rtl_dir / "cifar10_spikingformer_head_bias.mem").open("w", encoding="ascii") as fh:`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 333 `for value in bias_q.tolist():`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 334 `fh.write(signed_hex(int(value), 64) + "\n")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 335 `with (rtl_dir / "cifar10_spikingformer_head_labels.mem").open("w", encoding="ascii") as fh:`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 336 `for value in labels.tolist():`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 337 `fh.write(f"{int(value):01x}\n")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 338 `params = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 339 `f"`define CIFAR10_SF_HEAD_NUM_SAMPLES {features.shape[0]}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 340 `f"`define CIFAR10_SF_HEAD_FEATURE_DIM {FEATURE_DIM}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 341 `f"`define CIFAR10_SF_HEAD_NUM_CLASSES {NUM_CLASSES}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 342 `f"`define CIFAR10_SF_HEAD_FEATURE_BITS {args.feature_bits}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 343 `f"`define CIFAR10_SF_HEAD_WEIGHT_BITS {args.weight_bits}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 344 `"`define CIFAR10_SF_HEAD_ACC_BITS 64",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 345 `f"`define CIFAR10_SF_HEAD_TRACE_SAMPLES {args.trace_samples}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 346 `f"`define CIFAR10_SF_HEAD_INFER_SKIP_ABS_THRESH {args.infer_skip_abs_thresh}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 347 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 348 `(rtl_dir / "cifar10_spikingformer_head_params.svh").write_text("\n".join(params) + "\n", encoding="ascii")`

This writes machine-readable output for later reports. Keeping JSON/CSV artifacts makes the verification auditable.

Line 349 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 350 `fixed_ref = out_dir / "spikingformer_head_fixed_reference.csv"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 351 `with fixed_ref.open("w", newline="", encoding="utf-8") as fh:`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 352 `fieldnames = ["sample", "label", "float_pred", "fixed_pred", "float_correct", "fixed_correct"]`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 353 `fieldnames.extend([f"logit_q{c}" for c in range(NUM_CLASSES)])`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 354 `writer = csv.DictWriter(fh, fieldnames=fieldnames)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 355 `writer.writeheader()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 356 `for idx in range(features.shape[0]):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 357 `row = {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 358 `"sample": idx,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 359 `"label": int(labels[idx]),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 360 `"float_pred": int(pred_float[idx]),`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 361 `"fixed_pred": int(pred_q[idx]),`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 362 `"float_correct": int(correct_float[idx]),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 363 `"fixed_correct": int(correct_q[idx]),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 364 `}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 365 `for cls in range(NUM_CLASSES):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 366 `row[f"logit_q{cls}"] = int(logits_q[idx, cls])`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 367 `writer.writerow(row)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 368 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 369 `confusion = np.zeros((NUM_CLASSES, NUM_CLASSES), dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 370 `for label, pred in zip(labels.numpy(), pred_q.numpy()):`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 371 `confusion[int(label), int(pred)] += 1`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 372 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 373 `metrics: dict[str, object] = {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 374 `"checkpoint": str(args.checkpoint),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 375 `"samples": int(features.shape[0]),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 376 `"feature_dim": FEATURE_DIM,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 377 `"float_accuracy": float(correct_float.float().mean()),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 378 `"fixed_accuracy": float(correct_q.float().mean()),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 379 `"float_fixed_prediction_agreement": float(pred_float.eq(pred_q).float().mean()),`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 380 `"feature_scale": feature_scale,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 381 `"weight_scale": weight_scale,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 382 `"product_scale": product_scale,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 383 `"feature_bits": args.feature_bits,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 384 `"weight_bits": args.weight_bits,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 385 `"bias_bits": 64,`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 386 `"infer_skip_abs_thresh": args.infer_skip_abs_thresh,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 387 `"feature_mac_skip_fraction": float((features_q.abs() <= args.infer_skip_abs_thresh).float().mean()) if args.infer_skip_abs_thresh > 0 else 0.0,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 388 `"confusion": confusion.tolist(),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 389 `"files": {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 390 `"features": str(rtl_dir / "cifar10_spikingformer_head_features.mem"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 391 `"weights": str(rtl_dir / "cifar10_spikingformer_head_weights.mem"),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 392 `"bias": str(rtl_dir / "cifar10_spikingformer_head_bias.mem"),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 393 `"labels": str(rtl_dir / "cifar10_spikingformer_head_labels.mem"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 394 `"params": str(rtl_dir / "cifar10_spikingformer_head_params.svh"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 395 `"fixed_reference": str(fixed_ref),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 396 `},`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 397 `}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 398 `(out_dir / "spikingformer_head_mapping_metrics.json").write_text(json.dumps(metrics, indent=2), encoding="utf-8")`

This writes machine-readable output for later reports. Keeping JSON/CSV artifacts makes the verification auditable.

Line 399 `return metrics`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 400 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 401 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 402 `def write_report(args: argparse.Namespace, metrics: dict[str, object], mapping: dict[str, object]) -> None:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 403 `out_dir = args.out_dir`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 404 `confusion = np.array(metrics["confusion"], dtype=np.float64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 405 `norm_confusion = confusion / np.maximum(confusion.sum(axis=1, keepdims=True), 1.0)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 406 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 407 `plt.figure(figsize=(7, 6), dpi=160)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 408 `plt.imshow(norm_confusion, cmap="viridis", vmin=0, vmax=1)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 409 `plt.xticks(range(NUM_CLASSES), CIFAR10_CLASSES, rotation=45, ha="right")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 410 `plt.yticks(range(NUM_CLASSES), CIFAR10_CLASSES)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 411 `plt.colorbar(label="row-normalized count")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 412 `plt.tight_layout()`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 413 `plt.savefig(out_dir / "spikingformer_head_fixed_confusion.png")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 414 `plt.close()`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 415 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 416 `layer_names = [layer["name"] for layer in mapping["mapped_layers"]]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 417 `layer_counts = [layer["weight_values"] for layer in mapping["mapped_layers"]]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 418 `plt.figure(figsize=(12, 5), dpi=160)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 419 `plt.bar(range(len(layer_counts)), layer_counts)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 420 `plt.yscale("log")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 421 `plt.xticks(range(len(layer_names)), layer_names, rotation=90, fontsize=6)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 422 `plt.ylabel("trained weight values (log scale)")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 423 `plt.tight_layout()`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 424 `plt.savefig(out_dir / "spikingformer_weight_inventory.png")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 425 `plt.close()`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 426 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 427 `with PdfPages(out_dir / "spikingformer_head_accelerator_report.pdf") as pdf:`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 428 `fig = plt.figure(figsize=(10, 7), dpi=160)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 429 `fig.text(0.06, 0.93, "Trained Spikingformer Weight Mapping", fontsize=18, weight="bold")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 430 `lines = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 431 `f"Checkpoint: {Path(str(metrics['checkpoint'])).name}",`

This uses path objects so generated artifacts are addressed consistently across Windows shells and Python code.

Line 432 `f"Samples exported: {metrics['samples']}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 433 `f"Float top-1 on exported set:  {float(metrics['float_accuracy']) * 100:.2f}%",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 434 `f"Fixed-point head top-1:  {float(metrics['fixed_accuracy']) * 100:.2f}%",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 435 `f"Float/fixed prediction agreement:  {float(metrics['float_fixed_prediction_agreement']) * 100:.2f}%",`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 436 `f"Feature gate:  abs(feature) <= {metrics['infer_skip_abs_thresh']} maps to zero",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 437 `f"Feature-MAC skip opportunity:  {float(metrics['feature_mac_skip_fraction']) * 100:.2f}%",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 438 `f"Feature scale:  {metrics['feature_scale']:.8g}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 439 `f"Weight scale:  {metrics['weight_scale']:.8g}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 440 `f"Mapped trained parameters inventoried:  {mapping['parameter_counts']['total']:,}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 441 `"RTL scope in this run:  trained 384-to-10 Spikingformer classifier head over exported backbone features.",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 442 `"Full backbone scope captured in manifest:  tokenizer convs, q/k/v/proj attention, MLP, BN-folded weights.",`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 443 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 444 `fig.text(0.06, 0.83, "\n".join(lines), fontsize=11, va="top")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 445 `pdf.savefig(fig)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 446 `plt.close(fig)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 447 `for name in ["spikingformer_head_fixed_confusion.png", "spikingformer_weight_inventory.png"]:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 448 `img = plt.imread(out_dir / name)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 449 `fig, ax = plt.subplots(figsize=(10, 7), dpi=160)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 450 `ax.imshow(img)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 451 `ax.axis("off")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 452 `pdf.savefig(fig)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 453 `plt.close(fig)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 454 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 455 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 456 `def parse_args() -> argparse.Namespace:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 457 `root = repo_root()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 458 `parser = argparse.ArgumentParser(description="Export trained Spikingformer CIFAR-10 weights/features for RTL acceleration.")`

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 459
```
parser.add_argument("-checkpoint", type=Path, default=root / "cifar10_neuromorphic" / "outputs_upstream_spikingformer" / "spikingformer_4_384_400e" / "model_best.pth.tar")
```

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 460
```
parser.add_argument("-data-dir", type=Path, default=root / "datasets")
```

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 461
```
parser.add_argument("-rtl-dir", type=Path, default=root / "cifar10_neuromorphic" / "rtl")
```

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 462
```
parser.add_argument("-out-dir", type=Path, default=root / "cifar10_neuromorphic" / "outputs_spikingformer_head_accelerator")
```

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 463
```
parser.add_argument("-samples", type=int, default=10000)
```

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 464
```
parser.add_argument("-trace-samples", type=int, default=8)
```

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 465
```
parser.add_argument("-batch-size", type=int, default=64)
```

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 466
```
parser.add_argument("-workers", type=int, default=4)
```

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 467
```
parser.add_argument("-device", type=str, default="auto")
```

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 468
```
parser.add_argument("-amp", action=argparse.BooleanOptionalAction, default=True)
```

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 469
```
parser.add_argument("-feature-bits", type=int, default=16)
```

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 470
```
parser.add_argument("-weight-bits", type=int, default=16)
```

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 471
```
parser.add_argument("-infer-skip-abs-thresh", type=int, default=2048)
```

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 472 `return parser.parse_args()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 473 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 474 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 475 `def main() -> None:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 476 `args = parse_args()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 477 `features, logits, labels = collect_features(args)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 478 `mapping = full_checkpoint_mapping(args)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 479 `args.out_dir.mkdir(parents=True, exist_ok=True)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 480 `metrics = write_head_memories(args, features, logits, labels)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 481 `mapping["exported_float_top1_pct"] = float(metrics["float_accuracy"]) * 100.0`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 482 `mapping["exported_fixed_top1_pct"] = float(metrics["fixed_accuracy"]) * 100.0`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 483 `mapping["infer_skip_abs_thresh"] = int(metrics["infer_skip_abs_thresh"])`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 484 `mapping["feature_mac_skip_fraction_pct"] = float(metrics["feature_mac_skip_fraction"]) * 100.0`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 485 `(args.out_dir / "spikingformer_full_checkpoint_mapping.json").write_text(json.dumps(mapping, indent=2), encoding="utf-8")`

This writes machine-readable output for later reports. Keeping JSON/CSV artifacts makes the verification auditable.

Line 486 `write_report(args, metrics, mapping)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 487 `print(json.dumps({k:  v for k, v in metrics.items() if k not in {"confusion", "files"}}, indent=2))`

This writes machine-readable output for later reports. Keeping JSON/CSV artifacts makes the verification auditable.

Line 488 `print(f"outputs={args.out_dir}")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 489 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 490 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 491 `if __name__ == "__main__":`

This Python idiom runs main() only when the file is executed as a script. It still allows the file's functions to be imported for tests or reuse.

Line 492 `main()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

5.8 cifar10_neuromorphic\validate_spikingformer_head_rtl.py

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `from __future__ import annotations`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 2 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 3 `import argparse`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 4 `import csv`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 5 `import json`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 6 `from pathlib import Path`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 7 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 8 `import matplotlib.pyplot as plt`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 9 `import numpy as np`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 10 `from matplotlib.backends.backend_pdf import PdfPages`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 11 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 12 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 13 `NUM_CLASSES = 10`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 14 `FEATURE_DIM = 384`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 15 `INFER_SKIP_ABS_THRESH = 2048`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 16 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 17 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 18 `def repo_root() -> Path:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 19 `return Path(__file__).resolve().parents[1]`

This uses path objects so generated artifacts are addressed consistently across Windows shells and Python code.

Line 20 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 21 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 22 `def load_rows(path:  Path) -> list[dict[str, str]]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 23 `with path.open("r", newline="", encoding="utf-8") as fh:`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 24 `return list(csv.DictReader(fh))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 25 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 26 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 27 `def signed_hex_to_int(token: str, bits: int) -> int:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 28 `value = int(token.strip(), 16)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 29 `sign = 1 << (bits - 1)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 30 `return value - (1 << bits) if value & sign else value`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 31 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 32 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 33 `def load_mem(path: Path, bits: int) -> list[int]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 34 `return [signed_hex_to_int(line, bits) for line in path.read_text(encoding="ascii").splitlines() if line.strip()]`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 35 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 36 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 37 `def fixed_reference_from_mem(rtl_dir: Path, sample_count: int) -> tuple[np.ndarray, np.ndarray, np.ndarray, float]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 38 `features_flat = np.array(load_mem(rtl_dir / "cifar10_spikingformer_head_features.mem", 16), dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 39 `weights = np.array(load_mem(rtl_dir / "cifar10_spikingformer_head_weights.mem", 16), dtype=np.int64).reshape(NUM_CLASSES, FEATURE_DIM)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 40 `bias = np.array(load_mem(rtl_dir / "cifar10_spikingformer_head_bias.mem", 64), dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 41 `labels = np.array([int(line.strip(), 16) for line in (rtl_dir / "cifar10_spikingformer_head_labels.mem").read_text(encoding="ascii").splitlines() if line.strip()], dtype=np.int64)`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 42 `features = features_flat[: sample_count * FEATURE_DIM].reshape(sample_count, FEATURE_DIM)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 43 `if INFER_SKIP_ABS_THRESH > 0:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 44 `skipped = np.abs(features) <= INFER_SKIP_ABS_THRESH`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 45 `features = features.copy()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 46 `features[skipped] = 0`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 47 `skip_fraction = float(skipped.mean())`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 48 `else:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 49 `skip_fraction = 0.0`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 50 `scores = features @ weights.T + bias`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 51 `preds = scores.argmax(axis=1)`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 52 `return labels[:sample_count], scores, preds, skip_fraction`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 53 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 54 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 55 `def write_report(args:  argparse.Namespace) -> dict[str, object]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 56 `rtl_rows = load_rows(args.results_csv)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 57 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 58 `labels = np.array([int(r["label"]) for r in rtl_rows], dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 59 `preds = np.array([int(r["predicted"]) for r in rtl_rows], dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 60 `correct = preds == labels`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 61 `rtl_logits = np.array([[int(r[f"score{c}"]) for c in range(NUM_CLASSES)] for r in rtl_rows], dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 62 `labels_ref, ref_logits, pred_ref, skip_fraction = fixed_reference_from_mem(args.rtl_dir, len(rtl_rows))`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 63 `if not np.array_equal(labels, labels_ref):`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 64 `raise ValueError("RTL result labels do not match labels memory")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 65 `error = rtl_logits - ref_logits`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 66 `mismatch_count = int((error != 0).sum())`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 67 `pred_mismatch = int((preds != pred_ref).sum())`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 68 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 69 `confusion = np.zeros((NUM_CLASSES, NUM_CLASSES), dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 70 `for label, pred in zip(labels, preds):`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 71 `confusion[label, pred] += 1`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 72 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 73 `metrics = {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 74 `"samples": int(len(rtl_rows)),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 75 `"accuracy": float(correct.mean()) if len(rtl_rows) else 0.0,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 76 `"correct": int(correct.sum()),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 77 `"score_mismatch_count": mismatch_count,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 78 `"score_exact_match": mismatch_count == 0,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 79 `"prediction_mismatch_count": pred_mismatch,`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 80 `"prediction_exact_match": pred_mismatch == 0,`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 81 `"max_abs_score_error": int(np.abs(error).max()) if error.size else 0,`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 82 `"infer_skip_abs_thresh": INFER_SKIP_ABS_THRESH,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 83 `"feature_mac_skip_fraction": skip_fraction,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 84 `"results_csv": str(args.results_csv),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 85 `"reference": "recomputed from current RTL .mem files",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 86 `}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 87 `args.out_dir.mkdir(parents=True, exist_ok=True)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 88 `(args.out_dir / "spikingformer_head_rtl_validation.json").write_text(json.dumps(metrics, indent=2), encoding="utf-8")`

This writes machine-readable output for later reports. Keeping JSON/CSV artifacts makes the verification auditable.

Line 89 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 90 `plt.figure(figsize=(10, 5), dpi=160)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 91 `plt.imshow(error, aspect="auto", cmap="coolwarm")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 92 `plt.xlabel("class")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 93 `plt.ylabel("sample")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 94 `plt.colorbar(label="RTL score - fixed reference")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 95 `plt.tight_layout()`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 96 `plt.savefig(args.out_dir / "spikingformer_head_rtl_score_error.png")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 97 `plt.close()`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 98 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 99 `norm_confusion = confusion / np.maximum(confusion.sum(axis=1, keepdims=True), 1)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 100 `plt.figure(figsize=(7, 6), dpi=160)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 101 `plt.imshow(norm_confusion, cmap="viridis", vmin=0, vmax=1)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 102 `plt.xlabel("predicted")`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 103 `plt.ylabel("label")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 104 `plt.colorbar(label="row-normalized count")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 105 `plt.tight_layout()`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 106 `plt.savefig(args.out_dir / "spikingformer_head_rtl_confusion.png")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 107 `plt.close()`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 108 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 109 `with PdfPages(args.out_dir / "spikingformer_head_rtl_validation.pdf") as pdf:`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 110 `fig = plt.figure(figsize=(10, 7), dpi=160)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 111 `fig.text(0.06, 0.92, "Spikingformer Head RTL Validation", fontsize=18, weight="bold")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 112 `lines = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 113 `f"Samples: {metrics['samples']}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 114 `f"RTL top-1: {metrics['accuracy'] * 100:.2f}%",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 115 `f"Correct: {metrics['correct']} / {metrics['samples']}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 116 `f"Score exact match: {metrics['score_exact_match']}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 117 `f"Score mismatches: {metrics['score_mismatch_count']}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 118 `f"Prediction exact match: {metrics['prediction_exact_match']}",`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 119 `f"Prediction mismatches: {metrics['prediction_mismatch_count']}",`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 120 `f"Max absolute score error: {metrics['max_abs_score_error']}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 121 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 122 `fig.text(0.06, 0.82, "\n".join(lines), fontsize=12, va="top")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 123 `pdf.savefig(fig)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 124 `plt.close(fig)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 125 `for name in ["spikingformer_head_rtl_score_error.png", "spikingformer_head_rtl_confusion.png"]:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 126 `img = plt.imread(args.out_dir / name)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 127 `fig, ax = plt.subplots(figsize=(10, 7), dpi=160)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 128 `ax.imshow(img)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 129 `ax.axis("off")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 130 `pdf.savefig(fig)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 131 `plt.close(fig)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 132 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 133 `return metrics`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 134 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 135 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 136 `def parse_args() -> argparse.Namespace:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 137 `root = repo_root()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 138 `parser = argparse.ArgumentParser(description="Validate Spikingformer head RTL against exported fixed-point reference.")`

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 139 `parser.add_argument("-results-csv", type=Path, default=root / "cifar10_neuromorphic" / "cifar10_spikingformer_head_results.csv")`

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 140 `parser.add_argument("-rtl-dir", type=Path, default=root / "cifar10_neuromorphic" / "rtl")`

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 141 `parser.add_argument("-out-dir", type=Path, default=root / "cifar10_neuromorphic" / "outputs_spikingformer_head_accelerator")`

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 142 `return parser.parse_args()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 143 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 144 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 145 `def main() -> None:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 146 `metrics = write_report(parse_args())`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 147 `print(json.dumps(metrics, indent=2))`

This writes machine-readable output for later reports. Keeping JSON/CSV artifacts makes the verification auditable.

Line 148 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 149 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 150 `if __name__ == "__main__":`

This Python idiom runs main() only when the file is executed as a script. It still allows the file's functions to be imported for tests or reuse.

Line 151 `main()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

5.9 cifar10_neuromorphic\validate_spikingformer_head_onchip_learning.py

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `from __future__ import annotations`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 2 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 3 `import argparse`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 4 `import csv`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 5 `import json`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 6 `from pathlib import Path`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 7 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 8 `import numpy as np`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 9 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 10 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 11 `NUM_CLASSES = 10`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 12 `FEATURE_DIM = 384`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 13 `LEARN_SHIFT = 14`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 14 `BIAS_LEARN_DELTA = 1`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 15 `LEARN_MARGIN_Q = 268435456`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 16 `INFER_SKIP_ABS_THRESH = 2048`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 17 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 18 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 19 `def repo_root() -> Path:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 20 `return Path(__file__).resolve().parents[1]`

This uses path objects so generated artifacts are addressed consistently across Windows shells and Python code.

Line 21 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 22 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 23 `def signed_hex_to_int(token: str, bits: int) -> int:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 24 `value = int(token.strip(), 16)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 25 `sign = 1 << (bits - 1)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 26 `return value - (1 << bits) if value & sign else value`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 27 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 28 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 29 `def load_mem(path: Path, bits: int) -> list[int]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 30 `return [signed_hex_to_int(line, bits) for line in path.read_text(encoding="ascii").splitlines() if line.strip()]`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 31 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 32 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 33 `def clamp_i16(value: int) -> int:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 34 `return max(-32768, min(32767, value))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 35 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 36 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 37 `def effective_features(features: np.ndarray) -> np.ndarray:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 38 `if INFER_SKIP_ABS_THRESH <= 0:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 39 `return features`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 40 `out = features.copy()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 41 `out[np.abs(out) <= INFER_SKIP_ABS_THRESH] = 0`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 42 `return out`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 43 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 44 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 45 `def load_rows(path: Path) -> list[dict[str, str]]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 46 `with path.open("r", newline="", encoding="utf-8") as fh:`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 47 `return list(csv.DictReader(fh))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 48 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 49 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 50 `def simulate_online_learning(rtl_dir: Path, sample_count: int) -> tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 51 `features_flat = load_mem(rtl_dir / "cifar10_spikingformer_head_features.mem", 16)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 52
```
labels = np.array([int(line.strip(), 16) for line in (rtl_dir /
"cifar10_spikingformer_head_labels.mem").read_text(encoding="ascii").splitlines() if line.strip()], dtype=np.int64)
```

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 53
```
weights = np.array(load_mem(rtl_dir / "cifar10_spikingformer_head_weights.mem", 16), dtype=np.int64).reshape(NUM_CLASSES, FEATURE_DIM)
```

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 54
```
bias = np.array(load_mem(rtl_dir / "cifar10_spikingformer_head_bias.mem", 64), dtype=np.int64)
```

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 55
```
features = np.array(features_flat[:  sample_count * FEATURE_DIM], dtype=np.int64).reshape(sample_count, FEATURE_DIM)
```

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 56
```
blank line
```

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 57
```
scores_out = np.zeros((sample_count, NUM_CLASSES), dtype=np.int64)
```

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 58
```
preds = np.zeros(sample_count, dtype=np.int64)
```

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 59
```
update_applied = np.zeros(sample_count, dtype=np.int64)
```

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 60
```
update_count = np.zeros(sample_count, dtype=np.int64)
```

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 61
```
count = 0
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 62
```
blank line
```

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 63
```
for sample_idx in range(sample_count):
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 64
```
x = effective_features(features[sample_idx])
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 65 `scores = bias + weights @ x`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 66 `pred = int(scores.argmax())`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 67 `label = int(labels[sample_idx])`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 68 `scores_out[sample_idx] = scores`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 69 `preds[sample_idx] = pred`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 70 `if pred != label and (int(scores[pred]) - int(scores[label])) <= LEARN_MARGIN_Q:`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 71 `delta = np.array([int(v) » LEARN_SHIFT for v in x], dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 72 `weights[label] = np.array([clamp_i16(int(w) + int(d)) for w, d in zip(weights[label], delta)], dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 73 `weights[pred] = np.array([clamp_i16(int(w) - int(d)) for w, d in zip(weights[pred], delta)], dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 74 `bias[label] += BIAS_LEARN_DELTA`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 75 `bias[pred] -= BIAS_LEARN_DELTA`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 76 `count += 1`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 77 `update_applied[sample_idx] = 1`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 78 `update_count[sample_idx] = count`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 79 `return scores_out, preds, update_applied, update_count`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 80 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 81 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 82 `def main() -> None:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 83 `parser = argparse.ArgumentParser(description="Validate on-chip learning RTL against fixed-point online-learning golden model.")`

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 84 `parser.add_argument("-rtl-dir", type=Path, default=repo_root() / "cifar10_neuromorphic" / "rtl")`

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 85 `parser.add_argument("-results-csv", type=Path, default=repo_root() / "cifar10_neuromorphic" / "cifar10_spikingformer_head_onchip_learning_results.csv")`

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 86 `parser.add_argument("-out-dir", type=Path, default=repo_root() / "cifar10_neuromorphic" / "outputs_spikingformer_head_accelerator")`

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 87 `args = parser.parse_args()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 88 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 89 `rows = load_rows(args.results_csv)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 90 `scores_ref, preds_ref, updates_ref, update_count_ref = simulate_online_learning(args.rtl_dir, len(rows))`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 91 `scores_rtl = np.array([[int(row[f"score{c}"]) for c in range(NUM_CLASSES)] for row in rows], dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 92 `preds_rtl = np.array([int(row["predicted"]) for row in rows], dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 93 `labels = np.array([int(row["label"]) for row in rows], dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 94 `updates_rtl = np.array([int(row["update_applied"]) for row in rows], dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 95 `update_count_rtl = np.array([int(row["update_count"]) for row in rows], dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 96 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 97 `score_error = scores_rtl - scores_ref`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 98 `metrics = {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 99 `"samples": int(len(rows)),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 100 `"accuracy": float((preds_rtl == labels).mean()) if rows else 0.0,`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 101 `"correct": int((preds_rtl == labels).sum()),`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 102 `"updates": int(updates_rtl.sum()),`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 103 `"score_mismatch_count": int((score_error != 0).sum()),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 104 `"max_abs_score_error": int(np.abs(score_error).max()) if score_error.size else 0,`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 105 `"prediction_mismatch_count": int((preds_rtl != preds_ref).sum()),`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 106 `"update_flag_mismatch_count": int((updates_rtl != updates_ref).sum()),`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 107 `"update_count_mismatch_count": int((update_count_rtl != update_count_ref).sum()),`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 108 `"learn_shift": LEARN_SHIFT,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 109 `"bias_learn_delta": BIAS_LEARN_DELTA,`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 110 `"learn_margin_q": LEARN_MARGIN_Q,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 111 `"infer_skip_abs_thresh": INFER_SKIP_ABS_THRESH,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 112 `}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 113 `args.out_dir.mkdir(parents=True, exist_ok=True)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 114 `(args.out_dir / "spikingformer_head_onchip_learning_validation.json").write_text(json.dumps(metrics, indent=2), encoding="utf-8")`

This writes machine-readable output for later reports. Keeping JSON/CSV artifacts makes the verification auditable.

Line 115 `print(json.dumps(metrics, indent=2))`

This writes machine-readable output for later reports. Keeping JSON/CSV artifacts makes the verification auditable.

Line 116 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 117 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 118 `if __name__ == "__main__":`

This Python idiom runs main() only when the file is executed as a script. It still allows the file's functions to be imported for tests or reuse.

Line 119 `main()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

5.10 cifar10_neuromorphic\generate_spikingformer_debug_reports.py

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `from __future__ import annotations`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 2 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 3 `import csv`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 4 `import json`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 5 `import math`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 6 `import re`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 7 `import shutil`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 8 `import subprocess`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 9 `from collections import Counter`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 10 `from pathlib import Path`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 11 `from typing import Iterable`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 12 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 13 `import matplotlib.pyplot as plt`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 14 `import numpy as np`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 15 `from matplotlib.backends.backend_pdf import PdfPages`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 16 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 17 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 18 `NUM_CLASSES = 10`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 19 `FEATURE_DIM = 384`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 20 `CLASS_NAMES = [str(i) for i in range(NUM_CLASSES)]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 21 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 22 `STATE_NAMES = {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 23 `0: "IDLE",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 24 `1: "LOAD",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 25 `2: "CLASS_INIT",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 26 `3: "MAC_READ",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 27 `4: "MAC_MUL",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 28 `5: "MAC_ACCUM",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 29 `6: "CLASS_DONE",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 30 `7: "RESULT",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 31 `8: "TRAIN_CHECK",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 32 `9: "TRAIN_READ_TRUE",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 33 `10: "TRAIN_CALC_TRUE",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 34 `11: "TRAIN_WRITE_TRUE",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 35 `12: "TRAIN_READ_PRED",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 36 `13: "TRAIN_CALC_PRED",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 37 `14: "TRAIN_WRITE_PRED",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 38 `15: "TRAIN_BIAS_TRUE",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 39 `16:  "TRAIN_BIAS_PRED",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 40 `}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 41 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 42 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 43 `def repo_root() -> Path:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 44 `return Path(__file__).resolve().parents[1]`

This uses path objects so generated artifacts are addressed consistently across Windows shells and Python code.

Line 45 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 46 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 47 `def project_dir() -> Path:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 48 `return Path(__file__).resolve().parent`

This uses path objects so generated artifacts are addressed consistently across Windows shells and Python code.

Line 49 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 50 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 51 `def out_dir() -> Path:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 52 `return project_dir() / "outputs_spikingformer_head_accelerator"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 53 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 54 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 55 `def debug_dir() -> Path:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 56 `return out_dir() / "deep_debug"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 57 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 58 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 59 `def read_csv_rows(path: Path) -> list[dict[str, str]]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 60 `with path.open("r", newline="", encoding="utf-8") as fh:`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 61 `return list(csv.DictReader(fh))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 62 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 63 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 64 `def col(rows: list[dict[str, str]], name: str, dtype=float) -> np.ndarray:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 65 `if not rows:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 66 `return np.array([], dtype=dtype)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 67 `return np.array([dtype(row[name]) for row in rows])`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 68 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 69 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 70 `def score_matrix(rows: list[dict[str, str]]) -> np.ndarray:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 71 `return np.array([[int(row[f"score{i}"]) for i in range(NUM_CLASSES)] for row in rows], dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 72 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 73 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 74 `def signed_hex_to_int(token: str, bits: int) -> int:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 75 `value = int(token.strip(), 16)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 76 `sign = 1 << (bits - 1)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 77 `return value - (1 << bits) if value & sign else value`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 78 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 79 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 80 `def read_hex_mem(path: Path, bits: int) -> np.ndarray:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 81 `values = []`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 82 `for line in path.read_text(encoding="ascii").splitlines():`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 83 `token = line.strip()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 84 `if token:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 85 `values.append(signed_hex_to_int(token, bits))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 86 `return np.array(values, dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 87 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 88 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 89 `def read_decimal_mem(path: Path) -> np.ndarray:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 90 `values = []`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 91 `for line in path.read_text(encoding="ascii").splitlines():`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 92 `token = line.strip()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 93 `if token:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 94 `values.append(int(token))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 95 `return np.array(values, dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 96 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 97 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 98 `def read_json(path: Path) -> dict:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 99 `return json.loads(path.read_text(encoding="utf-8")) if path.exists() else {}`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 100 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 101 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 102 `def parse_first_float(pattern: str, text: str, default: float = float("nan")) -> float:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 103 `match = re.search(pattern, text, flags=re.IGNORECASE | re.MULTILINE)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 104 `return float(match.group(1)) if match else default`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 105 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 106 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 107 `def parse_first_int(pattern: str, text: str, default: int = 0) -> int:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 108 `match = re.search(pattern, text, flags=re.IGNORECASE | re.MULTILINE)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 109 `return int(match.group(1)) if match else default`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 110 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 111 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 112 `def parse_vivado_reports(vivado_dir: Path) -> dict[str, float | int | str]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 113 `timing = (vivado_dir / "spikingformer_head_timing_summary.rpt").read_text(encoding="utf-8", errors="ignore")`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 114 `util = (vivado_dir / "spikingformer_head_utilization.rpt").read_text(encoding="utf-8", errors="ignore")`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 115 `power = (vivado_dir / "spikingformer_head_power.rpt").read_text(encoding="utf-8", errors="ignore")`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 116 `summary = (vivado_dir / "spikingformer_head_synth_summary.txt").read_text(encoding="utf-8", errors="ignore")`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 117 `return {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 118 `"summary": summary,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 119 `"wns_ns": parse_first_float(r"\n\s*([+-]?\d+\.\d+)\s+\d+\.\d+\s+\d+\s+\d+\s+[+-]?\d+\.\d+", timing),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 120 `"whs_ns": parse_first_float(r"\n\s*[+-]?\d+\.\d+\s+\d+\.\d+\s+\d+\s+\d+\s+([+-]?\d+\.\d+)", timing),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 121 `"lut": parse_first_int(r"\|\s*Slice LUTs\s*\|\s*(\d+)", util),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 122 `"regs": parse_first_int(r"\|\s*Slice Registers\s*\|\s*(\d+)", util),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 123 `"bram": parse_first_int(r"\|\s*Block RAM Tile\s*\|\s*(\d+)", util),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 124 `"dsp": parse_first_int(r"\|\s*DSPs\s*\|\s*(\d+)", util),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 125 `"lut_mem": parse_first_int(r"\|\s*LUT as Memory\s*\|\s*(\d+)", util),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 126 `"total_power_w": parse_first_float(r"\|\s*Total On-Chip Power \(W\)\s*\|\s*([0-9.]+)", power),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 127 `"dynamic_power_w": parse_first_float(r"\|\s*Dynamic \(W\)\s*\|\s*([0-9.]+)", power),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 128 `"static_power_w": parse_first_float(r"\|\s*Device Static \(W\)\s*\|\s*([0-9.]+)", power),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 129 `"clock_power_w": parse_first_float(r"\|\s*Clocks\s*\|\s*([0-9.<]+)", power.replace("<0.001", "0.0005")),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 130 `"logic_power_w": parse_first_float(r"\|\s*Slice Logic\s*\|\s*([0-9.<]+)", power.replace("<0.001", "0.0005")),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 131 `"signal_power_w": parse_first_float(r"\|\s*Signals\s*\|\s*([0-9.<]+)", power.replace("<0.001", "0.0005")),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 132 `"bram_power_w": parse_first_float(r"\|\s*Block RAM\s*\|\s*([0-9.<]+)", power.replace("<0.001", "0.0005")),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 133 `"dsp_power_w": parse_first_float(r"\|\s*DSPs\s*\|\s*([0-9.<]+)", power.replace("<0.001", "0.0005")),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 134 `"io_power_w": parse_first_float(r"\|\s*I/O\s*\|\s*([0-9.<]+)", power.replace("<0.001", "0.0005")),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 135 `}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 136 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 137 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 138 `def save_pdf_page(pdf: PdfPages, fig: plt.Figure, png_dir: Path, name: str) -> None:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 139 `png_dir.mkdir(parents=True, exist_ok=True)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 140 `fig.tight_layout()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 141 `fig.savefig(png_dir / f"{name}.png", dpi=180, bbox_inches="tight")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 142 `pdf.savefig(fig, bbox_inches="tight")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 143 `plt.close(fig)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 144 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 145 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 146 `def confusion(labels: np.ndarray, preds: np.ndarray) -> np.ndarray:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 147 `mat = np.zeros((NUM_CLASSES, NUM_CLASSES), dtype=np.int64)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 148 `for label, pred in zip(labels, preds):`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 149 `mat[int(label), int(pred)] += 1`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 150 `return mat`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 151 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 152 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 153 `def precision_recall_f1(conf: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 154 `tp = np.diag(conf).astype(float)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 155 `precision = tp / np.maximum(conf.sum(axis=0), 1)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 156 `recall = tp / np.maximum(conf.sum(axis=1), 1)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 157 `f1 = 2 * precision * recall / np.maximum(precision + recall, 1e-12)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 158 `return precision, recall, f1`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 159 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 160 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 161 `def margin(scores: np.ndarray, labels: np.ndarray) -> tuple[np.ndarray, np.ndarray]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 162 `sorted_scores = np.sort(scores, axis=1)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 163 `top1 = sorted_scores[:, -1]`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 164 `top2 = sorted_scores[:, -2]`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 165 `true_scores = scores[np.arange(scores.shape[0]), labels.astype(int)]`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 166 `return top1 - top2, top1 - true_scores`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 167 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 168 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 169 `def plot_text_page(title: str, lines: Iterable[str]) -> plt.Figure:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 170 `fig = plt.figure(figsize=(11, 8.5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 171 `fig.text(0.04, 0.95, title, fontsize=18, weight="bold", va="top")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 172 `y = 0.90`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 173 `for line in lines:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 174 `fig.text(0.05, y, line, fontsize=10.5, family="monospace", va="top")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 175 `y -= 0.035`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 176 `if y < 0.05:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 177 `break`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 178 `return fig`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 179 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 180 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 181 `def draw_box(ax, xy, w, h, text, color="#e8f1ff"):`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 182 `rect = plt.Rectangle(xy, w, h, facecolor=color, edgecolor="#1d3557", linewidth=1.6)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 183 `ax.add_patch(rect)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 184 `ax.text(xy[0] + w / 2, xy[1] + h / 2, text, ha="center", va="center", fontsize=9, wrap=True)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 185 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 186 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 187 `def plot_architecture() -> plt.Figure:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 188 `fig, ax = plt.subplots(figsize=(11, 8.5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 189 `ax.set_axis_off()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 190 `ax.set_xlim(0, 10)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 191 `ax.set_ylim(0, 7)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 192 `blocks = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 193 `((0.4, 5.4), 1.6, 0.7, "PYNQ PS / DMA\nAXI-stream features", "#eef6e8"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 194 `((2.5, 5.4), 1.7, 0.7, "AXI-stream wrapper\nstart, TLAST, result pack", "#e8f1ff"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 195 `((4.8, 5.4), 1.6, 0.7, "Feature SRAM\n384 x int16", "#fff0d9"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 196 `((7.0, 5.4), 1.8, 0.7, "Score output\nlabel + update status", "#eef6e8"),`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 197 `((1.0, 3.4), 1.7, 0.7, "Class loop\n10 classes", "#f6e8ff"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 198 `((3.2, 3.4), 1.7, 0.7, "Weight BRAM\n10 x 384 x int16", "#fff0d9"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 199 `((5.4, 3.4), 1.7, 0.7, "DSP MAC\nint16 x int16 -> acc64", "#e8f1ff"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 200 `((7.6, 3.4), 1.7, 0.7, "Argmax score\nprediction", "#e8f1ff"),`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 201 `((2.1, 1.3), 1.8, 0.7, "Margin-gated\non-chip update", "#ffe8e8"),`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 202 `((4.5, 1.3), 1.8, 0.7, "Registered\nsat add/sub", "#ffe8e8"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 203 `((6.9, 1.3), 1.8, 0.7, "Writeback\ntrue/pred rows + bias", "#ffe8e8"),`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 204 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 205 `for args in blocks:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 206 `draw_box(ax, *args)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 207 `arrows = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 208 `((2.0, 5.75), (2.5, 5.75)),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 209 `((4.2, 5.75), (4.8, 5.75)),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 210 `((6.4, 5.75), (7.0, 5.75)),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 211 `((4.0, 3.75), (5.4, 3.75)),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 212 `((7.1, 3.75), (7.6, 3.75)),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 213 `((7.6, 3.35), (3.0, 2.0)),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 214 `((3.9, 1.65), (4.5, 1.65)),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 215 `((6.3, 1.65), (6.9, 1.65)),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 216 `((7.8, 2.0), (4.1, 3.35)),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 217 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 218 `for start, end in arrows:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 219 `ax.annotate("", xy=end, xytext=start, arrowprops={"arrowstyle": "->", "lw": 1.5, "color": "#333"})`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 220 `ax.set_title("NeuraEdge CIFAR-10 Spikingformer Head Accelerator Dataflow", fontsize=16, weight="bold")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 221 `ax.text(0.5, 0.3, "All numbers in this report come from the generated RTL memories, XSim CSV traces, and Vivado reports in outputs_spikingformer_head_accelerator.", fontsize=9)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 222 `return fig`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 223 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 224 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 225 `def plot_fsm() -> plt.Figure:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 226 `fig, ax = plt.subplots(figsize=(11, 8.5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 227 `ax.set_axis_off()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 228 `ax.set_xlim(0, 11)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 229 `ax.set_ylim(0, 8)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 230 `fsm = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 231 `("IDLE", 0.5, 6.5), ("LOAD", 2.0, 6.5), ("CLASS_INIT", 3.7, 6.5),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 232 `("MAC_READ", 5.6, 6.5), ("MAC_MUL", 7.3, 6.5), ("MAC_ACCUM", 9.0, 6.5),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 233 `("CLASS_DONE", 5.5, 5.0), ("TRAIN_CHECK", 3.2, 3.6), ("RESULT", 8.0, 3.6),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 234 `("READ_TRUE", 0.7, 2.2), ("CALC_TRUE", 2.4, 2.2), ("WRITE_TRUE", 4.1, 2.2),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 235 `("READ_PRED", 5.8, 2.2), ("CALC_PRED", 7.5, 2.2), ("WRITE_PRED", 9.2, 2.2),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 236 `("BIAS_TRUE", 4.2, 0.9), ("BIAS_PRED", 6.2, 0.9),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 237 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 238 `positions = {}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 239 `for name, x, y in fsm:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 240 `positions[name] = (x, y)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 241 `draw_box(ax, (x, y), 1.35, 0.55, name, "#f6fbff" if "TRAIN" not in name and "TRUE" not in name and "PRED" not in name and "BIAS" not in name else "#fff1f1")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 242 `edges = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 243 `("IDLE", "LOAD"), ("LOAD", "CLASS_INIT"), ("CLASS_INIT", "MAC_READ"), ("MAC_READ", "MAC_MUL"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 244 `("MAC_MUL", "MAC_ACCUM"), ("MAC_ACCUM", "CLASS_DONE"), ("CLASS_DONE", "MAC_READ"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 245 `("CLASS_DONE", "TRAIN_CHECK"), ("TRAIN_CHECK", "RESULT"), ("RESULT", "IDLE"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 246 `("TRAIN_CHECK", "READ_TRUE"), ("READ_TRUE", "CALC_TRUE"), ("CALC_TRUE", "WRITE_TRUE"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 247 `("WRITE_TRUE", "READ_PRED"), ("READ_PRED", "CALC_PRED"), ("CALC_PRED", "WRITE_PRED"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 248 `("WRITE_PRED", "READ_TRUE"), ("WRITE_PRED", "BIAS_TRUE"), ("BIAS_TRUE", "BIAS_PRED"), ("BIAS_PRED", "RESULT"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 249 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 250 `for a, b in edges:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 251
```
ax.annotate("", xy=(positions[b][0], positions[b][1] + 0.27), xytext=(positions[a][0] + 1.35, positions[a][1] + 0.27),
arrowprops={"arrowstyle":  "->", "lw":  1.0})
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 252 `ax.set_title("RTL State Machine Including On-Chip Learning", fontsize=16, weight="bold")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 253 `return fig`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 254 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 255 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 256 `def generate_debug_pdf() -> None:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 257 `pdir = project_dir()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 258 `odir = out_dir()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 259 `ddir = debug_dir()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 260 `png_dir = ddir / "plots"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 261 `ddir.mkdir(parents=True, exist_ok=True)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 262 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 263 `inference_rows = read_csv_rows(pdir / "cifar10_spikingformer_head_results.csv")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 264 `learn_rows = read_csv_rows(pdir / "cifar10_spikingformer_head_onchip_learning_results.csv")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 265 `trace_rows = read_csv_rows(pdir / "cifar10_spikingformer_head_trace.csv")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 266 `learn_trace_rows = read_csv_rows(pdir / "cifar10_spikingformer_head_onchip_learning_trace.csv")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 267 `rtl_val = read_json(odir / "spikingformer_head_rtl_validation.json")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 268 `learn_val = read_json(odir / "spikingformer_head_onchip_learning_validation.json")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 269 `mapping = read_json(odir / "spikingformer_head_mapping_metrics.json")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 270 `vivado = parse_vivado_reports(odir / "vivado")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 271 `activity_power = read_json(odir / "activity_power" / "spikingformer_head_activity_power_summary.json")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 272 `infer_skip_abs_thresh = int(rtl_val.get("infer_skip_abs_thresh", mapping.get("infer_skip_abs_thresh", 0)) or 0)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 273 `feature_mac_skip_fraction = float(rtl_val.get("feature_mac_skip_fraction", mapping.get("feature_mac_skip_fraction", 0.0)) or 0.0)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 274 `rtl_infer_skip_count = int(inference_rows[-1].get("infer_skip_count", 0)) if inference_rows else 0`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 275 `learn_infer_skip_count = int(learn_rows[-1].get("infer_skip_count", 0)) if learn_rows else 0`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 276 `learn_train_skip_count = int(learn_rows[-1].get("train_skip_count", 0)) if learn_rows else 0`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 277 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 278 `labels = col(inference_rows, "label", int)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 279 `preds = col(inference_rows, "predicted", int)`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 280 `correct = col(inference_rows, "correct", int)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 281 `learn_preds = col(learn_rows, "predicted", int)`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 282 `learn_correct = col(learn_rows, "correct", int)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 283 `updates = col(learn_rows, "update_applied", int)`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 284 `update_count = col(learn_rows, "update_count", int)`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 285 `scores = score_matrix(inference_rows)`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 286 `learn_scores = score_matrix(learn_rows)`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 287 `conf = confusion(labels, preds)`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 288 `precision, recall, f1 = precision_recall_f1(conf)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 289 `margins, true_gaps = margin(scores, labels)`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 290 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 291 `rtl_dir = pdir / "rtl"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 292 `features = read_hex_mem(rtl_dir / "cifar10_spikingformer_head_features.mem", 16)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 293 `weights = read_hex_mem(rtl_dir / "cifar10_spikingformer_head_weights.mem", 16).reshape(NUM_CLASSES, FEATURE_DIM)`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 294 `bias = read_hex_mem(rtl_dir / "cifar10_spikingformer_head_bias.mem", 64)`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 295 `after_weights_path = pdir / "cifar10_spikingformer_head_onchip_learning_weights_after.mem"`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 296 `after_bias_path = pdir / "cifar10_spikingformer_head_onchip_learning_bias_after.mem"`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 297 `after_weights = read_decimal_mem(after_weights_path).reshape(NUM_CLASSES, FEATURE_DIM) if after_weights_path.exists() else weights.copy()`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 298 `after_bias = read_decimal_mem(after_bias_path) if after_bias_path.exists() else bias.copy()`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 299 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 300 `pdf_path = ddir / "neuraedge_spikingformer_deep_debug_report.pdf"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 301 `with PdfPages(pdf_path) as pdf:`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 302
```
summary_lines = [
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 303
```
f"RTL inference samples: {rtl_val.get('samples')} accuracy: {float(rtl_val.get('accuracy', float('nan'))):.4f} exact score match:
{rtl_val.get('score_exact_match')}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 304
```
f"On-chip learning samples: {learn_val.get('samples')} updates: {learn_val.get('updates')} score mismatches:
{learn_val.get('score_mismatch_count')}",
```

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 305
```
f"Activity gate: abs(feature) <= {infer_skip_abs_thresh} is forced to zero; measured feature-MAC skip fraction:
{feature_mac_skip_fraction * 100:.2f}%",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 306
```
f"RTL debug counters: inference skips={rtl_infer_skip_count:,}; learning-run inference skips={learn_infer_skip_count:,}; learning feature
skips={learn_train_skip_count:,}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 307
```
f"Vivado top: cifar10_spikingformer_head_pynq_axi_stream_top part: xc7z020clg400-1 clock: 10 ns",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 308
```
f"Timing: WNS={vivado['wns_ns']:.3f} ns WHS={vivado['whs_ns']:.3f} ns timing_met={'timing_met=1' in vivado['summary']}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 309
```
f"Utilization: LUT={vivado['lut']} FF={vivado['regs']} BRAM36={vivado['bram']} DSP={vivado['dsp']} LUTRAM={vivado['lut_mem']}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 310
```
f"Power: total={vivado['total_power_w']:.3f} W dynamic={vivado['dynamic_power_w']:.3f} W static={vivado['static_power_w']:.3f} W",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 311
```
f"SAIF power: inference dynamic={activity_power.get('inference_saif', {}).get('dynamic_w', float('nan')):.3f} W; learning
dynamic={activity_power.get('learning_saif', {}).get('dynamic_w', float('nan')):.3f} W; confidence={activity_power.get('inference_saif',
{}).get('confidence', 'missing')}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 312
```
f"Mapping metrics file fields: {', '.join(sorted(mapping.keys())[:10])}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 313
```
"Important interpretation: the RTL accelerates the trained Spikingformer feature-head path, gates low-magnitude feature activity, and
implements a margin-gated on-chip classifier update.",
```

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 314 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 315 `save_pdf_page(pdf, plot_text_page("Measured Build Summary", summary_lines), png_dir, "00_summary")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 316 `save_pdf_page(pdf, plot_architecture(), png_dir, "01_architecture")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 317 `save_pdf_page(pdf, plot_fsm(), png_dir, "02_fsm")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 318 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 319 `for name, rows in [("inference", trace_rows), ("onchip_learning", learn_trace_rows)]:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 320 `cycles = col(rows, "cycle", int)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 321 `fig, axes = plt.subplots(5, 1, figsize=(12, 9), sharex=True)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 322 `axes[0].step(cycles, col(rows, "state", int), where="post")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 323 `axes[0].set_ylabel("state")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 324 `axes[1].step(cycles, col(rows, "feature_valid", int), where="post", label="valid")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 325 `axes[1].step(cycles, col(rows, "feature_ready", int), where="post", label="ready")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 326 `axes[1].legend(loc="upper right")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 327 `axes[1].set_ylabel("feature")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 328 `axes[2].plot(cycles, col(rows, "feature_idx", int))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 329 `axes[2].set_ylabel("feature_idx")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 330 `axes[3].plot(cycles, col(rows, "class_idx", int))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 331 `axes[3].set_ylabel("class_idx")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 332 `axes[4].step(cycles, col(rows, "result_valid", int), where="post", label="result")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 333 `if "train_update_applied" in rows[0]:`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 334 `axes[4].step(cycles, col(rows, "train_update_applied", int), where="post", label="update")`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 335 `axes[4].legend(loc="upper right")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 336 `axes[4].set_xlabel("cycle")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 337 `axes[4].set_ylabel("flags")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 338 `fig.suptitle(f"Cycle-Level Trace From XSim CSV: {name}")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 339 `save_pdf_page(pdf, fig, png_dir, f"03_trace_{name}")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 340 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 341 `counts = Counter(col(rows, "state", int))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 342 `states = sorted(counts)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 343 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 344 `ax.bar([STATE_NAMES.get(int(s), str(s)) for s in states], [counts[s] for s in states])`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 345 `ax.tick_params(axis="x", rotation=45)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 346 `ax.set_ylabel("cycles in traced samples")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 347 `ax.set_title(f"FSM State Occupancy: {name}")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 348 `save_pdf_page(pdf, fig, png_dir, f"04_state_occupancy_{name}")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 349 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 350 `sample_idx = np.arange(len(labels))`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 351 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 352 `ax.plot(sample_idx, np.cumsum(correct) / np.maximum(sample_idx + 1, 1), label="inference")`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 353 `ax.plot(sample_idx, np.cumsum(learn_correct) / np.maximum(sample_idx + 1, 1), label="on-chip learning")`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 354 `ax.set_xlabel("sample")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 355 `ax.set_ylabel("cumulative accuracy")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 356 `ax.set_title("Cumulative Accuracy Over Full CIFAR-10 Test Stream")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 357 `ax.legend()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 358 `save_pdf_page(pdf, fig, png_dir, "05_cumulative_accuracy")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 359 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 360 `window = 200`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 361 `roll = np.convolve(correct, np.ones(window) / window, mode="valid")`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 362 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 363 `ax.plot(np.arange(len(roll)) + window, roll)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 364 `ax.set_xlabel("sample")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 365 `ax.set_ylabel(f"rolling accuracy ({window})")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 366 `ax.set_title("Rolling Accuracy Debug View")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 367 `save_pdf_page(pdf, fig, png_dir, "06_rolling_accuracy")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 368 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 369 `fig, ax = plt.subplots(figsize=(8, 7))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 370 `im = ax.imshow(conf, cmap="Blues")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 371 `ax.set_xlabel("predicted")`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 372 `ax.set_ylabel("label")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 373 `ax.set_title("Confusion Matrix")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 374 `ax.set_xticks(range(NUM_CLASSES), CLASS_NAMES)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 375 `ax.set_yticks(range(NUM_CLASSES), CLASS_NAMES)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 376 `for i in range(NUM_CLASSES):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 377 `for j in range(NUM_CLASSES):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 378 `ax.text(j, i, str(conf[i, j]), ha="center", va="center", fontsize=7)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 379 `fig.colorbar(im, ax=ax)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 380 `save_pdf_page(pdf, fig, png_dir, "07_confusion")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 381 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 382 `norm_conf = conf / np.maximum(conf.sum(axis=1, keepdims=True), 1)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 383 `fig, ax = plt.subplots(figsize=(8, 7))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 384 `im = ax.imshow(norm_conf, cmap="viridis", vmin=0, vmax=1)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 385 `ax.set_xlabel("predicted")`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 386 `ax.set_ylabel("label")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 387 `ax.set_title("Normalized Confusion Matrix")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 388 `ax.set_xticks(range(NUM_CLASSES), CLASS_NAMES)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 389 `ax.set_yticks(range(NUM_CLASSES), CLASS_NAMES)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 390 `fig.colorbar(im, ax=ax)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 391 `save_pdf_page(pdf, fig, png_dir, "08_confusion_normalized")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 392 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 393 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 394 `x = np.arange(NUM_CLASSES)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 395 `ax.bar(x - 0.25, precision, width=0.25, label="precision")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 396 `ax.bar(x, recall, width=0.25, label="recall")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 397 `ax.bar(x + 0.25, f1, width=0.25, label="F1")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 398 `ax.set_xticks(x, CLASS_NAMES)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 399 `ax.set_ylim(0, 1)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 400 `ax.set_title("Per-Class Precision / Recall / F1")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 401 `ax.legend()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 402 `save_pdf_page(pdf, fig, png_dir, "09_precision_recall_f1")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 403 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 404 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 405 `ax.hist(margins, bins=80, alpha=0.75)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 406 `ax.set_title("Top-1 Minus Top-2 Score Margin Histogram")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 407 `ax.set_xlabel("fixed-point score margin")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 408 `ax.set_ylabel("samples")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 409 `save_pdf_page(pdf, fig, png_dir, "10_top12_margin_hist")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 410 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 411 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 412 `ax.scatter(sample_idx[correct == 1], margins[correct == 1], s=3, alpha=0.35, label="correct")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 413 `ax.scatter(sample_idx[correct == 0], margins[correct == 0], s=8, alpha=0.75, label="wrong")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 414 `ax.set_xlabel("sample")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 415 `ax.set_ylabel("top1-top2 margin")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 416 `ax.set_title("Margin Timeline With Misclassification Overlay")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 417 `ax.legend()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 418 `save_pdf_page(pdf, fig, png_dir, "11_margin_timeline")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 419 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 420 `fig, ax = plt.subplots(figsize=(8, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 421 `ax.boxplot([margins[correct == 1], margins[correct == 0]], tick_labels=["correct", "wrong"], showfliers=False)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 422 `ax.set_ylabel("top1-top2 margin")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 423 `ax.set_title("Margins Split By Correctness")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 424 `save_pdf_page(pdf, fig, png_dir, "12_margin_box")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 425 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 426 `fig, ax = plt.subplots(figsize=(12, 6))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 427 `im = ax.imshow(scores[:80] / 1e9, aspect="auto", cmap="coolwarm")`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 428 `ax.set_xlabel("class")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 429 `ax.set_ylabel("sample")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 430 `ax.set_title("Score Heatmap:  First 80 Samples (billions of fixed-point units)")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 431 `ax.set_xticks(range(NUM_CLASSES), CLASS_NAMES)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 432 `fig.colorbar(im, ax=ax)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 433 `save_pdf_page(pdf, fig, png_dir, "13_score_heatmap")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 434 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 435 `diff = learn_scores - scores`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 436 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 437 `ax.hist(diff.reshape(-1), bins=80)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 438 `ax.set_title("Score Difference Caused By Online Weight Updates")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 439 `ax.set_xlabel("on-chip-learning score - fixed inference score")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 440 `save_pdf_page(pdf, fig, png_dir, "14_learning_score_diff")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 441 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 442 `update_samples = sample_idx[updates == 1]`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 443 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 444 `ax.vlines(update_samples, 0, 1, color="#c1121f")`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 445 `ax.set_ylim(0, 1.05)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 446 `ax.set_xlabel("sample")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 447 `ax.set_yticks([])`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 448 `ax.set_title("On-Chip Learning Update Events")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 449 `save_pdf_page(pdf, fig, png_dir, "15_update_events")`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 450 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 451 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 452 `ax.step(sample_idx, update_count, where="post")`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 453 `ax.set_xlabel("sample")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 454 `ax.set_ylabel("cumulative updates")`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 455 `ax.set_title("Hardware Update Counter")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 456 `save_pdf_page(pdf, fig, png_dir, "16_update_count")`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 457 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 458 `if "infer_skip_count" in inference_rows[0]:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 459 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 460 `ax.step(sample_idx, col(inference_rows, "infer_skip_count", int), where="post", label="inference run")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 461 `if "infer_skip_count" in learn_rows[0]:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 462 `ax.step(sample_idx, col(learn_rows, "infer_skip_count", int), where="post", label="learning run inference counter", alpha=0.75)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 463 `ax.set_xlabel("sample")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 464 `ax.set_ylabel("cumulative skipped feature-MACs")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 465 `ax.set_title("RTL Activity-Gate Skip Counter")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 466 `ax.legend()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 467 `save_pdf_page(pdf, fig, png_dir, "16b_skip_count")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 468 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 469 `if "train_skip_count" in learn_rows[0]:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 470 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 471 `ax.step(sample_idx, col(learn_rows, "train_skip_count", int), where="post")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 472 `ax.set_xlabel("sample")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 473 `ax.set_ylabel("cumulative skipped learning features")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 474 `ax.set_title("RTL On-Chip-Learning Feature Gate Counter")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 475 `save_pdf_page(pdf, fig, png_dir, "16c_train_skip_count")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 476 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 477 `if len(update_samples):`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 478 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 479 `width = 0.35`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 480 `upd_labels = labels[updates == 1]`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 481 `upd_preds = learn_preds[updates == 1]`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 482 `ax.bar(np.arange(len(update_samples)) - width / 2, upd_labels, width, label="true label")`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 483 `ax.bar(np.arange(len(update_samples)) + width / 2, upd_preds, width, label="predicted")`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 484 `ax.set_xticks(np.arange(len(update_samples)), update_samples, rotation=90)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 485 `ax.set_ylabel("class")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 486 `ax.set_xlabel("sample where update occurred")`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 487 `ax.set_title("Classes Involved In Each On-Chip Update")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 488 `ax.legend()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 489 `save_pdf_page(pdf, fig, png_dir, "17_update_classes")`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 490 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 491 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 492 `ax.hist(features, bins=100)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 493 `if infer_skip_abs_thresh > 0:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 494 `ax.axvline(-infer_skip_abs_thresh, color="#c1121f", linestyle="-", label="-gate threshold")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 495 `ax.axvline(infer_skip_abs_thresh, color="#c1121f", linestyle="-", label="+gate threshold")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 496 `ax.legend()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 497 `ax.set_title("Exported Spikingformer Feature Distribution")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 498 `ax.set_xlabel("int16 feature value")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 499 `save_pdf_page(pdf, fig, png_dir, "18_feature_hist")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 500 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 501 `abs_features = np.abs(features)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 502 `thresholds = np.array([0, 16, 32, 64, 128, 256, 512, 1024, 1536, 2048, 4096, 8192, 16384])`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 503 `skip_fracs = np.array([(abs_features <= t).mean() for t in thresholds], dtype=float)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 504 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 505 `ax.plot(thresholds, skip_fracs * 100.0, marker="o")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 506 `if infer_skip_abs_thresh > 0:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 507 `ax.axvline(infer_skip_abs_thresh, color="#c1121f", linestyle="-")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 508 `ax.text(infer_skip_abs_thresh, feature_mac_skip_fraction * 100.0, f" selected {feature_mac_skip_fraction * 100:.2f}%", va="center")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 509 `ax.set_xscale("symlog", linthresh=16)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 510 `ax.set_xlabel("abs(feature) gate threshold")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 511 `ax.set_ylabel("feature-MACs skipped (%)")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 512 `ax.set_title("Measured Feature Activity Gating Curve")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 513 `save_pdf_page(pdf, fig, png_dir, "18b_feature_gate_sweep")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 514 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 515 `first_eff = features[:FEATURE_DIM].copy()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 516 `if infer_skip_abs_thresh > 0:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 517 `first_eff[np.abs(first_eff) <= infer_skip_abs_thresh] = 0`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 518 `fig, ax = plt.subplots(figsize=(12, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 519 `ax.plot(features[:FEATURE_DIM], label="raw")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 520 `ax.plot(first_eff, label="after gate", alpha=0.8)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 521 `ax.axhline(0, color="black", linewidth=0.6)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 522 `ax.set_title("First Feature Vector Before/After Activity Gate")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 523 `ax.set_xlabel("feature index")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 524 `ax.set_ylabel("int16 value")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 525 `ax.legend()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 526 `save_pdf_page(pdf, fig, png_dir, "18c_first_feature_gate_overlay")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 527 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 528 `fig, ax = plt.subplots(figsize=(12, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 529 `ax.plot(features[:FEATURE_DIM])`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 530 `ax.set_title("First Exported 384-D Feature Vector")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 531 `ax.set_xlabel("feature index")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 532 `ax.set_ylabel("int16 value")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 533 `save_pdf_page(pdf, fig, png_dir, "19_first_feature_vector")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 534 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 535 `fig, ax = plt.subplots(figsize=(12, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 536 `im = ax.imshow(features[:FEATURE_DIM].reshape(16, 24), aspect="auto", cmap="coolwarm")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 537 `ax.set_title("First Feature Vector Reshaped 16 x 24 For Inspection")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 538 `fig.colorbar(im, ax=ax)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 539 `save_pdf_page(pdf, fig, png_dir, "20_first_feature_grid")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 540 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 541 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 542 `ax.hist(weights.reshape(-1), bins=100)`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 543 `ax.set_title("Classifier Weight Distribution Before On-Chip Updates")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 544 `ax.set_xlabel("int16 weight")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 545 `save_pdf_page(pdf, fig, png_dir, "21_weight_hist")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 546 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 547 `fig, ax = plt.subplots(figsize=(12, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 548 `im = ax.imshow(weights, aspect="auto", cmap="coolwarm")`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 549 `ax.set_xlabel("feature index")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 550 `ax.set_ylabel("class")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 551 `ax.set_title("Classifier Weight Matrix:  Class x Feature")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 552 `fig.colorbar(im, ax=ax)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 553 `save_pdf_page(pdf, fig, png_dir, "22_weight_heatmap")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 554 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 555 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 556 `ax.bar(np.arange(NUM_CLASSES), np.linalg.norm(weights, axis=1))`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 557 `ax.set_xticks(range(NUM_CLASSES), CLASS_NAMES)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 558 `ax.set_ylabel("L2 norm")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 559 `ax.set_title("Per-Class Weight Norm")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 560 `save_pdf_page(pdf, fig, png_dir, "23_weight_norms")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 561 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 562 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 563 `ax.bar(np.arange(NUM_CLASSES) - 0.18, bias / 1e9, width=0.36, label="before")`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 564 `ax.bar(np.arange(NUM_CLASSES) + 0.18, after_bias / 1e9, width=0.36, label="after")`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 565 `ax.set_xticks(range(NUM_CLASSES), CLASS_NAMES)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 566 `ax.set_ylabel("bias / 1e9")`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 567 `ax.set_title("Classifier Bias Before/After On-Chip Updates")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 568 `ax.legend()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 569 `save_pdf_page(pdf, fig, png_dir, "24_bias_before_after")`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 570 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 571 `wdiff = after_weights - weights`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 572 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 573 `ax.hist(wdiff.reshape(-1), bins=np.arange(wdiff.min() - 1, wdiff.max() + 2) if wdiff.size and wdiff.max() - wdiff.min() < 200 else 100)`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 574 `ax.set_title("Weight Delta Distribution After Online Learning")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 575 `ax.set_xlabel("after - before")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 576 `save_pdf_page(pdf, fig, png_dir, "25_weight_delta_hist")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 577 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 578 `fig, ax = plt.subplots(figsize=(12, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 579 `im = ax.imshow(wdiff, aspect="auto", cmap="coolwarm")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 580 `ax.set_xlabel("feature index")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 581 `ax.set_ylabel("class")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 582 `ax.set_title("Weight Delta Matrix After On-Chip Updates")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 583 `fig.colorbar(im, ax=ax)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 584 `save_pdf_page(pdf, fig, png_dir, "26_weight_delta_heatmap")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 585 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 586 `fig, ax = plt.subplots(figsize=(11, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 587 `names = ["LUT", "FF", "BRAM36", "DSP", "LUTRAM"]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 588 `vals = [vivado["lut"], vivado["regs"], vivado["bram"], vivado["dsp"], vivado["lut_mem"]]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 589 `ax.bar(names, vals)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 590 `ax.set_title("Vivado Utilization Counts")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 591 `for i, value in enumerate(vals):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 592 `ax.text(i, value, str(value), ha="center", va="bottom")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 593 `save_pdf_page(pdf, fig, png_dir, "27_utilization")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 594 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 595 `fig, ax = plt.subplots(figsize=(8, 6))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 596 `power_names = ["clock", "logic", "signals", "BRAM", "DSP", "IO"]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 597 `power_vals = [vivado["clock_power_w"], vivado["logic_power_w"], vivado["signal_power_w"], vivado["bram_power_w"], vivado["dsp_power_w"], vivado["io_power_w"]]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 598 `ax.pie(power_vals, labels=power_names, autopct="%1.1f%%")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 599 `ax.set_title("Vivado Dynamic Power Breakdown")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 600 `save_pdf_page(pdf, fig, png_dir, "28_power_pie")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 601 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 602 `if activity_power:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 603 `labels_power = ["vectorless", "inference SAIF", "learning SAIF"]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 604 `dynamic_power = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 605 `float(activity_power.get("vectorless", {}).get("dynamic_w", vivado["dynamic_power_w"]) or 0.0),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 606 `float(activity_power.get("inference_saif", {}).get("dynamic_w", 0.0) or 0.0),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 607 `float(activity_power.get("learning_saif", {}).get("dynamic_w", 0.0) or 0.0),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 608 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 609 `total_power = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 610 `float(activity_power.get("vectorless", {}).get("total_on_chip_w", vivado["total_power_w"]) or 0.0),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 611 `float(activity_power.get("inference_saif", {}).get("total_on_chip_w", 0.0) or 0.0),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 612 `float(activity_power.get("learning_saif", {}).get("total_on_chip_w", 0.0) or 0.0),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 613 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 614 `fig, ax = plt.subplots(figsize=(9, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 615 `x = np.arange(len(labels_power))`

This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 616 `width = 0.35`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 617 `ax.bar(x - width / 2, total_power, width, label="total")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 618 `ax.bar(x + width / 2, dynamic_power, width, label="dynamic")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 619 `ax.set_xticks(x)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 620 `ax.set_xticklabels(labels_power)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 621 `ax.set_ylabel("Vivado power (W)")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 622 `ax.set_title("Vectorless Power vs XSim SAIF-Annotated Power")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 623 `ax.grid(axis="y", alpha=0.25)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 624 `ax.legend()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 625 `save_pdf_page(pdf, fig, png_dir, "28b_activity_power")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 626 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 627 `fig, ax = plt.subplots(figsize=(9, 5))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 628 `ax.bar(["WNS", "WHS"], [vivado["wns_ns"], vivado["whs_ns"]])`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 629 `ax.axhline(0, color="black", linewidth=0.8)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 630 `ax.set_ylabel("ns")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 631 `ax.set_title("Routed Timing Slack At 100 MHz")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 632 `save_pdf_page(pdf, fig, png_dir, "29_timing_slack")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 633 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 634 `saved_infer_cycles = 2 * rtl_infer_skip_count`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 635 `saved_train_cycles = 5 * learn_train_skip_count`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 636 `latency_lines = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 637 `"Feature load:  384 accepted feature words per frame through AXI-stream wrapper.",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 638 `"Per class:  CLASS_INIT plus one MAC_READ per feature; active features run MAC_MUL and MAC_ACCUM, while gated features jump directly to CLASS_DONE.",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 639
```
f"Activity gate:  abs(feature) <= {infer_skip_abs_thresh} bypasses DSP multiply/accumulate for inference; measured exported-set skip fraction is {feature_mac_skip_fraction * 100:.2f}%.",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 640
```
f"RTL inference skip counter {rtl_infer_skip_count:,} removes {saved_infer_cycles:,} MAC_MUL/MAC_ACCUM cycles over the 10k fixture.",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 641
```
"Learning update, when applied:  active features update true and predicted class rows; gated features skip the no-op read/calc/write sequence.",
```

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 642
```
f"RTL learning skip counter {learn_train_skip_count:,} removes approximately {saved_train_cycles:,} true/pred update cycles in the full learning run.",
```

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 643
```
f"Learning is margin-gated, so the full 10k validation stream applied {learn_val.get('updates')} updates.",
```

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 644
```
]
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 645
```
save_pdf_page(pdf, plot_text_page("Pipeline / Latency Accounting", latency_lines), png_dir, "30_latency")
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 646
```
blank line
```

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 647
```
print(f"Wrote {pdf_path}")
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 648
```
print(f"Wrote plot PNGs in {png_dir}")
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 649
```
blank line
```

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 650
```
blank line
```

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 651
```
def latex_escape(text:  str) -> str:
```

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 652 `replacements = {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 653 `"\\":  r"\textbackslash{}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 654 `"&":  r"\&",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 655 `"%":  r"\%",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 656 `"$":  r"\$",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 657 `"#":  r"\#",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 658 `"_":  r"\_",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 659 `"{":  r"\{",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 660 `"}":  r"\}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 661 `"~":  r"\textasciitilde{}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 662 `"^":  r"\textasciicircum{}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 663 `}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 664 `return "".join(replacements.get(ch, ch) for ch in text)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 665 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 666 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 667 `def explain_line(path: Path, line: str, index: int, previous_nonblank: str) -> str:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 668 `stripped = line.strip()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 669 `suffix = path.suffix.lower()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 670 `name = path.name`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 671 `if not stripped:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 672 `return "This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 673 `if stripped.startswith("//") or stripped.startswith("#"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 674 `return "This is a human-facing comment. It documents intent for the reader and is ignored by the compiler or interpreter."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 675 `if suffix in {".sv", ".v", ".svh"}:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 676 `if stripped.startswith("`timescale"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 677 `return "This simulator directive sets the time unit and precision used by testbenches and waveform timestamps. It keeps delays such as clock half-periods numerically meaningful in XSim."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 678 `if stripped.startswith("`default_nettype none"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 679 `return "This disables implicit one-bit wires.  It prevents spelling mistakes from silently creating unwanted hardware nets."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 680 `if stripped.startswith("`include"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 681 `return "This includes shared compile-time constants such as feature dimensions and fixed-point widths.  The RTL and testbenches therefore agree on the same trained-model shape."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 682 `if stripped.startswith("module "):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 683 `return "This begins a hardware module, which is the FPGA equivalent of a reusable circuit block.  Everything until the matching endmodule describes ports, storage, combinational equations, and clocked behavior for this block."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 684 `if stripped.startswith("endmodule"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 685 `return "This closes the module definition.  Synthesis stops adding logic to this circuit block at this point."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 686 `if "INFER_SKIP_ABS_THRESH" in stripped:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 687 `return "This line controls the feature-activity gate.  When the absolute fixed-point feature is at or below this threshold, the RTL treats it as zero so the exported Spikingformer head ignores small noisy activations and records a measurable MAC-skip opportunity."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 688 `if "skip_feature" in stripped:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 689 `return "This line is part of the combinational decision that detects low-magnitude features.  It is the hardware equivalent of asking whether a small activation should be silent for this inference/update pass."`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 690 `if "effective_feature" in stripped:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 691
```
return "This line is part of the denoised feature path.  The returned value is either the original signed feature or exactly zero, and that registered value is what feeds both the DSP multiply and the online-learning delta."
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 692
```
if "skip_count" in stripped:
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 693
```
return "This line exposes or updates a hardware activity counter.  The counter makes the feature gate visible in waveforms and CSV traces, so the power-saving opportunity is observable from RTL rather than only from Python statistics."
```

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 694
```
if "parameter" in stripped:
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 695
```
return "This is a compile-time hardware setting.  Vivado resolves it before synthesis, so it controls the generated circuit size, fixed-point width, or learning rule without runtime overhead."
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 696
```
if re.search(r"\binput\b|\boutput\b", stripped):
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 697
```
return "This line declares a module port.  A port is a physical or wrapper-visible signal used to move clocks, control pulses, features, labels, predictions, or debug information across module boundaries."
```

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 698
```
if "typedef enum" in stripped:
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 699
```
return "This declares the finite-state-machine state type.  Using named states makes the cycle-by-cycle control flow explicit instead of relying on anonymous numeric constants."
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 700
```
if stripped.startswith("ST_") or stripped.startswith("WR_"):
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 701
```
return "This names one state in the control FSM. The numeric encoding is what appears in waveform traces, while the name explains the pipeline phase being executed."
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 702
```
if "feature_mem" in stripped:
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 703
```
return "This line touches the feature memory that holds one 384-dimensional fixed-point Spikingformer feature vector for the current image.  Keeping the feature vector local avoids repeatedly streaming the same values for every class score."
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 704 `if "weight_mem" in stripped:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 705 `return "This line touches the trained classifier weight store.  In the final RTL the large 10 x 384 int16 matrix is inferred as true dual-port BRAM, so weights remain on chip and can also be updated by the learning state machine."`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 706 `if "bias_mem" in stripped:`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 707 `return "This line touches the class-bias store.  Bias values are part of the affine classifier score and are updated on chip in separate one-write states to keep synthesis clean."`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 708 `if "class_score_mem" in stripped:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 709 `return "This stores or reads the ten class scores for the current sample.  Those scores are needed for argmax prediction, CSV testbench reporting, and the margin test used before an online update."`

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 710 `if stripped.startswith("assign "):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 711 `return "This is continuous combinational wiring.  The output follows its source expression immediately in hardware rather than waiting for a clock edge."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 712 `if stripped.startswith("initial"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 713 `return "This simulation/synthesis initialization block loads trained memories from generated .mem files.  On FPGA, Vivado maps these initial values into BRAM/LUTRAM initialization data."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 714 `if "$readmemh" in stripped:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 715 `return "This reads a hexadecimal memory file into an RTL array.  It is how exported trained weights and biases become deterministic FPGA memory contents."`

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 716 `if stripped.startswith("function"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 717
```
return "This begins a pure helper function used inside the RTL. It describes fixed combinational math, not a software function running on a CPU."
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 718
```
if "sat_weight" in stripped:
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 719
```
return "This line is part of signed saturation for 16-bit weights.  Saturation prevents online learning from wrapping around when an update would exceed the representable fixed-point range."
```

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 720
```
if "learn_delta" in stripped:
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 721
```
return "This line implements the hardware learning step size.  The feature is sign-extended and arithmetically shifted so positive and negative features update weights symmetrically."
```

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 722
```
if "always_ff" in stripped:
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 723
```
return "This begins clocked sequential logic.  Registers and RAM writes inside this block update only on a clock edge, which is what makes the design synthesizable and waveform-predictable."
```

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 724
```
if "if (!rst_n)" in stripped:
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 725
```
return "This is the active-low reset branch.  It puts control registers and visible outputs into a known starting state before a frame is processed."
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 726
```
if "unique case" in stripped:
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 727
```
return "This starts a state-machine branch table.  The unique qualifier tells tools that one legal state should match at a time, improving diagnostics and optimization."
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 728
```
if "state <=" in stripped:
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 729
```
return "This schedules the next FSM state for the following clock edge.  That transition is the hardware pipeline control path."
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 730 `if "<=" in stripped:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 731 `return "This is a nonblocking register or memory assignment.  The right side is sampled now and the left side updates together with other clocked assignments at the end of the clock edge."`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 732 `if "=" in stripped:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 733 `return "This is a local compile-time or procedural assignment.  In context it either defines a helper value or connects a temporary expression used by nearby hardware logic."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 734 `return "This SystemVerilog line contributes to the structural or sequential description of the accelerator.  Its effect is resolved by XSim for verification and by Vivado during synthesis."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 735 `if suffix == ".py":`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 736 `if stripped.startswith("from ") or stripped.startswith("import "):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 737 `return "This imports a Python library or symbol.  The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 738 `if stripped.startswith("def "):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 739 `return "This defines a Python function.  The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 740 `if "argparse" in stripped or "add_argument" in stripped:`

This contributes to the command-line interface. It lets the rerun flow choose paths, sample counts, or output locations without editing code.

Line 741 `return "This contributes to the command-line interface.  It lets the rerun flow choose paths, sample counts, or output locations without editing code."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 742 `if "Path" in stripped:`

This uses path objects so generated artifacts are addressed consistently across Windows shells and Python code.

Line 743
```
return "This uses path objects so generated artifacts are addressed consistently across Windows shells and Python code."
```
This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 744
```
if "read_text" in stripped or "open(" in stripped:
```
This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 745
```
return "This reads an artifact produced by training, export, simulation, or Vivado.  The validator/report depends on files generated by the actual run rather than hidden constants."
```
This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 746
```
if "write_text" in stripped or "json.dump" in stripped:
```
This writes machine-readable output for later reports. Keeping JSON/CSV artifacts makes the verification auditable.

Line 747
```
return "This writes machine-readable output for later reports.  Keeping JSON/CSV artifacts makes the verification auditable."
```
This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 748
```
if "np." in stripped or "numpy" in stripped:
```
This performs vectorized numerical work. The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable.

Line 749
```
return "This performs vectorized numerical work.  The fixed-point golden models use NumPy so score, margin, and update calculations are explicit and repeatable."
```
This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 750
```
if "weights" in stripped or "bias" in stripped:
```
This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 751
```
return "This line handles the classifier parameters.  Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug."
```
This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 752
```
if "scores" in stripped:
```
This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 753
```
return "This line handles class scores.  The score vector is the direct software equivalent of the RTL MAC loop over weights and features."
```
This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 754
```
if "pred" in stripped:
```
This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 755
```
return "This line handles predicted labels.  It checks that argmax behavior in Python and RTL chooses the same class."
```
This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 756
```
if "update" in stripped:
```
This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 757 `return "This line handles the online-learning rule or its validation fields.  It is central to proving that the FPGA update path is not bypassed."`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 758 `if "plt." in stripped or "PdfPages" in stripped:`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 759 `return "This creates a debug visualization.  The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 760 `if stripped.startswith("if __name__"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 761 `return "This Python idiom runs main() only when the file is executed as a script.  It still allows the file's functions to be imported for tests or reuse."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 762 `return "This Python line participates in the export, validation, or report-generation pipeline.  It is executed on the host PC to prepare data for RTL or to check RTL outputs."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 763 `if suffix == ".ps1":`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 764 `if stripped.startswith("param"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 765 `return "This begins the PowerShell parameter block.  It exposes repeatable knobs such as sample count, Vivado path, board part, and skip switches."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 766 `if stripped.startswith("["):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 767 `return "This declares one typed PowerShell parameter.  Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 768 `if "$ErrorActionPreference" in stripped:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 769 `return "This makes failures stop the script instead of continuing after a broken export, simulation, or synthesis step."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 770 `if "Assert-LastExitCode" in stripped:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 771 `return "This calls the helper that turns a failed external tool exit code into a hard pipeline failure."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 772 `if "Vivado" in stripped or "xvlog" in stripped or "xelab" in stripped or "xsim" in stripped:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 773 `return "This line invokes or locates Vivado simulator/implementation tools.  The flow uses the installed AMD toolchain on D: rather than a substitute simulator."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 774 `if "python" in stripped.lower():`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 775 `return "This line invokes the Python environment used for export, validation, or report generation.  It ties host-side math to the hardware artifacts."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 776 `return "This PowerShell line orchestrates the reproducible full flow.  It exists so changes can be rerun without manually retyping many commands."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 777 `if suffix == ".tcl":`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 778 `if stripped.startswith("read_verilog"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 779 `return "This tells Vivado which SystemVerilog files to synthesize.  The wrapper and accelerator are both read so the routed report matches the PYNQ-facing design."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 780 `if stripped.startswith("synth_design"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 781 `return "This starts synthesis for the selected FPGA part and top module.  Synthesis maps RTL arrays, arithmetic, and FSMs into FPGA primitives."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 782 `if stripped.startswith("create_clock"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 783 `return "This creates the 100 MHz timing constraint.  Vivado uses it to decide whether every register-to-register path is fast enough."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 784 `if stripped.startswith("place_design") or stripped.startswith("route_design") or stripped.startswith("opt_design"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 785 `return "This runs an implementation stage.  These commands turn the synthesized netlist into a placed and routed design with real timing and power estimates."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 786 `if stripped.startswith("report_") or stripped.startswith("write_checkpoint"):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 787 `return "This emits a signoff artifact.  The generated reports are the source for utilization, timing, and power numbers in the debug PDF."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 788 `return "This TCL line controls the Vivado batch implementation flow.  It makes the hardware report reproducible from the command line."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 789 `return "This line is part of the accelerator support files and is included so the full rerun flow is documented, not just the RTL."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 790 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 791 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 792 `def generate_explainer_tex() -> Path:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 793 `pdir = project_dir()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 794 `ddir = debug_dir()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 795 `ddir.mkdir(parents=True, exist_ok=True)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 796 `odir = out_dir()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 797 `rtl_val = read_json(odir / "spikingformer_head_rtl_validation.json")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 798 `learn_val = read_json(odir / "spikingformer_head_onchip_learning_validation.json")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 799 `mapping = read_json(odir / "spikingformer_head_mapping_metrics.json")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 800 `vivado = parse_vivado_reports(odir / "vivado")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 801 `activity_power = read_json(odir / "activity_power" / "spikingformer_head_activity_power_summary.json")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 802 `acc_pct = float(rtl_val.get("accuracy", float("nan"))) * 100.0`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 803 `sample_count = int(rtl_val.get("samples", 0) or 0)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 804 `update_count = int(learn_val.get("updates", 0) or 0)`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 805 `infer_skip_abs_thresh = int(rtl_val.get("infer_skip_abs_thresh", mapping.get("infer_skip_abs_thresh", 0)) or 0)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 806 `feature_mac_skip_fraction = float(rtl_val.get("feature_mac_skip_fraction", mapping.get("feature_mac_skip_fraction", 0.0)) or 0.0) * 100.0`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 807 `files = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 808 `pdir / "rtl" / "cifar10_spikingformer_head_accelerator.sv",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 809 `pdir / "rtl" / "cifar10_spikingformer_head_pynq_axi_stream_top.sv",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 810 `pdir / "rtl" / "tb_spikingformer_head_learning_unit.sv",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 811 `pdir / "rtl" / "tb_cifar10_spikingformer_head_accelerator.sv",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 812 `pdir / "rtl" / "tb_cifar10_spikingformer_head_onchip_learning.sv",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 813 `pdir / "rtl" / "tb_cifar10_spikingformer_head_axi_activity.sv",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 814 `pdir / "export_spikingformer_head_accelerator.py",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 815 `pdir / "validate_spikingformer_head_rtl.py",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 816 `pdir / "validate_spikingformer_head_onchip_learning.py",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 817 `pdir / "generate_spikingformer_debug_reports.py",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 818 `pdir / "summarize_spikingformer_head_activity_power.py",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 819 `pdir / "run_spikingformer_head_accelerator.ps1",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 820 `pdir / "run_spikingformer_head_activity_power.ps1",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 821 `pdir / "spikingformer_head_synth.tcl",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 822 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 823 `tex_path = ddir / "neuraedge_full_accelerator_line_by_line_explainer.tex"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 824 `lines: list[str] = []`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 825 `lines.extend(`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 826 `[`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 827 `r"\documentclass[10pt]{article}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 828 `r"\usepackage[margin=0.65in]{geometry}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 829 `r"\usepackage[T1]{fontenc}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 830 `r"\usepackage{hyperref}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 831 `r"\usepackage{xcolor}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 832 `r"\usepackage{listings}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 833 `r"\lstset{basicstyle=\ttfamily\scriptsize,breaklines=true,columns=fullflexible,keepspaces=true,showstringspaces=false}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 834 `r"\title{NeuraEdge CIFAR-10 Neuromorphic Accelerator: Full Pipeline and Line-by-Line Explainer}",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 835
```
r"\author{Generated from repository source and measured Vivado/XSim artifacts}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 836
```
r"\date{\today}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 837
```
r"\begin{document}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 838
```
r"\maketitle",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 839
```
r"\section{What Was Built}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 840
```
"This document describes the implemented CIFAR-10 accelerator path.  The trained Spikingformer backbone is used to export 384-dimensional
fixed-point features and a trained classifier head.  The FPGA accelerator stores one feature vector, applies a measured low-magnitude
feature activity gate, computes ten fixed-point class scores with an on-chip weight memory and one DSP MAC datapath, returns the predicted
label, and includes a margin-gated on-chip learning path that updates the true-label and predicted-label classifier rows in FPGA memory.",
```

This line handles class scores. The score vector is the direct software equivalent of the RTL MAC loop over weights and features.

Line 841
```
r"\section{Measured Verification Summary}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 842
```
r"\begin{itemize}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 843
```
rf"\item Full inference XSim run:  {sample_count:,} CIFAR-10 samples, {acc_pct:.2f}\% top-1 accuracy, exact score match against the
fixed-point Python reference.",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 844
```
rf"\item Feature activity gate:  abs(feature) $\leq$ {infer_skip_abs_thresh} is treated as zero before inference and learning; the
exported test set skips {feature_mac_skip_fraction:.2f}\% of feature-MAC opportunities.",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 845
```
rf"\item Full on-chip-learning XSim run:  {sample_count:,} samples, {update_count} RTL weight/bias updates, zero score mismatches, zero
prediction mismatches, zero update-flag mismatches, and zero update-count mismatches against the independent Python online-learning golden
model.",
```

This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 846
```
rf"\item Vivado implementation target:  PYNQ Zynq-7020, xc7z020clg400-1, 100 MHz.  Routed timing met with WNS {vivado['wns_ns']:.3f} ns
and WHS {vivado['whs_ns']:.3f} ns.",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 847
```
rf"\item Resource result:  {vivado['lut']} LUTs, {vivado['regs']} FFs, {vivado['bram']} RAMB36E1 blocks, and {vivado['dsp']} DSP48E1.",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 848
```
rf"\item Vectorless Vivado power:  {vivado['total_power_w']:.3f} W total, {vivado['dynamic_power_w']:.3f} W dynamic,
{vivado['static_power_w']:.3f} W static.",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 849
```
rf"\item XSim-SAIF annotated Vivado power:  inference {float(activity_power.get('inference_saif', {}).get('total_on_chip_w',
float('nan'))):.3f} W total / {float(activity_power.get('inference_saif', {}).get('dynamic_w', float('nan'))):.3f} W dynamic; learning
{float(activity_power.get('learning_saif', {}).get('total_on_chip_w', float('nan'))):.3f} W total /
{float(activity_power.get('learning_saif', {}).get('dynamic_w', float('nan'))):.3f} W dynamic.  Vivado reports medium confidence because
SAIF annotates the wrapper/core activity while remaining unmatched nets use probabilistic propagation.",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 850
```
r"\end{itemize}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 851
```
r"\section{Research Context And Comparison}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 852
```
rf"The model side follows the Spikingformer family of spike-driven transformer SNNs, whose reported CIFAR-10 accuracy is about
95.81-95.95\% in the cited paper/repository.  The local fixed-point exported checkpoint reaches {acc_pct:.2f}\% after mapping into this
routed hardware head with the measured feature gate.  Recent SNN-transformer targets to beat include Max-Former at 97.04\% on CIFAR-10 and
alpha-XNOR spiking self-attention at 96.15\% for the reported multi-stage ViT setting.  The implemented hardware contribution here is the
verified FPGA path:  local feature and weight memories, exact fixed-point RTL/golden agreement, activity-gated feature silence, and
on-chip classifier learning.",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 853
```
r"\begin{itemize}",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 854
```
r"\item Spikingformer paper/repository:  \url{https://arxiv.org/html/2304.11954v5}, \url{https://github.com/TheBrainLab/Spikingformer}.",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 855
```
r"\item Max-Former high-frequency spiking transformer reference:  \url{https://arxiv.org/html/2505.18608v4}.",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 856
```
r"\item Alpha-XNOR spiking self-attention reference:
\url{https://openaccess.thecvf.com/content/CVPR2025/papers/Xiao_Rethinking_Spiking_Self-Attention_Mechanism_Implementing_a-XNOR_Similarity
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 857
```
r"\item TrueNorth reference:
\url{https://research.ibm.com/publications/truenorth-design-and-tool-flow-of-a-65-mw-1-million-neuron-programmable-neurosynaptic-chip}.",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 858
```
r"\item Loihi on-chip learning reference:  \url{https://redwood.berkeley.edu/wp-content/uploads/2021/08/Davies2018.pdf}.",
```

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 859
```
r"\end{itemize}",
```
This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 860
```
r"\section{End-to-End Dataflow}",
```
This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 861
```
r"\begin{enumerate}",
```
This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 862
```
r"\item Python loads the trained Spikingformer checkpoint and CIFAR-10 data, runs the trained network, and exports fixed-point feature, label, weight, and bias memories.",
```
This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 863
```
r"\item XSim streams each 384-word feature vector into the RTL through the same handshake the PYNQ AXI-stream wrapper uses.",
```
This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 864
```
r"\item The RTL stores the feature vector locally, zeroes features whose magnitude is below the selected gate threshold, loops over the ten classes, reads each class row from BRAM, multiplies effective feature and weight in a DSP, accumulates a 64-bit score, and chooses argmax.",
```
This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 865
```
r"\item If training is enabled and the prediction is wrong but close enough by margin, the RTL latches the label, reads the true and predicted class rows, computes signed shifted deltas from the same effective feature path, saturates int16 weight updates, writes updated weights, and updates the two class biases.",
```
This line handles the classifier parameters. Those arrays mirror the RTL memories, so mismatches reveal an export, quantization, or hardware-update bug.

Line 866
```
r"\item Python validators independently replay the same fixed-point inference and online-learning rule from the exported memories, then compare every RTL score, prediction, update flag, and update count.",
```
This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 867
```
r"\item Vivado synthesizes and routes the PYNQ AXI-stream top, producing timing, utilization, power, and checkpoint artifacts.",
```
This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 868
```
r"\end{enumerate}",
```
This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 869
```
r"\section{Line-by-Line Source Explanation}",
```
This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 870
```
]
```
This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 871
```
)
```
This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 872 `for path in files:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 873 `rel = path.relative_to(repo_root())`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 874 `lines.append(r"\clearpage")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 875 `lines.append(r"\subsection{" + latex_escape(str(rel)) + "}")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 876 `lines.append("Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 877 `previous = ""`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 878 `for i, raw in enumerate(path.read_text(encoding="utf-8", errors="ignore").splitlines(), start=1):`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 879 `explanation = explain_line(path, raw, i, previous)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 880 `if raw.strip():`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 881 `previous = raw.strip()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 882 `lines.append(r"\paragraph{Line " + str(i) + "}")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 883 `code_text = latex_escape(raw) if raw else r"\emph{blank line}"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 884 `lines.append(`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 885 `r"\noindent\colorbox{gray!8}{\parbox{\dimexpr\linewidth-2\fboxsep\relax}{\raggedright\ttfamily\scriptsize "`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 886 `+ code_text`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 887 `+ r"}}"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 888 `)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 889 `lines.append(r"\par\smallskip")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 890 `lines.append(latex_escape(explanation))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 891 `lines.append(r"\end{document}")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 892 `tex_path.write_text("\n".join(lines), encoding="utf-8")`

This writes machine-readable output for later reports. Keeping JSON/CSV artifacts makes the verification auditable.

Line 893 `return tex_path`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 894 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 895 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 896 `def compile_tex(tex_path:  Path) -> None:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 897 `pdflatex = shutil.which("pdflatex")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 898 `if not pdflatex:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 899 `print("pdflatex not found; wrote TeX only.")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 900 `return`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 901 `for _ in range(2):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 902 `subprocess.run(`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 903 `[pdflatex, "-interaction=nonstopmode", "-halt-on-error", tex_path.name],`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 904 `cwd=tex_path.parent,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 905 `check=True,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 906 `stdout=subprocess.PIPE,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 907 `stderr=subprocess.STDOUT,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 908 `text=True,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 909 `)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 910 `print(f"Wrote {tex_path.with_suffix('.pdf')}")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 911 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 912 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 913 `def main() -> None:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 914 `generate_debug_pdf()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 915 `tex_path = generate_explainer_tex()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 916 `print(f"Wrote {tex_path}")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 917 `compile_tex(tex_path)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 918 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 919 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 920 `if __name__ == "__main__":`

This Python idiom runs main() only when the file is executed as a script. It still allows the file's functions to be imported for tests or reuse.

Line 921 `main()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

5.11 cifar10_neuromorphic\summarize_spikingformer_head_activity_power.py

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `from __future__ import annotations`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 2 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 3 `import json`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 4 `import re`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 5 `from pathlib import Path`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 6 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 7 `import matplotlib.pyplot as plt`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 8 `from matplotlib.backends.backend_pdf import PdfPages`

This imports a Python library or symbol. The scripts use standard parsing, numeric arrays, plotting, and file APIs to make the hardware artifacts reproducible.

Line 9 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 10 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 11 `def repo_root() -> Path:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 12 `return Path(__file__).resolve().parents[1]`

This uses path objects so generated artifacts are addressed consistently across Windows shells and Python code.

Line 13 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 14 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 15 `def parse_power_report(path: Path) -> dict[str, float | str]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 16 `text = path.read_text(errors="ignore")`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 17 `patterns = {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 18 `"total_on_chip_w": r"Total On-Chip Power \(W\)\s*\|\s*([0-9.]+)",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 19 `"dynamic_w": r"Dynamic \(W\)\s*\|\s*([0-9.]+)",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 20 `"device_static_w": r"Device Static \(W\)\s*\|\s*([0-9.]+)",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 21 `"junction_temp_c": r"Junction Temperature \(C\)\s*\|\s*([0-9.]+)",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 22 `}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 23 `parsed: dict[str, float | str] = {"report": str(path)}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 24 `for key, pattern in patterns.items():`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 25 `match = re.search(pattern, text)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 26 `if match:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 27 `parsed[key] = float(match.group(1))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 28 `confidence = re.search(r"Overall confidence level\s*\|\s*([^|]+)\|", text)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 29 `if confidence:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 30 `parsed["confidence"] = confidence.group(1).strip()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 31 `return parsed`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 32 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 33 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 34 `def load_json(path: Path) -> dict[str, object]:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 35 `if not path.exists():`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 36 `return {}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 37 `return json.loads(path.read_text(encoding="utf-8"))`

This reads an artifact produced by training, export, simulation, or Vivado. The validator/report depends on files generated by the actual run rather than hidden constants.

Line 38 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 39 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 40 `def main() -> None:`

This defines a Python function. The function packages one repeatable step such as loading a memory file, simulating the golden learning rule, validating CSVs, or generating plots.

Line 41 `project = repo_root() / "cifar10_neuromorphic"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 42 `out_dir = project / "outputs_spikingformer_head_accelerator"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 43 `activity_dir = out_dir / "activity_power"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 44 `activity_dir.mkdir(parents=True, exist_ok=True)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 45 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 46 `vectorless = parse_power_report(out_dir / "vivado" / "spikingformer_head_power.rpt")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 47 `inference = parse_power_report(activity_dir / "spikingformer_head_power_inference_saif.rpt")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 48 `learning = parse_power_report(activity_dir / "spikingformer_head_power_learning_saif.rpt")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 49 `rtl_val = load_json(out_dir / "spikingformer_head_rtl_validation.json")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 50 `learn_val = load_json(out_dir / "spikingformer_head_onchip_learning_validation.json")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 51 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 52 `summary: dict[str, object] = {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 53 `"note": (`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 54 `"Vivado-estimated FPGA power for the routed Spikingformer-head accelerator. "`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 55 `"Vectorless is Vivado's default activity estimate; inference_saif and "`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 56 `"learning_saif are annotated from XSim SAIF traces of the RTL testbenches."`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 57 `),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 58 `"target": {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 59 `"part": "xc7z020clg400-1",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 60 `"clock_ns": 10.0,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 61 `"top": "cifar10_spikingformer_head_pynq_axi_stream_top",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 62 `"annotated_instance": "u_head",`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 63 `},`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 64 `"validation": {`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 65 `"samples": rtl_val.get("samples"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 66 `"accuracy": rtl_val.get("accuracy"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 67 `"score_mismatch_count": rtl_val.get("score_mismatch_count"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 68 `"prediction_mismatch_count": rtl_val.get("prediction_mismatch_count"),`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 69 `"onchip_learning_updates": learn_val.get("updates"),`

This line handles the online-learning rule or its validation fields. It is central to proving that the FPGA update path is not bypassed.

Line 70 `"onchip_learning_score_mismatch_count": learn_val.get("score_mismatch_count"),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 71 `"onchip_learning_prediction_mismatch_count": learn_val.get("prediction_mismatch_count"),`

This line handles predicted labels. It checks that argmax behavior in Python and RTL chooses the same class.

Line 72 `},`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 73 `"vectorless": vectorless,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 74 `"inference_saif": inference,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 75 `"learning_saif": learning,`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 76 `}`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 77 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 78 `for name, data in [("inference_saif", inference), ("learning_saif", learning)]:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 79 `dyn = data.get("dynamic_w")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 80 `base_dyn = vectorless.get("dynamic_w")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 81 `total = data.get("total_on_chip_w")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 82 `base_total = vectorless.get("total_on_chip_w")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 83 `if isinstance(dyn, float) and isinstance(base_dyn, float) and base_dyn > 0:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 84 `summary[f"{name}_dynamic_vs_vectorless"] = dyn / base_dyn`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 85 `if isinstance(total, float) and isinstance(base_total, float) and base_total > 0:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 86 `summary[f"{name}_total_vs_vectorless"] = total / base_total`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 87 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 88 `summary_path = activity_dir / "spikingformer_head_activity_power_summary.json"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 89 `summary_path.write_text(json.dumps(summary, indent=2), encoding="utf-8")`

This writes machine-readable output for later reports. Keeping JSON/CSV artifacts makes the verification auditable.

Line 90 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 91 `cases = [`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 92 `("vectorless", vectorless),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 93 `("inference_saif", inference),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 94 `("learning_saif", learning),`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 95 `]`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 96 `labels = []`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 97 `dynamic = []`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 98 `static = []`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 99 `for name, data in cases:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 100 `if isinstance(data.get("dynamic_w"), float):`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 101 `labels.append(name)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 102 `dynamic.append(float(data.get("dynamic_w", 0.0)))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 103 `static.append(float(data.get("device_static_w", 0.0)))`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 104 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 105 `if labels:`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 106 `fig, ax = plt.subplots(figsize=(8.5, 4.8))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 107 `ax.bar(labels, dynamic, label="dynamic")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 108 `ax.bar(labels, static, bottom=dynamic, label="device static")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 109 `ax.set_ylabel("Vivado power (W)")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 110 `ax.set_title("Spikingformer Head Power:  Vectorless vs XSim SAIF")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 111 `ax.grid(axis="y", alpha=0.25)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 112 `ax.legend()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 113 `fig.tight_layout()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 114 `plot_path = activity_dir / "spikingformer_head_activity_power.png"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 115 `fig.savefig(plot_path, dpi=180)`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 116 `plt.close(fig)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 117 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 118 `pdf_path = activity_dir / "spikingformer_head_activity_power_report.pdf"`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 119 `with PdfPages(pdf_path) as pdf:`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 120 `img = plt.imread(plot_path)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 121 `fig = plt.figure(figsize=(10, 7))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 122 `plt.imshow(img)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 123 `plt.axis("off")`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 124 `pdf.savefig(fig, bbox_inches="tight")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 125 `plt.close(fig)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 126 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 127 `fig = plt.figure(figsize=(10, 7))`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 128 `fig.text(0.06, 0.93, "Spikingformer Head Activity Power Summary", fontsize=16, weight="bold")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 129 `fig.text(0.06, 0.85, json.dumps(summary, indent=2), fontsize=7.7, family="monospace", va="top")`

This writes machine-readable output for later reports. Keeping JSON/CSV artifacts makes the verification auditable.

Line 130 `pdf.savefig(fig, bbox_inches="tight")`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

Line 131 `plt.close(fig)`

This creates a debug visualization. The plot is derived from generated CSV or Vivado report data so it can be regenerated after RTL changes.

Line 132 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 133 `print(json.dumps(summary, indent=2))`

This writes machine-readable output for later reports. Keeping JSON/CSV artifacts makes the verification auditable.

Line 134 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 135 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 136 `if __name__ == "__main__":`

This Python idiom runs main() only when the file is executed as a script. It still allows the file's functions to be imported for tests or reuse.

Line 137 `main()`

This Python line participates in the export, validation, or report-generation pipeline. It is executed on the host PC to prepare data for RTL or to check RTL outputs.

5.12 cifar10_neuromorphic\run_spikingformer_head_accelerator.ps1

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `param(`

This begins the PowerShell parameter block. It exposes repeatable knobs such as sample count, Vivado path, board part, and skip switches.

Line 2 `[int]$Samples = 10000,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 3 `[int]$BatchSize = 64,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 4 `[int]$Workers = 4,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 5 `[int]$TraceSamples = 8,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 6 `[int]$InferSkipAbsThresh = 2048,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 7 `[string]$Checkpoint = "cifar10_neuromorphic\outputs_upstream_spikingformer\spikingformer_4_384_400e\model_best.pth.tar",`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 8 `[string]$OutDir = "cifar10_neuromorphic\outputs_spikingformer_head_accelerator",`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 9 `[string]$VivadoBin = "D:\AMDDesignTools\2026.1\Vivado\bin",`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 10 `[string]$Part = "xc7z020clg400-1",`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 11 `[double]$ClockPeriodNs = 10.0,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 12 `[switch]$SkipExport,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 13 `[switch]$SkipVivado,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 14 `[switch]$SkipOnChipLearning,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 15 `[switch]$SkipSynthesis`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 16 `)`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 17 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 18 `$ErrorActionPreference = "Stop"`

This makes failures stop the script instead of continuing after a broken export, simulation, or synthesis step.

Line 19 `$ScriptBoundParameters = @{} + $PSBoundParameters`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 20 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 21 `function Use-PowerShell7 {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 22 `param([hashtable]$BoundParameters)`

This begins the PowerShell parameter block. It exposes repeatable knobs such as sample count, Vivado path, board part, and skip switches.

Line 23 `if ($PSVersionTable.PSEdition -eq "Core" -and $PSVersionTable.PSVersion.Major -ge 7) { return }`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 24 `$pwsh = (Get-Command pwsh -CommandType Application -ErrorAction SilentlyContinue).Source`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 25 `if (!$pwsh) { throw "PowerShell 7+ (pwsh) is required.  Install it with:  winget install -id Microsoft.PowerShell -source winget" }`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 26 `$argList = @("-NoLogo", "-NoProfile", "-ExecutionPolicy", "Bypass", "-File", $PSCommandPath)`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 27 `foreach ($key in $BoundParameters.Keys) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 28 `$value = $BoundParameters[$key]`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 29 `if ($value -is [System.Management.Automation.SwitchParameter]) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 30 `if ($value.IsPresent) { $argList += "-$key" }`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 31 `} elseif ($null -ne $value) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 32 `$argList += "-$key"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 33 `$argList += "$value"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 34 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 35 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 36 `& $pwsh @argList`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 37 `exit $LASTEXITCODE`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 38 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 39 `Use-PowerShell7 -BoundParameters $ScriptBoundParameters`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 40 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 41 `$ProjectDir = Split-Path -Parent $MyInvocation.MyCommand.Path`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 42 `$RepoRoot = Resolve-Path (Join-Path $ProjectDir "..")`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 43 `$Python = Join-Path $RepoRoot "external_research\spikingformer_py310\Scripts\python.exe"`

This line invokes the Python environment used for export, validation, or report generation. It ties host-side math to the hardware artifacts.

Line 44 `$TorchLib = Join-Path $RepoRoot "external_research\spikingformer_py310\Lib\site-packages\torch\lib"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 45 `$VivadoXvlog = Join-Path $VivadoBin "xvlog.bat"`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 46 `$VivadoXelab = Join-Path $VivadoBin "xelab.bat"`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 47 `$VivadoXsim = Join-Path $VivadoBin "xsim.bat"`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 48 `$Vivado = Join-Path $VivadoBin "vivado.bat"`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 49 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 50 `if (!(Test-Path $Python)) { throw "Missing Python env:  $Python" }`

This line invokes the Python environment used for export, validation, or report generation. It ties host-side math to the hardware artifacts.

Line 51 `if (Test-Path $TorchLib) { $env:PATH = "$TorchLib;$env:PATH" }`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 52 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 53 `function Assert-LastExitCode {`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 54 `param([string]$Step)`

This begins the PowerShell parameter block. It exposes repeatable knobs such as sample count, Vivado path, board part, and skip switches.

Line 55 `if ($LASTEXITCODE -ne 0) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 56 `throw "$Step failed with exit code $LASTEXITCODE"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 57 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 58 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 59 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 60 `Push-Location $RepoRoot`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 61 `try {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 62 `if (!$SkipExport) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 63 `Write-Host "[export] trained Spikingformer checkpoint to fixed-point accelerator memories"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 64 `& $Python "cifar10_neuromorphic\export_spikingformer_head_accelerator.py" ‘`

This line invokes the Python environment used for export, validation, or report generation. It ties host-side math to the hardware artifacts.

Line 65 `-checkpoint "$Checkpoint" ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 66 `-samples "$Samples" ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 67 `-batch-size "$BatchSize" ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 68 `-workers "$Workers" ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 69 `-trace-samples "$TraceSamples" ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 70 `-infer-skip-abs-thresh "$InferSkipAbsThresh" ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 71 `-out-dir "$OutDir"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 72 `Assert-LastExitCode "Spikingformer head export"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 73 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 74 `} finally {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 75 `Pop-Location`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 76 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 77 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 78 `if (!$SkipVivado) {`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 79 `if (!(Test-Path $VivadoXvlog)) { throw "Vivado xvlog not found at $VivadoXvlog" }`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 80 `if (!(Test-Path $VivadoXelab)) { throw "Vivado xelab not found at $VivadoXelab" }`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 81 `if (!(Test-Path $VivadoXsim)) { throw "Vivado xsim not found at $VivadoXsim" }`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 82 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 83 `Push-Location $ProjectDir`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 84 `try {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 85 `Write-Host "[vivado] compiling trained Spikingformer head accelerator"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 86 `& $VivadoXvlog -sv ‘`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 87 `rtl\cifar10_spikingformer_head_accelerator.sv ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 88 `rtl\tb_cifar10_spikingformer_head_accelerator.sv`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 89 `Assert-LastExitCode "Vivado xvlog"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 90 `& $VivadoXelab tb_cifar10_spikingformer_head_accelerator -debug typical`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 91 `Assert-LastExitCode "Vivado xelab"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 92 `& $VivadoXsim tb_cifar10_spikingformer_head_accelerator -runall`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 93 `Assert-LastExitCode "Vivado xsim"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 94 `} finally {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 95 `Pop-Location`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 96 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 97 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 98 `Push-Location $RepoRoot`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 99 `try {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 100 `Write-Host "[validate] checking RTL scores against fixed-point reference"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 101 `& $Python "cifar10_neuromorphic\validate_spikingformer_head_rtl.py" -out-dir "$OutDir"`

This line invokes the Python environment used for export, validation, or report generation. It ties host-side math to the hardware artifacts.

Line 102 `Assert-LastExitCode "Spikingformer head RTL validation"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 103 `} finally {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 104 `Pop-Location`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 105 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 106 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 107 `if (!$SkipOnChipLearning) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 108 `Push-Location $ProjectDir`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 109 `try {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 110 `Write-Host "[vivado] verifying on-chip learning update datapath"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 111 `& $VivadoXvlog -sv ‘`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 112 `rtl\cifar10_spikingformer_head_accelerator.sv ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 113 `rtl\tb_spikingformer_head_learning_unit.sv`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 114 `Assert-LastExitCode "Vivado xvlog on-chip learning unit"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 115 `& $VivadoXelab tb_spikingformer_head_learning_unit -debug typical`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 116 `Assert-LastExitCode "Vivado xelab on-chip learning unit"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 117 `& $VivadoXsim tb_spikingformer_head_learning_unit -runall`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 118 `Assert-LastExitCode "Vivado xsim on-chip learning unit"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 119 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 120 `Write-Host "[vivado] running full CIFAR-10 on-chip learning simulation"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 121 `& $VivadoXvlog -sv ‘`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 122 `rtl\cifar10_spikingformer_head_accelerator.sv ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 123 `rtl\tb_cifar10_spikingformer_head_onchip_learning.sv`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 124 `Assert-LastExitCode "Vivado xvlog on-chip learning full"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 125 `& $VivadoXelab tb_cifar10_spikingformer_head_onchip_learning -debug typical`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 126 `Assert-LastExitCode "Vivado xelab on-chip learning full"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 127 `& $VivadoXsim tb_cifar10_spikingformer_head_onchip_learning -runall`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 128 `Assert-LastExitCode "Vivado xsim on-chip learning full"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 129 `} finally {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 130 `Pop-Location`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 131 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 132 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 133 `Push-Location $RepoRoot`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 134 `try {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 135 `Write-Host "[validate] checking RTL on-chip learning against golden online update model"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 136 `& $Python "cifar10_neuromorphic\validate_spikingformer_head_onchip_learning.py" -out-dir "$OutDir"`

This line invokes the Python environment used for export, validation, or report generation. It ties host-side math to the hardware artifacts.

Line 137 `Assert-LastExitCode "Spikingformer head on-chip learning validation"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 138 `} finally {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 139 `Pop-Location`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 140 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 141 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 142 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 143 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 144 `if (!$SkipSynthesis) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 145 `if (!(Test-Path $Vivado)) { throw "Vivado not found at $Vivado" }`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 146 `$SynthOut = Join-Path (Resolve-Path $OutDir) "vivado"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 147 `New-Item -ItemType Directory -Force -Path $SynthOut | Out-Null`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 148 `$env:NEURAEDGE_PART = $Part`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 149 `$env:NEURAEDGE_CLOCK_NS = "$ClockPeriodNs"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 150 `$env:NEURAEDGE_OUT_DIR = "$SynthOut"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 151 `Push-Location $ProjectDir`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 152 `try {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 153 `Write-Host "[synth] routing trained Spikingformer head accelerator for $Part at ${ClockPeriodNs}ns"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 154 `& $Vivado -mode batch -source "spikingformer_head_synth.tcl"`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 155 `Assert-LastExitCode "Spikingformer head synthesis/implementation"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 156 `} finally {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 157 `Pop-Location`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 158 `Remove-Item Env:\NEURAEDGE_PART -ErrorAction SilentlyContinue`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 159 `Remove-Item Env:\NEURAEDGE_CLOCK_NS -ErrorAction SilentlyContinue`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 160 `Remove-Item Env:\NEURAEDGE_OUT_DIR -ErrorAction SilentlyContinue`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 161 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 162 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 163 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 164 `Write-Host "[done] trained Spikingformer head accelerator flow completed"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 165 `Write-Host " outputs:  $OutDir"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 166 `Write-Host " rtl csv:  cifar10_neuromorphic\cifar10_spikingformer_head_results.csv"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

5.13 cifar10_neuromorphic\run_spikingformer_head_activity_power.ps1

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `param(`

This begins the PowerShell parameter block. It exposes repeatable knobs such as sample count, Vivado path, board part, and skip switches.

Line 2 `[string]$VivadoBin = "D:\AMDDesignTools\2026.1\Vivado\bin",`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 3 `[string]$OutDir = "cifar10_neuromorphic\outputs_spikingformer_head_accelerator",`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 4 `[string]$Part = "xc7z020clg400-1",`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 5 `[double]$ClockPeriodNs = 10.0,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 6 `[switch]$SkipInferenceSaif,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 7 `[switch]$SkipLearningSaif,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 8 `[switch]$SkipPowerReports`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 9 `)`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 10 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 11 `$ErrorActionPreference = "Stop"`

This makes failures stop the script instead of continuing after a broken export, simulation, or synthesis step.

Line 12 `$ScriptBoundParameters = @{} + $PSBoundParameters`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 13 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 14 `function Use-PowerShell7 {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 15 `param([hashtable]$BoundParameters)`

This begins the PowerShell parameter block. It exposes repeatable knobs such as sample count, Vivado path, board part, and skip switches.

Line 16 `if ($PSVersionTable.PSEdition -eq "Core" -and $PSVersionTable.PSVersion.Major -ge 7) { return }`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 17 `$pwsh = (Get-Command pwsh -CommandType Application -ErrorAction SilentlyContinue).Source`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 18 `if (!$pwsh) { throw "PowerShell 7+ (pwsh) is required.  Install it with:  winget install -id Microsoft.PowerShell -source winget" }`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 19 `$argList = @("-NoLogo", "-NoProfile", "-ExecutionPolicy", "Bypass", "-File", $PSCommandPath)`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 20 `foreach ($key in $BoundParameters.Keys) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 21 `$value = $BoundParameters[$key]`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 22 `if ($value -is [System.Management.Automation.SwitchParameter]) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 23 `if ($value.IsPresent) { $argList += "-$key" }`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 24 `} elseif ($null -ne $value) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 25 `$argList += "-$key"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 26 `$argList += "$value"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 27 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 28 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 29 `& $pwsh @argList`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 30 `exit $LASTEXITCODE`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 31 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 32 `Use-PowerShell7 -BoundParameters $ScriptBoundParameters`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 33 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 34 `$ProjectDir = Split-Path -Parent $MyInvocation.MyCommand.Path`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 35 `$RepoRoot = Resolve-Path (Join-Path $ProjectDir "..")`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 36 `$Python = Join-Path $RepoRoot "external_research\spikingformer_py310\Scripts\python.exe"`

This line invokes the Python environment used for export, validation, or report generation. It ties host-side math to the hardware artifacts.

Line 37 `$TorchLib = Join-Path $RepoRoot "external_research\spikingformer_py310\Lib\site-packages\torch\lib"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 38 `$VivadoXvlog = Join-Path $VivadoBin "xvlog.bat"`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 39 `$VivadoXelab = Join-Path $VivadoBin "xelab.bat"`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 40 `$VivadoXsim = Join-Path $VivadoBin "xsim.bat"`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 41 `$Vivado = Join-Path $VivadoBin "vivado.bat"`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 42 `$ResolvedOut = Resolve-Path (Join-Path $RepoRoot $OutDir)`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 43 `$ActivityDir = Join-Path $ResolvedOut "activity_power"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 44 `$RoutedDcp = Join-Path $ResolvedOut "vivado\spikingformer_head_routed.dcp"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 45 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 46 `if (!(Test-Path $Python)) { throw "Missing Python env:  $Python" }`

This line invokes the Python environment used for export, validation, or report generation. It ties host-side math to the hardware artifacts.

Line 47 `if (Test-Path $TorchLib) { $env:PATH = "$TorchLib;$env:PATH" }`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 48 `foreach ($tool in @($VivadoXvlog, $VivadoXelab, $VivadoXsim, $Vivado)) {`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 49 `if (!(Test-Path $tool)) { throw "Missing Vivado tool:  $tool" }`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 50 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 51 `if (!(Test-Path $RoutedDcp)) { throw "Missing routed checkpoint:  $RoutedDcp.  Run run_spikingformer_head_accelerator.ps1 first." }`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 52 `New-Item -ItemType Directory -Force -Path $ActivityDir | Out-Null`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 53 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 54 `function Assert-LastExitCode {`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 55 `param([string]$Step)`

This begins the PowerShell parameter block. It exposes repeatable knobs such as sample count, Vivado path, board part, and skip switches.

Line 56 `if ($LASTEXITCODE -ne 0) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 57 `throw "$Step failed with exit code $LASTEXITCODE"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 58 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 59 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 60 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 61 `function Write-XsimSaifTcl {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 62 `param(`

This begins the PowerShell parameter block. It exposes repeatable knobs such as sample count, Vivado path, board part, and skip switches.

Line 63 `[string]$Path,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 64 `[string]$SaifPath,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 65 `[string]$Scope`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 66 `)`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 67 `@(`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 68 `('open_saif {{{0}}}' -f $SaifPath.Replace('\', '/')),`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 69 `('log_saif [get_objects -r {0}/*]' -f $Scope),`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 70 `"run all",`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 71 `"close_saif",`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 72 `"quit"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 73 `) | Set-Content -LiteralPath $Path -Encoding ASCII`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 74 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 75 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 76 `function Run-SaifCase {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 77 `param(`

This begins the PowerShell parameter block. It exposes repeatable knobs such as sample count, Vivado path, board part, and skip switches.

Line 78 `[string]$Name,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 79 `[string]$Top,`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 80 `[string]$TbFile`

This declares one typed PowerShell parameter. Strong typing catches accidental string/numeric mistakes before the long training or Vivado flow starts.

Line 81 `)`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 82 `$saifPath = Join-Path $ActivityDir "spikingformer_head_${Name}.saif"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 83 `$tclName = "xsim_spikingformer_head_${Name}_saif.tcl"`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 84 `$tclPath = Join-Path $ProjectDir $tclName`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 85 `Write-XsimSaifTcl -Path $tclPath -SaifPath $saifPath -Scope "/$Top/base/dut"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 86 `Push-Location $ProjectDir`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 87 `try {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 88 `Write-Host "[saif] compiling $Name"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 89 `& $VivadoXvlog -sv ‘`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 90 `rtl\cifar10_spikingformer_head_accelerator.sv ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 91 `rtl\cifar10_spikingformer_head_pynq_axi_stream_top.sv ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 92 `$TbFile`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 93 `Assert-LastExitCode "xvlog $Name"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 94 `& $VivadoXelab $Top -debug typical`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 95 `Assert-LastExitCode "xelab $Name"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 96 `Write-Host "[saif] running $Name full XSim SAIF"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 97 `& $VivadoXsim $Top -tclbatch $tclName`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 98 `Assert-LastExitCode "xsim $Name"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 99 `} finally {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 100 `Pop-Location`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 101 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 102 `if (!(Test-Path $saifPath)) { throw "Expected SAIF was not produced:  $saifPath" }`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 103 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 104 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 105 `if (!$SkipInferenceSaif) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 106 `Run-SaifCase ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 107 `-Name "inference" ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 108 `-Top "tb_cifar10_spikingformer_head_axi_inference_activity" ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 109 `-TbFile "rtl\tb_cifar10_spikingformer_head_axi_activity.sv"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 110 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 111 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 112 `if (!$SkipLearningSaif) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 113 `Run-SaifCase ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 114 `-Name "learning" ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 115 `-Top "tb_cifar10_spikingformer_head_axi_learning_activity" ‘`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 116 `-TbFile "rtl\tb_cifar10_spikingformer_head_axi_activity.sv"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 117 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 118 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 119 `if (!$SkipPowerReports) {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 120 `$powerTcl = Join-Path $ActivityDir "spikingformer_head_activity_power.tcl"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 121 `@(`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 122 `('open_checkpoint {{{0}}}' -f $RoutedDcp.Replace('\', '/')),`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 123 `('if {{[llength [get_clocks ap_clk -quiet]] == 0}} {{ create_clock -period {0} -name ap_clk [get_ports ap_clk] }}' -f $ClockPeriodNs),`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 124 `('set inference_saif {{{0}}}' -f (Join-Path $ActivityDir "spikingformer_head_inference.saif").Replace('\', '/')),`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 125 `('set learning_saif {{{0}}}' -f (Join-Path $ActivityDir "spikingformer_head_learning.saif").Replace('\', '/')),`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 126 `('set inference_report {{{0}}}' -f (Join-Path $ActivityDir "spikingformer_head_power_inference_saif.rpt").Replace('\', '/')),`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 127 `('set learning_report {{{0}}}' -f (Join-Path $ActivityDir "spikingformer_head_power_learning_saif.rpt").Replace('\', '/')),`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 128 `'if {[file exists $inference_saif]} {',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 129 `' reset_switching_activity [get_nets -hierarchical *]',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 130 `' read_saif -strip_path tb_cifar10_spikingformer_head_axi_inference_activity/base/dut $inference_saif',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 131 `' report_power -file $inference_report -hier all',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 132 `'} else {',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 133 `' puts "Missing inference SAIF: $inference_saif"',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 134 `'}',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 135 `'if {[file exists $learning_saif]} {',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 136 `' reset_switching_activity [get_nets -hierarchical *]',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 137 `' read_saif -strip_path tb_cifar10_spikingformer_head_axi_learning_activity/base/dut $learning_saif',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 138 `' report_power -file $learning_report -hier all',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 139 `'} else {',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 140 `' puts "Missing learning SAIF: $learning_saif"',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 141 `'}',`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 142 `"exit"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 143 `) | Set-Content -LiteralPath $powerTcl -Encoding ASCII`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 144 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 145 `Write-Host "[power] reporting routed power with XSim SAIF activity"`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 146 `& $Vivado -mode batch -source $powerTcl -notrace`

This line invokes or locates Vivado simulator/implementation tools. The flow uses the installed AMD toolchain on D: rather than a substitute simulator.

Line 147 `Assert-LastExitCode "Vivado SAIF power report"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 148 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 149 `blank line`

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 150 `Push-Location $RepoRoot`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 151 `try {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 152 `& $Python "cifar10_neuromorphic\summarize_spikingformer_head_activity_power.py"`

This line invokes the Python environment used for export, validation, or report generation. It ties host-side math to the hardware artifacts.

Line 153 `Assert-LastExitCode "activity power summary"`

This calls the helper that turns a failed external tool exit code into a hard pipeline failure.

Line 154 `} finally {`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 155 `Pop-Location`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

Line 156 `}`

This PowerShell line orchestrates the reproducible full flow. It exists so changes can be rerun without manually retyping many commands.

5.14 cifar10_neuromorphic\spikingformer_head_synth.tcl

Each entry below gives the exact source line followed by its hardware/software role in the implemented accelerator pipeline.

Line 1 `set project_dir [file normalize [file dirname [info script]]]`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 2 `set repo_root [file normalize [file join $project_dir ".."]]`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 3 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 4 `if {![info exists ::env(NEURAEDGE_PART)]} {`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 5 `set part_name "xc7z020clg400-1"`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 6 `} else {`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 7 `set part_name $::env(NEURAEDGE_PART)`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 8 `}`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 9 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 10 `if {![info exists ::env(NEURAEDGE_CLOCK_NS)]} {`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 11 `set clock_ns 10.0`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 12 `} else {`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 13 `set clock_ns $::env(NEURAEDGE_CLOCK_NS)`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 14 `}`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 15 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 16 `if {![info exists ::env(NEURAEDGE_OUT_DIR)]} {`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 17 `set out_dir [file join $project_dir "outputs_spikingformer_head_accelerator" "vivado"]`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 18 `} else {`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 19 `set out_dir [file normalize $::env(NEURAEDGE_OUT_DIR)]`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 20 `}`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 21 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 22 `file mkdir $out_dir`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 23 `cd $project_dir`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 24 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 25 `read_verilog -sv [file join $project_dir "rtl" "cifar10_spikingformer_head_accelerator.sv"]`

This tells Vivado which SystemVerilog files to synthesize. The wrapper and accelerator are both read so the routed report matches the PYNQ-facing design.

Line 26 `read_verilog -sv [file join $project_dir "rtl" "cifar10_spikingformer_head_pynq_axi_stream_top.sv"]`

This tells Vivado which SystemVerilog files to synthesize. The wrapper and accelerator are both read so the routed report matches the PYNQ-facing design.

Line 27 `synth_design -top cifar10_spikingformer_head_pynq_axi_stream_top -part $part_name`

This starts synthesis for the selected FPGA part and top module. Synthesis maps RTL arrays, arithmetic, and FSMs into FPGA primitives.

Line 28 `create_clock -period $clock_ns -name ap_clk [get_ports ap_clk]`

This creates the 100 MHz timing constraint. Vivado uses it to decide whether every register-to-register path is fast enough.

Line 29 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 30 `opt_design`

This runs an implementation stage. These commands turn the synthesized netlist into a placed and routed design with real timing and power estimates.

Line 31 `place_design`

This runs an implementation stage. These commands turn the synthesized netlist into a placed and routed design with real timing and power estimates.

Line 32 `route_design`

This runs an implementation stage. These commands turn the synthesized netlist into a placed and routed design with real timing and power estimates.

Line 33 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 34 `report_utilization -file [file join $out_dir "spikingformer_head_utilization.rpt"]`

This emits a signoff artifact. The generated reports are the source for utilization, timing, and power numbers in the debug PDF.

Line 35 `report_timing_summary -file [file join $out_dir "spikingformer_head_timing_summary.rpt"]`

This emits a signoff artifact. The generated reports are the source for utilization, timing, and power numbers in the debug PDF.

Line 36 `report_power -file [file join $out_dir "spikingformer_head_power.rpt"]`

This emits a signoff artifact. The generated reports are the source for utilization, timing, and power numbers in the debug PDF.

Line 37 `write_checkpoint -force [file join $out_dir "spikingformer_head_routed.dcp"]`

This emits a signoff artifact. The generated reports are the source for utilization, timing, and power numbers in the debug PDF.

Line 38 *blank line*

This blank line is not hardware or software behavior. It separates nearby declarations or operations so the file remains readable and reviewable.

Line 39 `set timing_ok [expr {[get_property SLACK [get_timing_paths -max_paths 1 -setup]] >= 0.0}]`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 40 `set fh [open [file join $out_dir "spikingformer_head_synth_summary.txt"] "w"]`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 41 `puts $fh "top=cifar10_spikingformer_head_pynq_axi_stream_top"`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 42 `puts $fh "part=$part_name"`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 43 `puts $fh "clock_ns=$clock_ns"`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 44 `puts $fh "timing_met=$timing_ok"`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.

Line 45 `close $fh`

This TCL line controls the Vivado batch implementation flow. It makes the hardware report reproducible from the command line.